머신러닝 · 트리 앙상블

의사결정트리와 앙상블

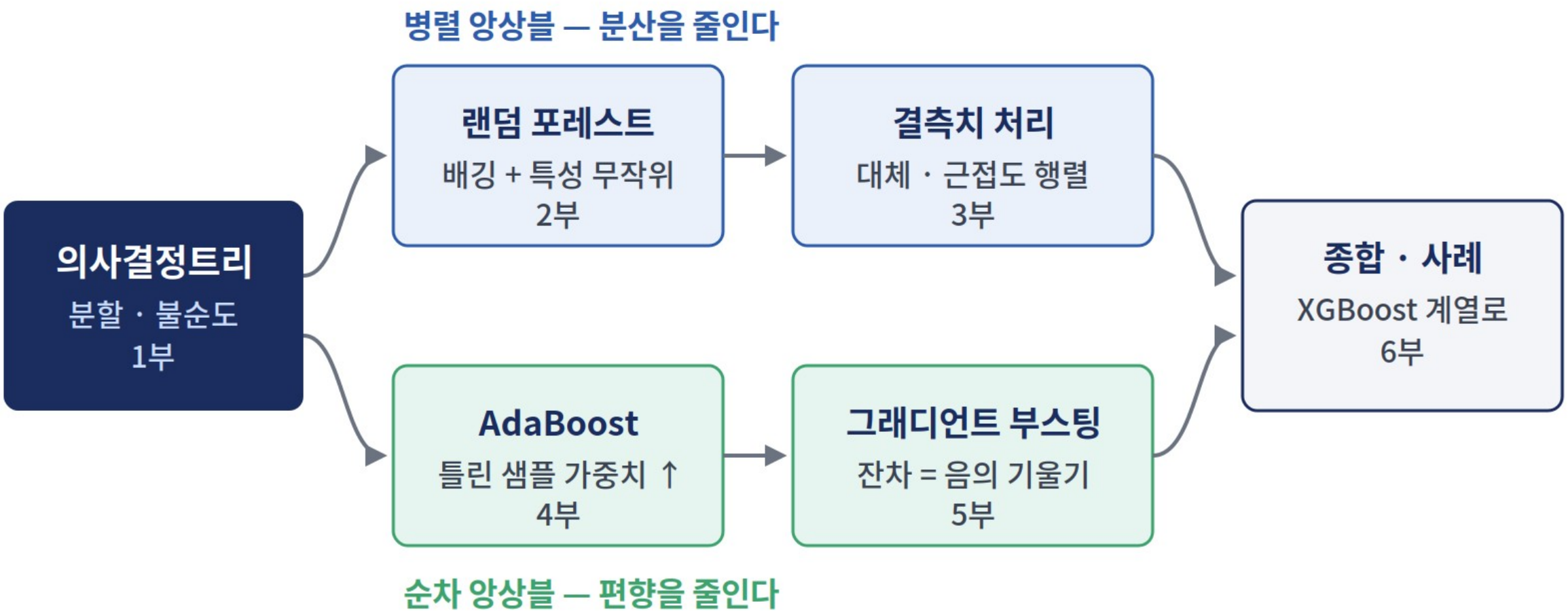
Decision Tree → Random Forest → AdaBoost → Gradient Boosting

분할과 불순도 · 배깅과 결측치 · 부스팅과 잔차 학습

강의자료 5종(의사결정트리 · RF 결측치 · AdaBoost · 그래디언트 부스팅 2종)을 한 흐름으로 재정렬하고,
알고리즘 수식 · scikit-learn 최신 동작 · 금융 사례를 보강했다.

오늘의 지도

트리 하나에서 출발해 병렬(배깅)과 순차(부스팅) 두 갈래로 발전한다



오늘 가져갈 세 가지 메시지

1

트리는 "불순도를 가장 많이 줄이는 질문"을 탐욕적으로 고른다

Gini · 엔트로피로 분할을 평가한다. 깊게 키우면 훈련 데이터를 외워 과대적합된다.

2

배깅(RF)은 분산을, 부스팅은 편향을 줄인다

RF는 서로 다른 트리를 평균해 흔들림을 줄이고, 부스팅은 앞 모델의 실수(가중치·잔차)를 이어서 고친다.

3

그래디언트 부스팅의 잔차 = 손실함수의 음의 기울기

이 한 줄이 AdaBoost와 GBM을 잇고, XGBoost · LightGBM · CatBoost로 가는 다리가 된다.

원 강의 예제(심장병 · 몸무게)는 살리고, 순서는 트리 → RF → 결측치 → 부스팅으로

기본 지식 ① 지도학습 · 분류와 회귀 · 손실함수

지도학습 (supervised learning)

- 입력 X(특성, feature)와 정답 y(타깃, label)의 쌍으로 f(X) ≈ y 를 학습
- 학습에 쓴 데이터 = 훈련 세트, 성능 확인용 = 테스트 세트

분류 vs 회귀

- 분류: y가 범주 — 심장병 Yes/No, 부도 여부
- 회귀: y가 연속값 — 몸무게, 다음 달 수익률

손실함수 (loss function) — 모델이 "얼마나 틀렸나"를 재는 자

- 회귀는 주로 제곱오차 $(y - \hat{y})^2$, 분류는 오분류율 · 로그 손실을 쓴다
- 트리는 분할 기준으로 불순도(Gini · 엔트로피 · MSE)를, 부스팅은 손실의 기울기를 쓴다 → 5부의 핵심

기본 지식 ② 과대적합과 편향–분산

모델 오차는 편향², 분산, 줄일 수 없는 잡음의 합으로 나뉜다

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \underbrace{\text{Bias}^2}_{\text{too simple}} + \underbrace{\text{Var}}_{\text{too sensitive}} + \underbrace{\sigma_\varepsilon^2}_{\text{noise}}$$

편향(bias)이 큰 모델

- 너무 단순해 패턴을 못 잡는다 (과소적합)
- 예: 깊이 1짜리 트리(stump)
- 부스팅이 겨냥하는 오차

분산(variance)이 큰 모델

- 훈련 데이터가 조금만 바뀌어도 예측이 크게 흔들린다 (과대적합)
- 예: 끝까지 키운 깊은 트리
- 배깅 · RF가 겨냥하는 오차

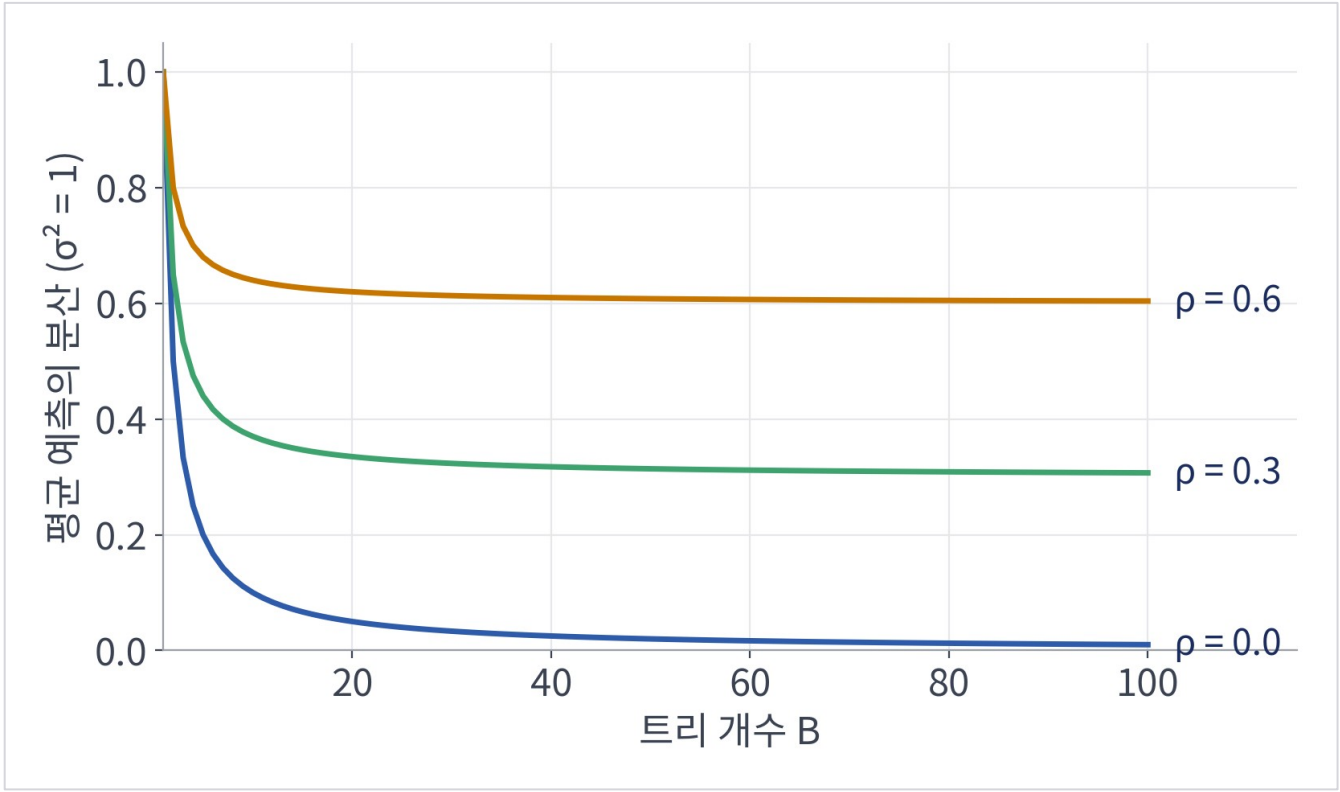
기본 지식 ③ 앙상블은 왜 작동하는가

B개 모델을 평균하면 분산이 줄지만, 모델 간 상관 ρ만큼은 남는다

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(x)\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2$$

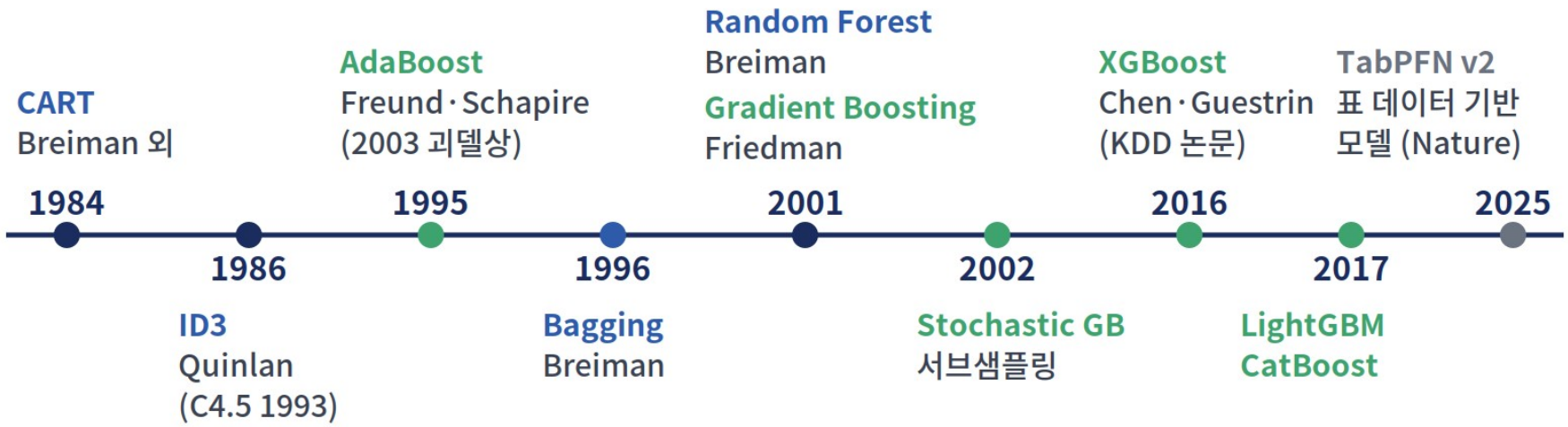
읽는 법

- ρ = 0 이면 B를 늘릴수록 분산 → 0
- ρ가 크면 아무리 많이 평균해도 첫 항이 남는다
- 그래서 RF는 특성까지 무작위로 뽑아 트리 간 상관을 낮춘다 (2부)



에피소드 — 트리 앙상블 40년 계보

CART(1984)에서 XGBoost(2016)까지: 오늘 다루는 알고리즘이 나온 순서



Breiman은 배깅과 RF를, Friedman은 GBM을 만들었다. 둘은 CART의 공동 저자다. AdaBoost 논문은 2003년 이론전산학의 괴델상을 받았다.

01

1부 · 의사결정트리

의사결정트리 — 스무고개로 데이터를 나눈다

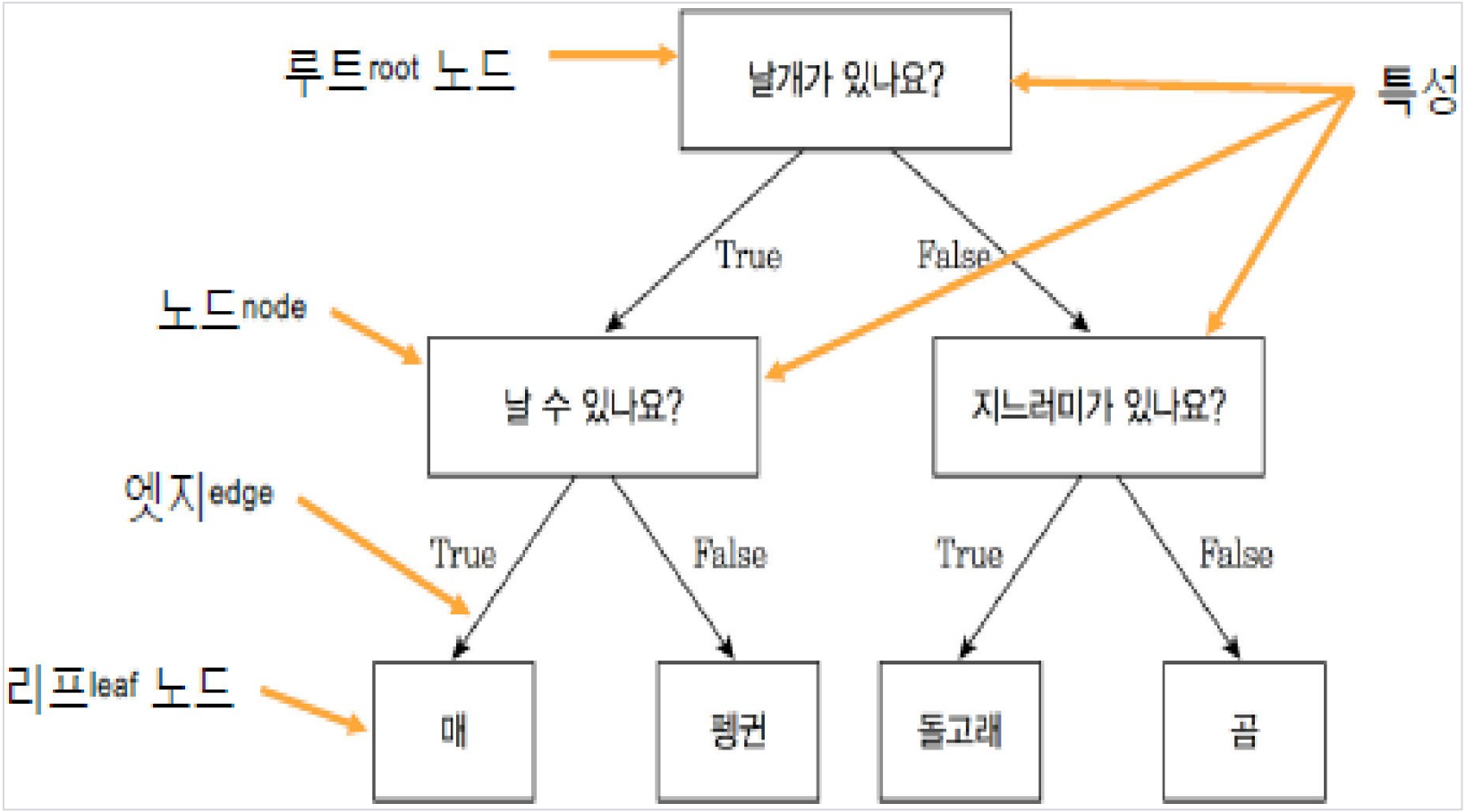
구조 · 불순도 · 정보이득 · 심장병 예제 · 수치/순서/범주형 특성 · 과대적합 제어

결정 트리 = 스무고개

질문(특성) 하나로 데이터를 둘로 나누는 일을 반복한다

용어 정리

- 루트(root) 노드: 전체 데이터에 던지는 첫 질문
- 노드(node): 중간 질문
- 엣지(edge): True/False 답에 따른 가지
- 리프(leaf) 노드: 더 나누지 않는 끝 — 여기서 예측값이 나온다

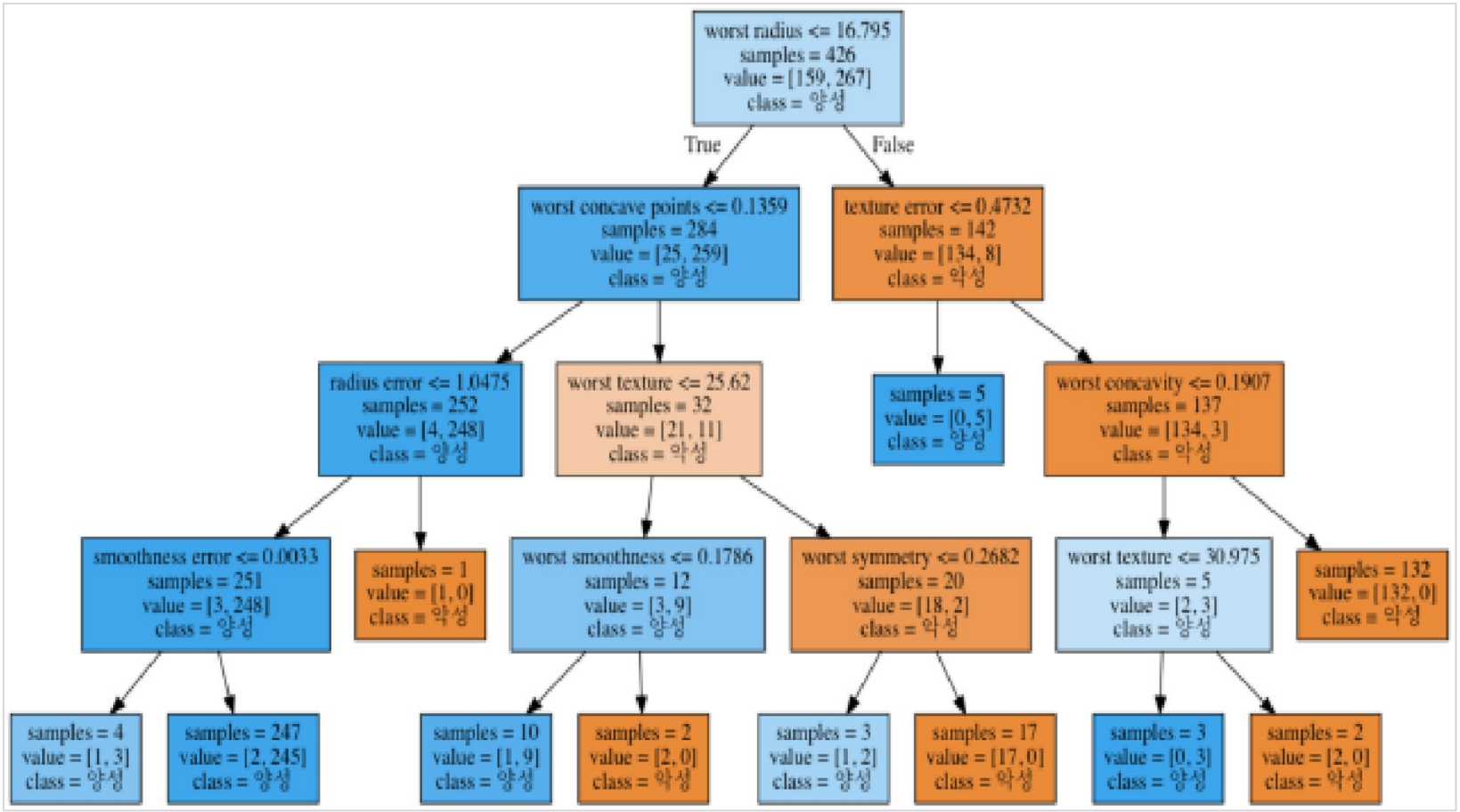


결정 트리 분석 — 유방암 데이터

scikit-learn load_breast_cancer: 569개 샘플 · 30개 특성 · 악성/양성

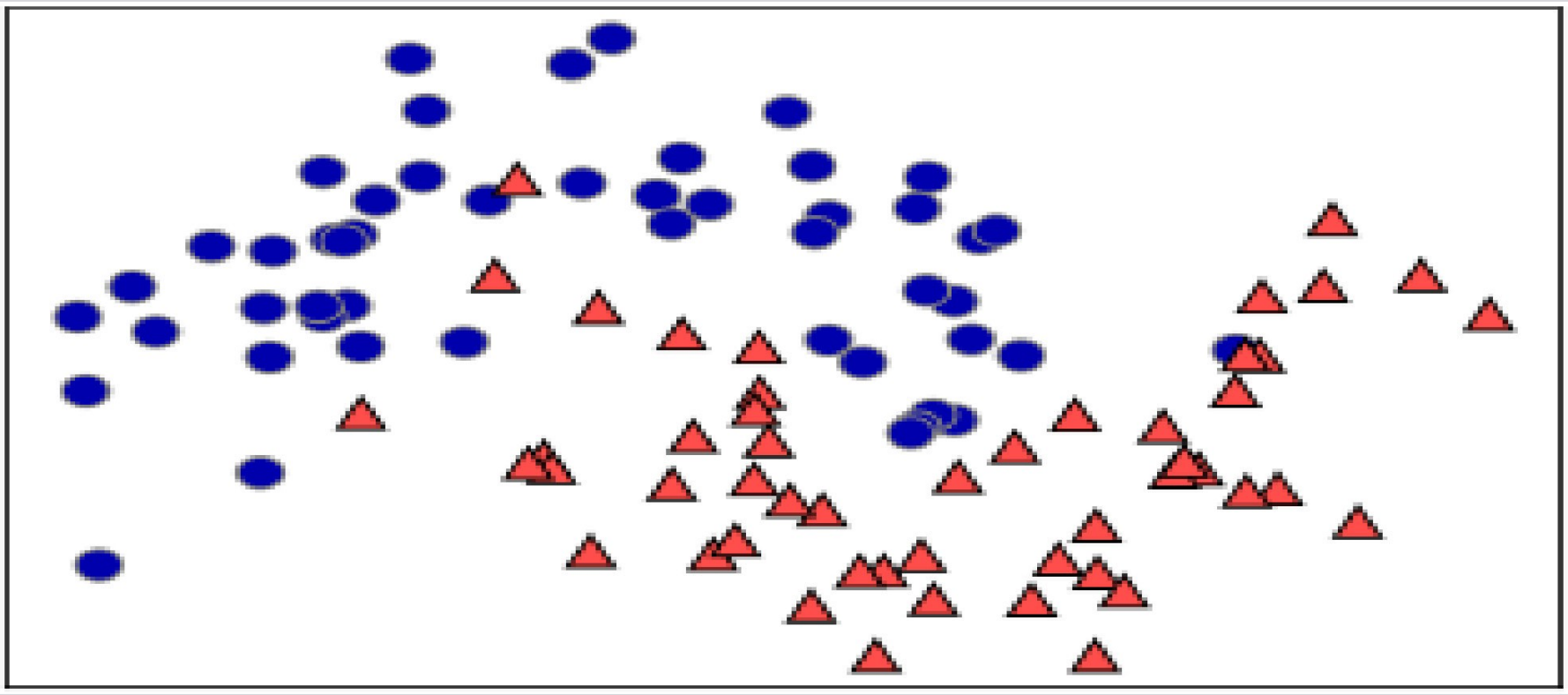
노드 읽는 법

- samples: 이 노드에 도달한 샘플 수 (루트 = 훈련 세트 426개)
- value: [악성, 양성] 개수
- class: 다수 클래스 = 이 노드의 예측
- 색이 진할수록 한 클래스로 순수하다



결정 트리 만들기 — two_moons 데이터

각 노드는 "특성 i가 a보다 큰가?" 같은 yes/no 테스트(test)를 학습한다



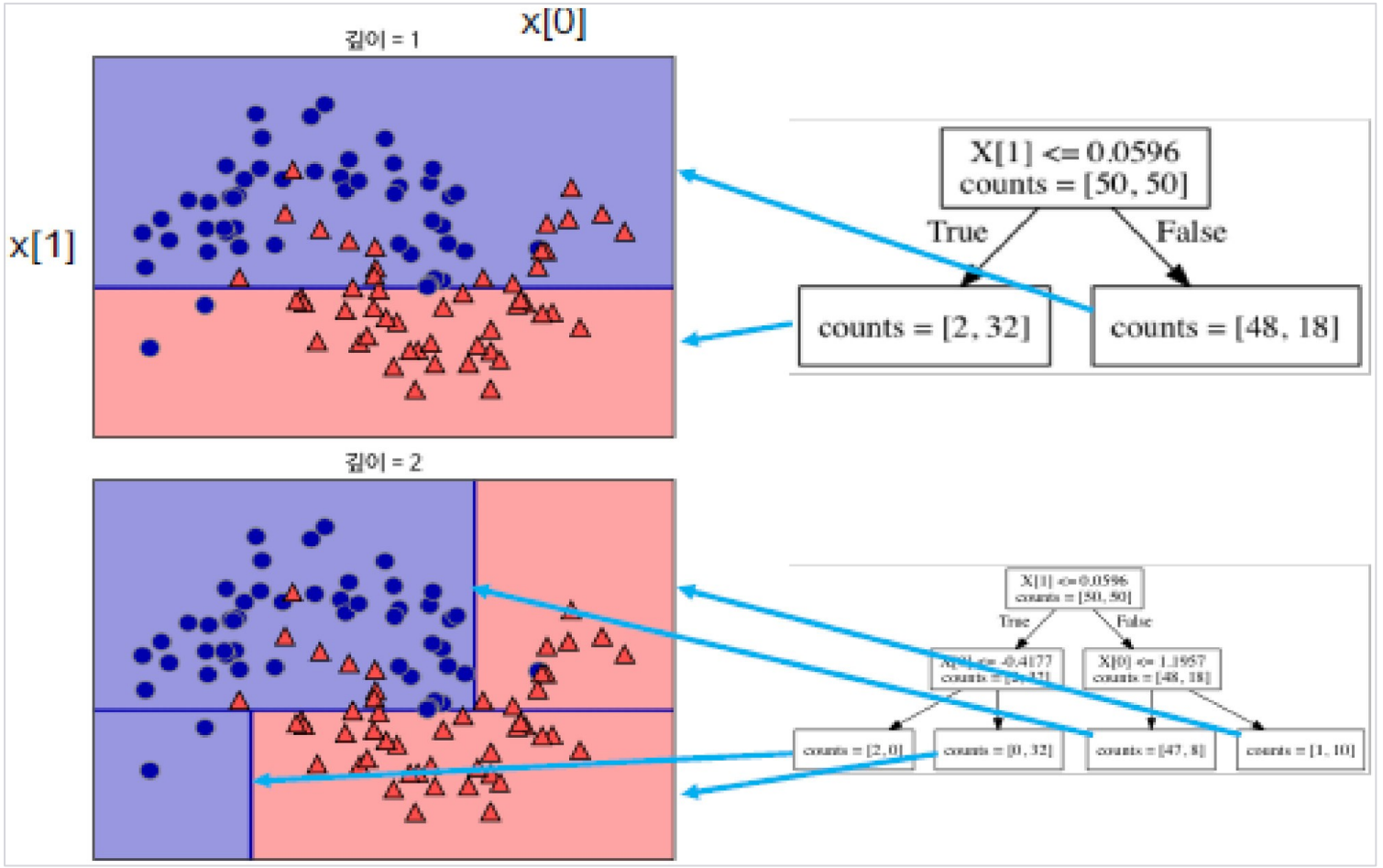
연속형 특성에서는 모든 임계값 a 후보를 비교해 두 그룹을 가장 잘 가르는 값을 고른다.

결정 트리 훈련 — 깊이 1 → 깊이 2

분할할 때마다 영역이 한 번 더 쪼개진다

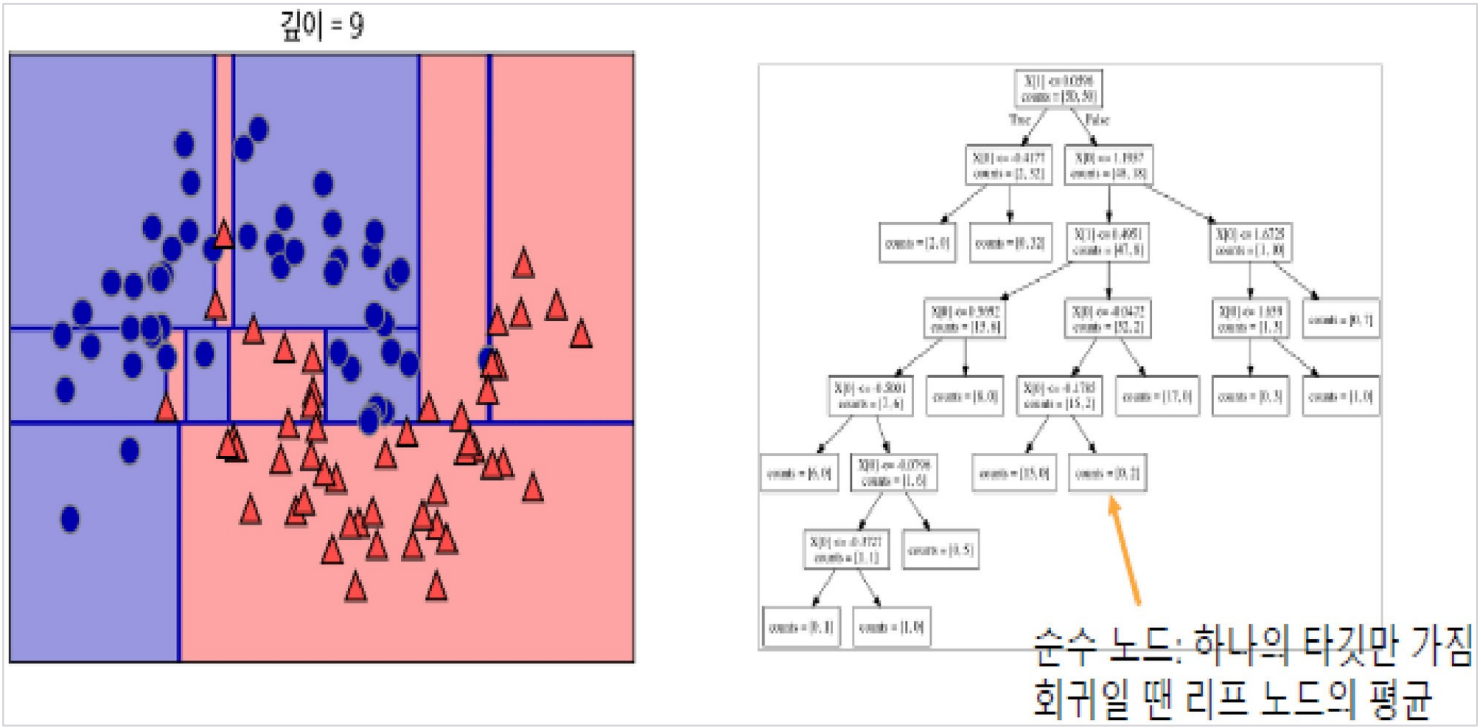
무슨 일이 일어나나

- 깊이 1: X[1] ≤ 0.0596 하나로 [50, 50] → [2, 32] / [48, 18]
- 깊이 2: 두 영역을 각각 다시 분할
- 매번 불순도를 가장 크게 줄이는 (특성, 임계값)을 고른다 — 탐욕적 (greedy) 탐색



결정 트리 예측

테스트가 한 번에 한 특성만 쓰므로 결정 경계는 축에 평행한 계단 모양이다



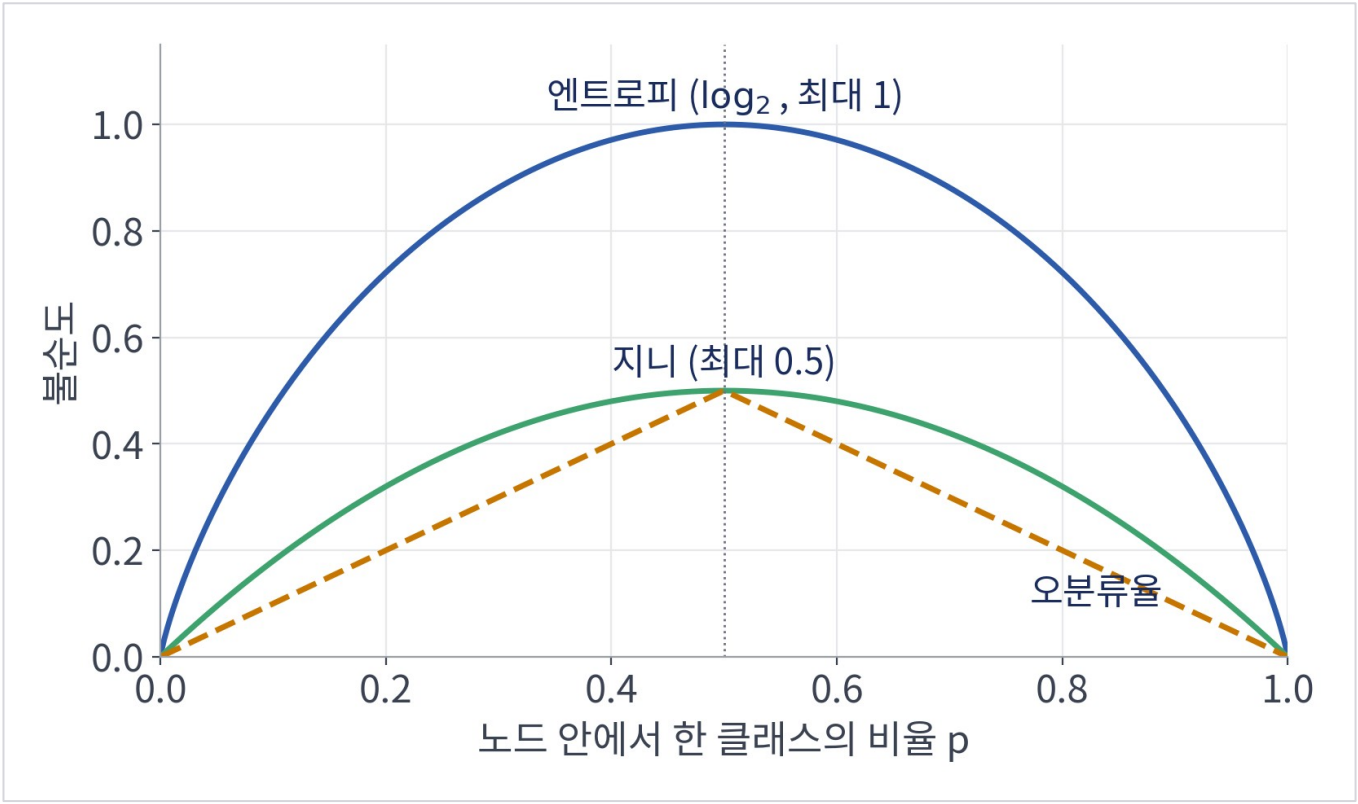
새 샘플은 루트부터 질문을 따라 내려가 도착한 리프의 다수 클래스(분류) 또는 평균값(회귀)을 예측값으로 받는다.

불확실성 척도 — 엔트로피와 지니 불순도

노드 안 클래스 비율 $p_1, \ldots, p_n$ (합 = 1)으로 "섞인 정도"를 잰다

$$\text{Entropy}(t) = -\sum_{i=1}^{n} p_i \log_2 p_i$$

$$\text{Gini}(t) = 1 - \sum_{i=1}^{n} p_i^2$$



짚고 갈 점

- 순수 노드면 둘 다 0
- 로그 밑(2 또는 e)은 분할 순위에 영향 없음
- scikit-learn 기본: criterion="gini"

정보 이득 (Information Gain)

정보 이득 = 분할로 얻는 불확실성 감소의 기대값

$$\mathrm{IG} = H(\text{parent}) - \sum_k \frac{N_k}{N} H(\text{child}_k)$$

원 강의 예제

- 직업 제의를 받을 확률 20%
- 관련 학위 보유 확률 50%
- 학위 있으면 30%, 없으면 10%로 바뀐다
- 질문: "학위 보유" 정보의 가치는?

$$H_0 = -\left[0.2 \log_2 0.2 + 0.8 \log_2 0.8\right] = 0.7219$$

$$\bar{H} = 0.5\, H(0.1) + 0.5\, H(0.3) = 0.5(0.4690) + 0.5(0.8813) = 0.6751$$

$$\mathrm{IG} = 0.7219 - 0.6751 = 0.0468$$

원자료 수치를 다시 계산해 확인했다 ($\log_2$ 기준). 학위 정보는 불확실성을 약 0.047비트 줄인다.

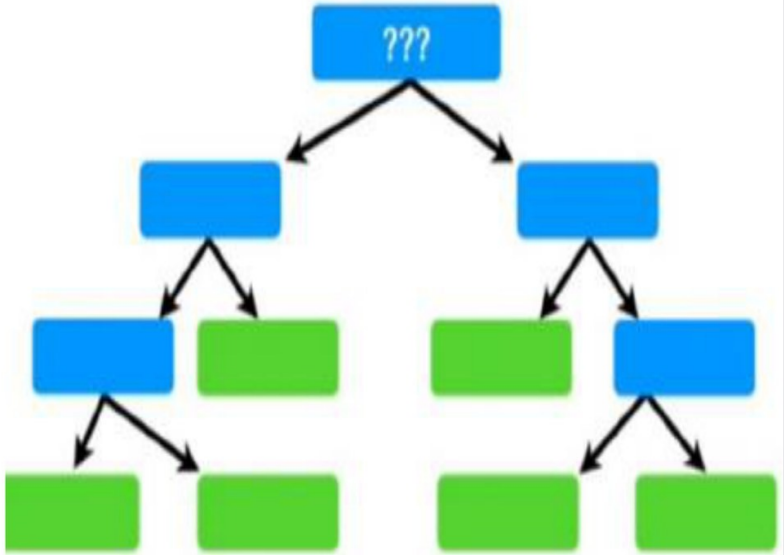
심장병 예제 — 어떤 질문을 루트에 둘까

특성 3개: 흉부통(Chest Pain) · 혈액순환(Good Blood Circulation) · 동맥경화(Blocked Arteries)

과제

- 타깃: 심장병(Heart Disease) Yes/No
- 루트 후보는 CP · BC · BA 셋 중 하나
- "???"는 결측치 — 3부에서 다룬다
- 그림: StatQuest(Josh Starmer) 강의 슬라이드 인용

Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...



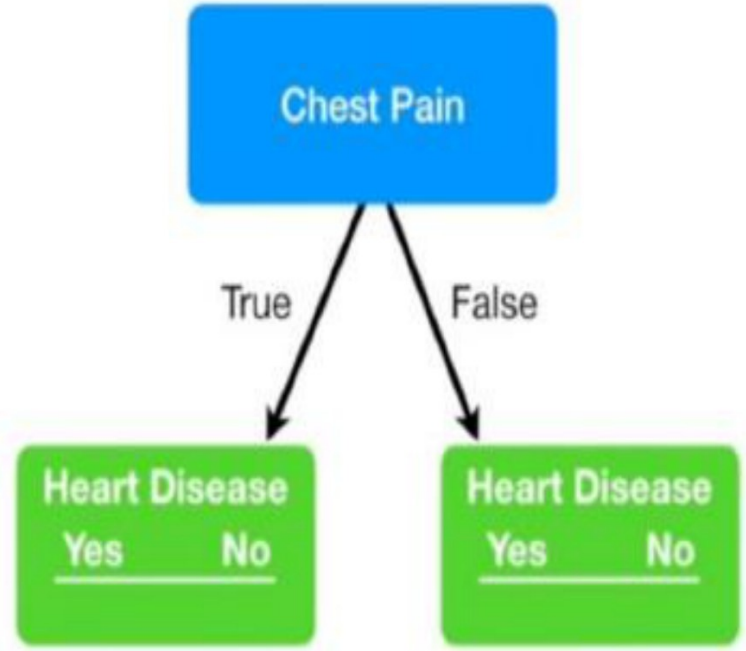
루트 후보 ① 흉부통(CP)으로 나눠 보기

CP의 True/False에 따라 각 환자의 심장병 여부를 리프에 기록한다

절차

- CP 열과 타깃 열만 본다
- CP = Yes → 왼쪽 리프, No → 오른쪽 리프
- 각 리프에 심장병 Yes/No 개수를 센다

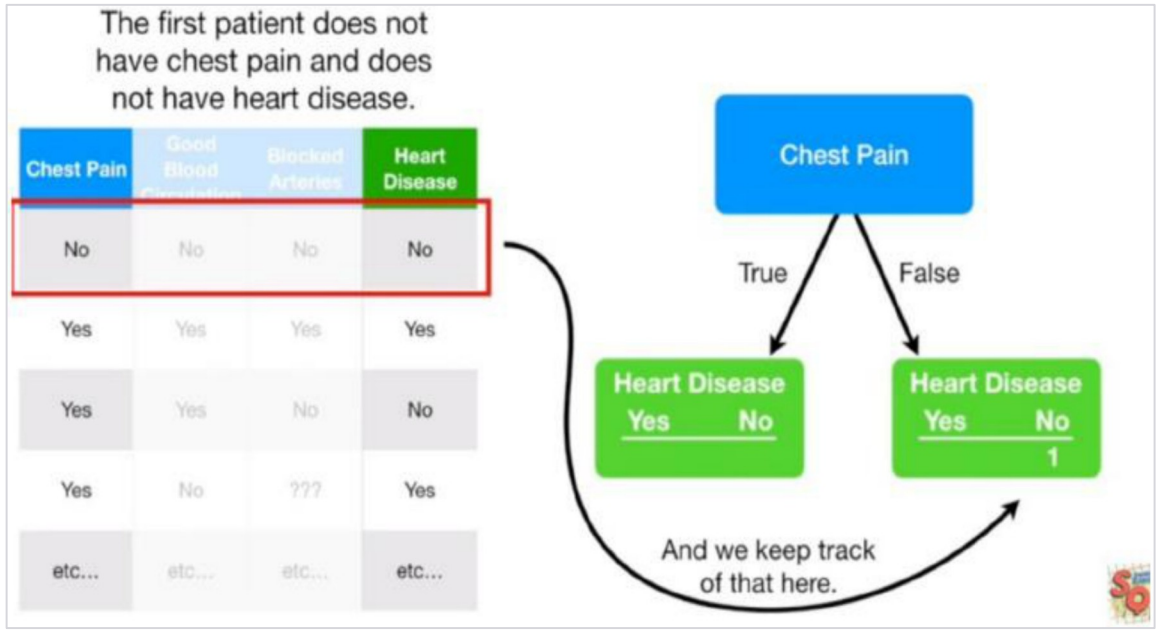
Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...



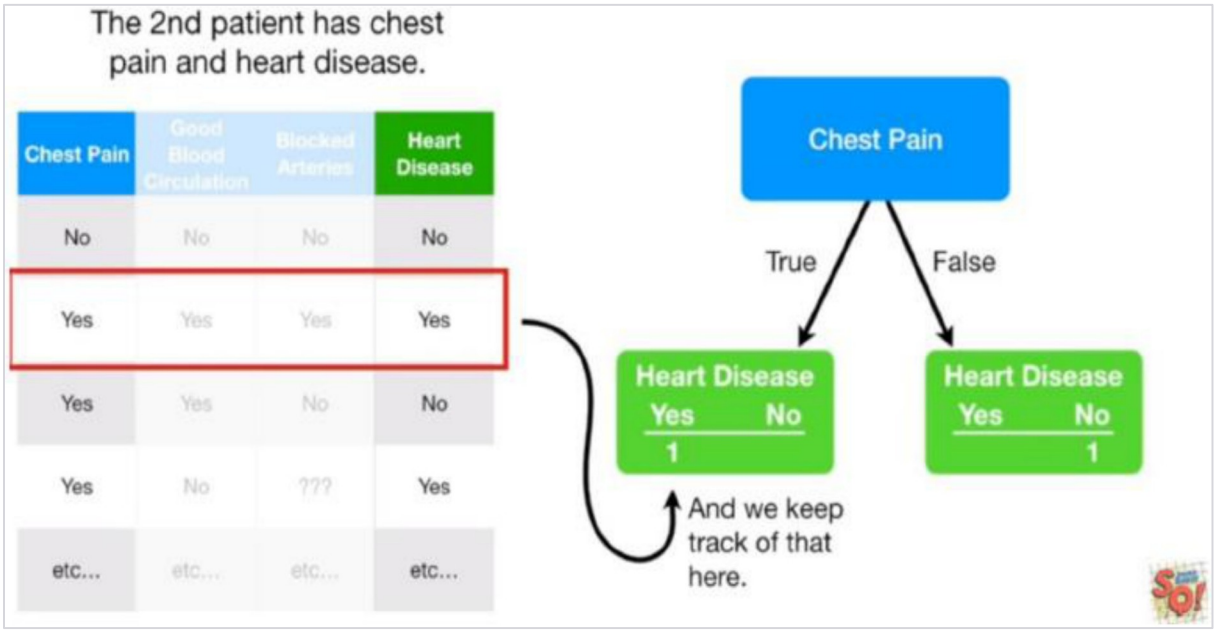
환자 한 명씩 리프에 기록한다 (1)

1번: CP 없음 · 심장병 없음 → 오른쪽 리프 No / 2번: CP 있음 · 심장병 있음 → 왼쪽 리프 Yes

환자 1



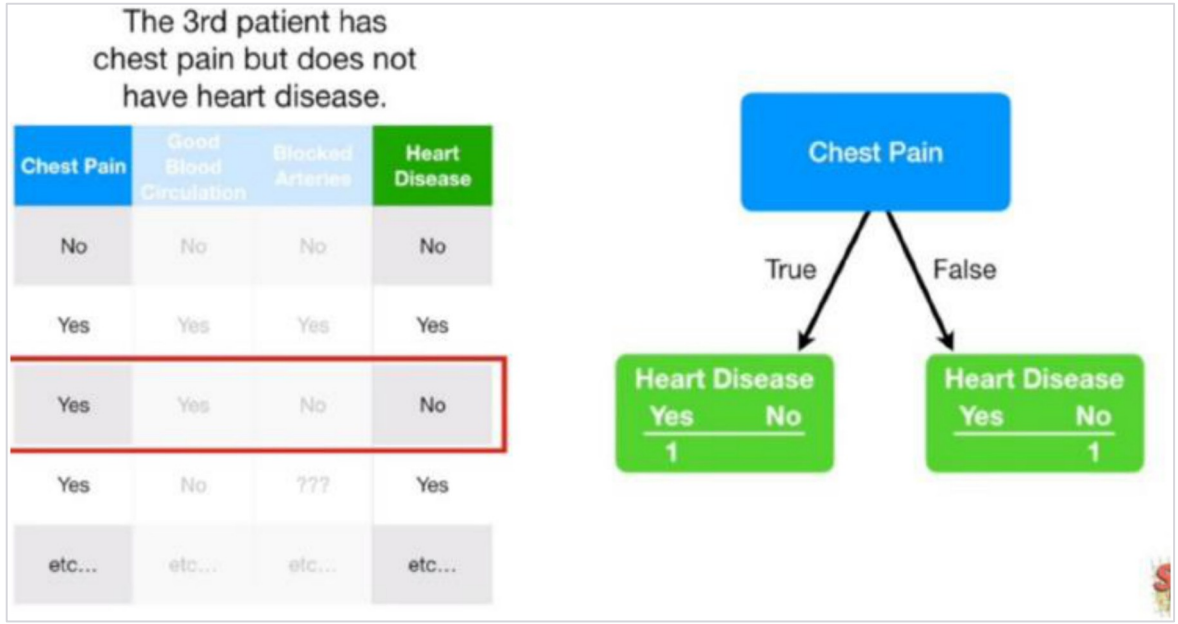
환자 2



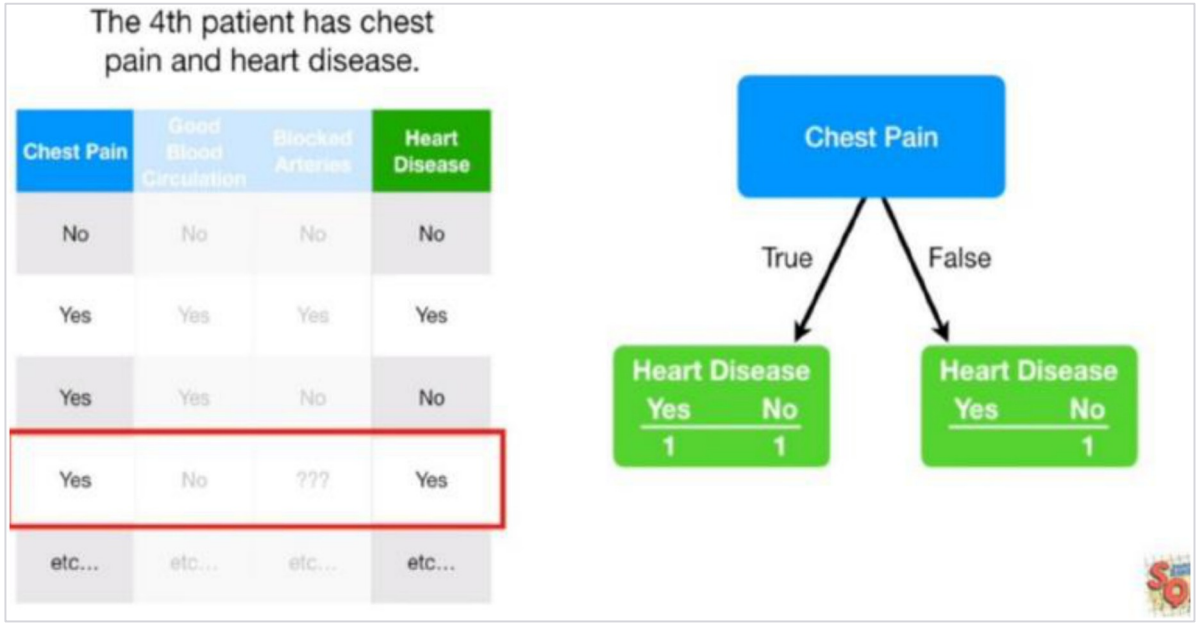
환자 한 명씩 리프에 기록한다 (2)

3번: CP 있음 · 심장병 없음 → 왼쪽 리프 No / 4번: CP 있음 · 심장병 있음 → 왼쪽 리프 Yes

환자 3



환자 4

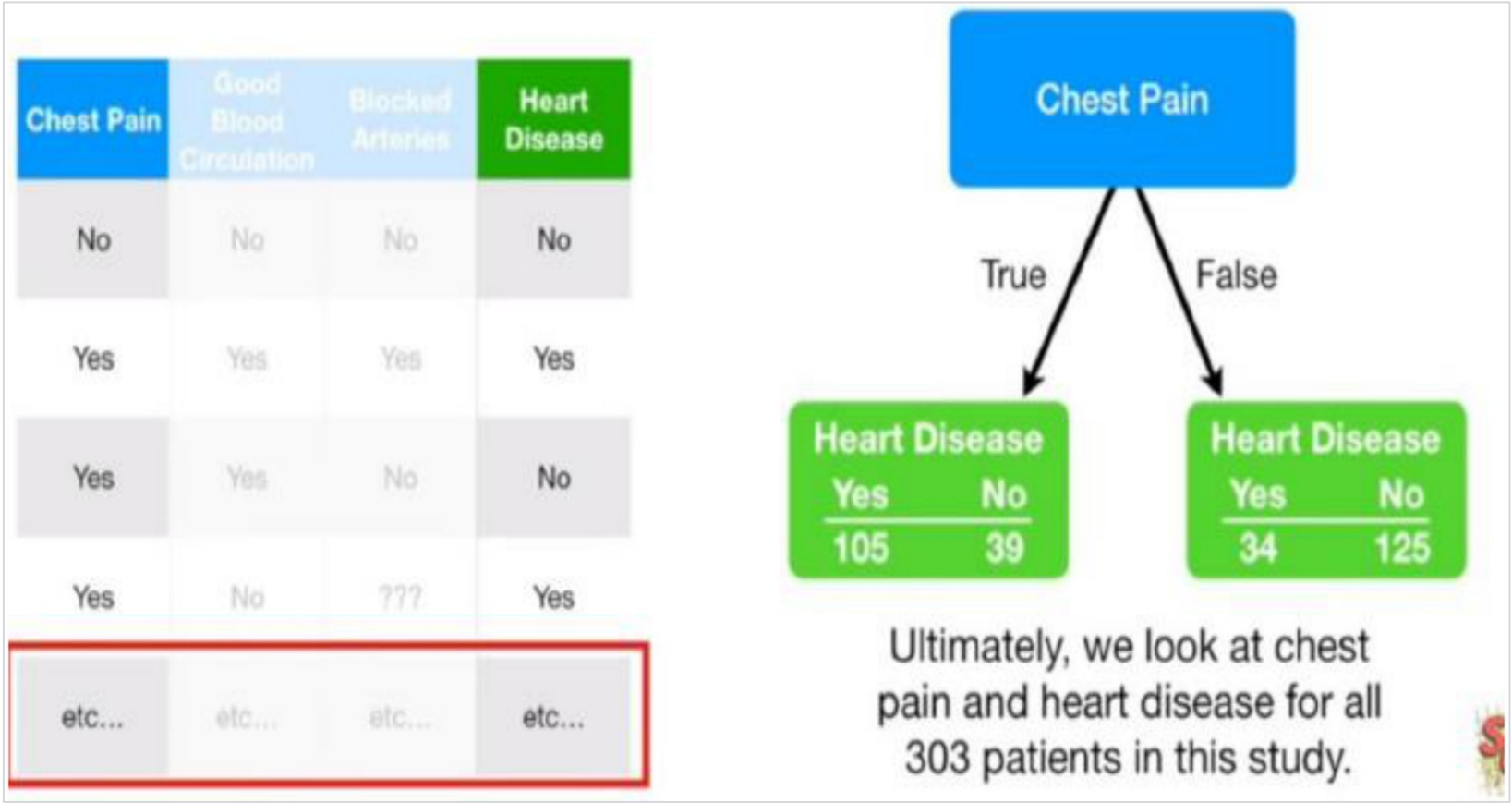


303명 전체를 집계하면

CP 기준 왼쪽 리프 105/39, 오른쪽 리프 34/125

관찰

- 어느 리프도 Yes 100% 또는 No 100%가 아니다
- → 두 리프 모두 "불순(impure)"하다
- 다른 후보도 같은 방식으로 집계해 비교한다



루트 후보 ② 혈액순환(BC) 기준 집계

같은 방법으로 BC의 True/False 리프를 채운다: 37/127 · 100/33

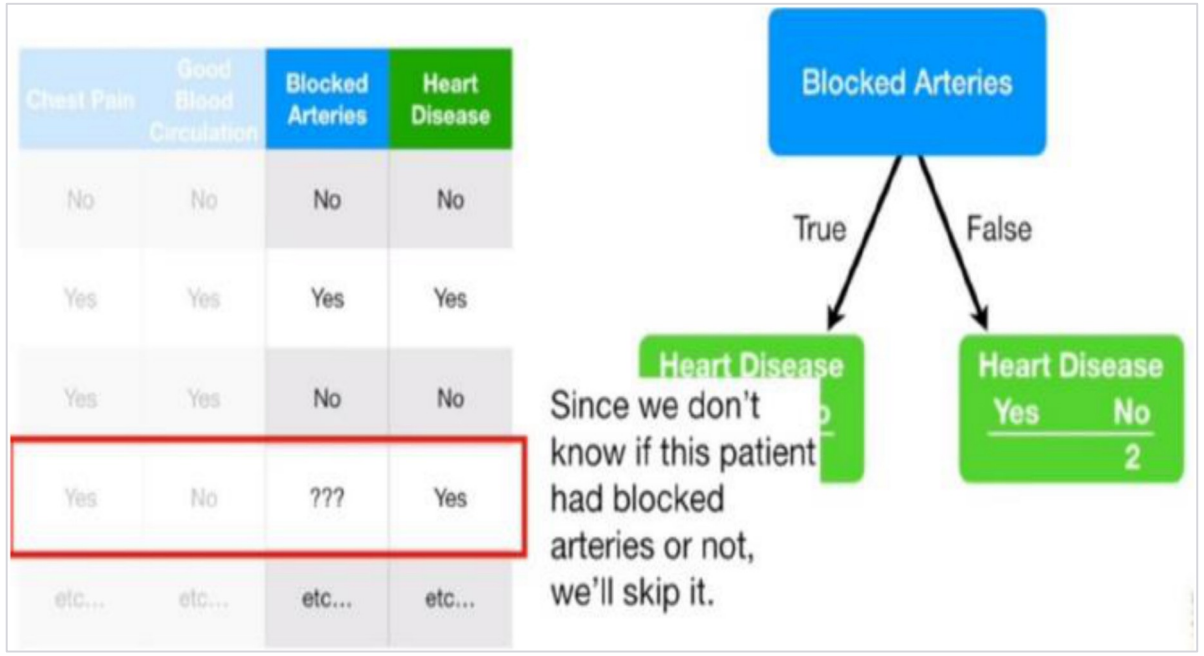
Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...



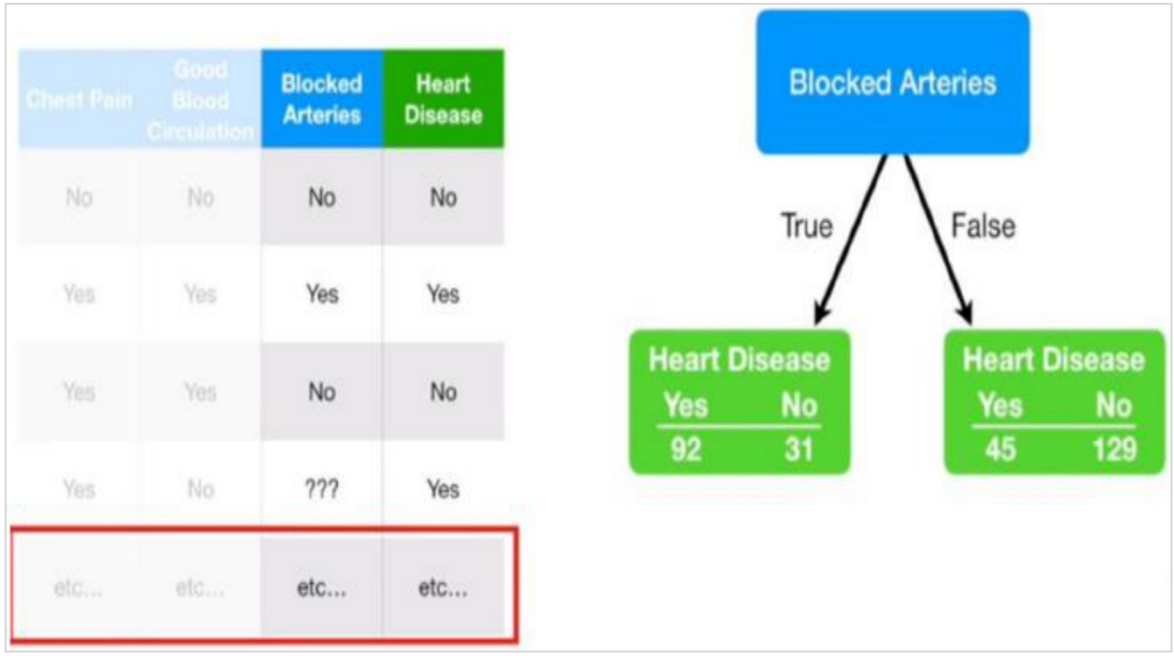
루트 후보 ③ 동맥경화(BA) — 결측은 건너뛴다

BA 값이 "???"인 환자는 일단 건너뛰고(skip) 집계한다

BA가 ???인 환자는 skip



BA 기준 전체 집계: 92/31 · 45/129

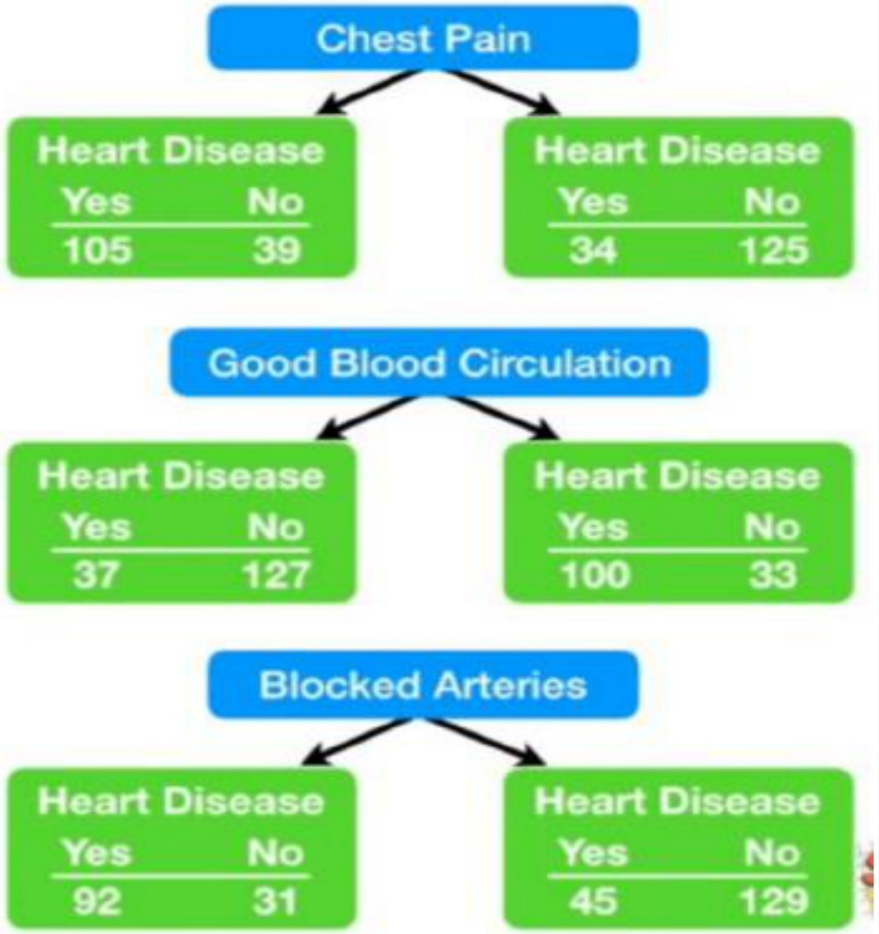


세 후보 모두 리프가 섞여 있다 → 어느 쪽이 "덜 섞였는지" 잴 척도 = 불순도

불순도 — 세 후보의 리프 비교

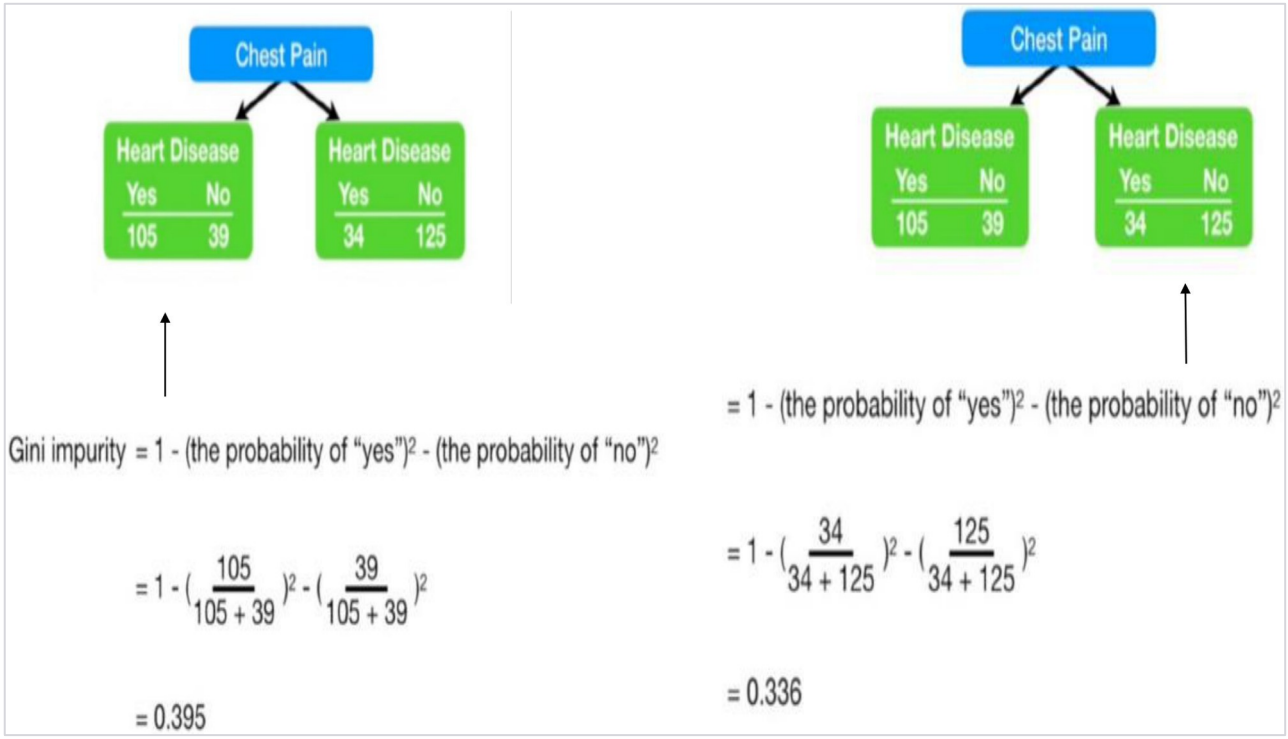
리프에 Yes 또는 No가 100%인 경우가 없으므로 모두 불순하다. 척도 중 하나가 지니(Gini)다

리프노드에서 YES 또는 NO가 100%는 없으므로 모두 불순(impure)하다. 이 불순도를 비교할 것이며 이를 위한 척도 중 하나가 “지니(Gini)”이다.



지니 불순도 계산 — 리프별

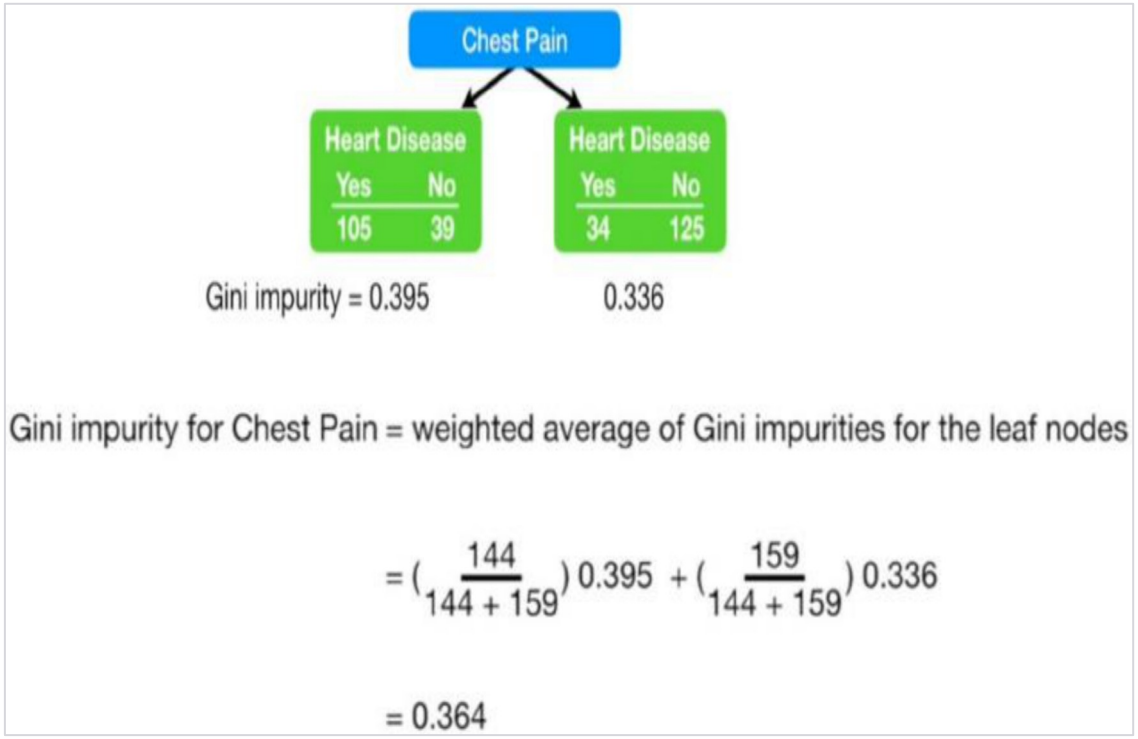
Gini = 1 − (Yes 확률)² − (No 확률)² · 흉부통의 두 리프



$$G_L = 1 - \left(\frac{105}{144}\right)^2 - \left(\frac{39}{144}\right)^2 = 0.395 \qquad G_R = 1 - \left(\frac{34}{159}\right)^2 - \left(\frac{125}{159}\right)^2 = 0.336$$

분할 전체의 지니 = 리프 지니의 가중평균

리프에 속한 환자 수로 가중한다 — 흉부통(CP)의 지니 = 0.364



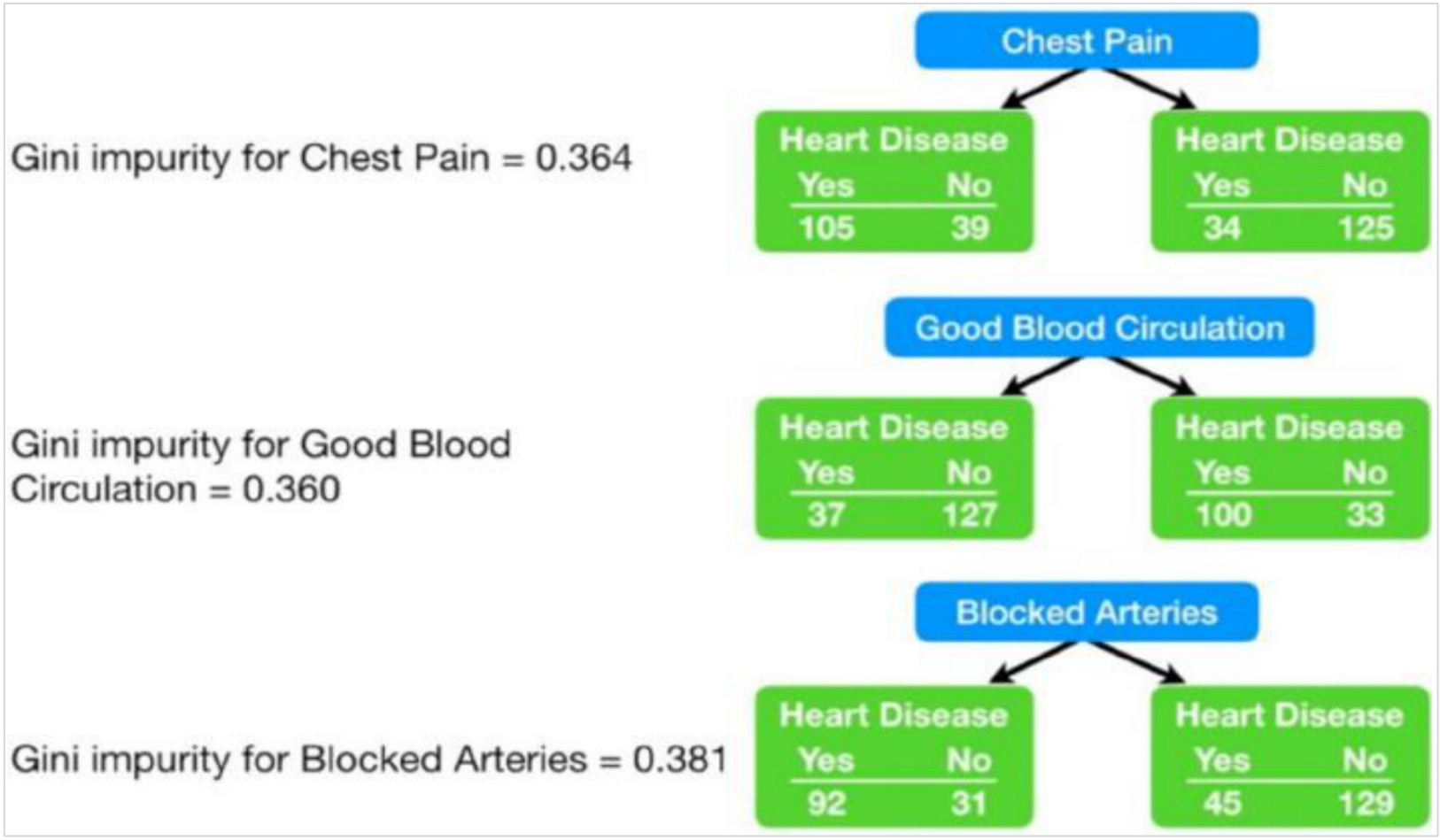
$$G_{\text{CP}} = \frac{144}{303}(0.395) + \frac{159}{303}(0.336) = 0.364$$

루트 결정 — 지니가 가장 낮은 혈액순환(BC)

CP 0.364 · BC 0.360 · BA 0.381

결정

- 가장 불순도가 낮은 BC를 루트 노드에 둔다
- 리프 크기로 가중한 평균 지니를 비교한다 (앞 장의 식)
- 차이가 작을 때는 샘플이 조금만 바뀌어도 루트가 달라진다 → 트리의 높은 분산

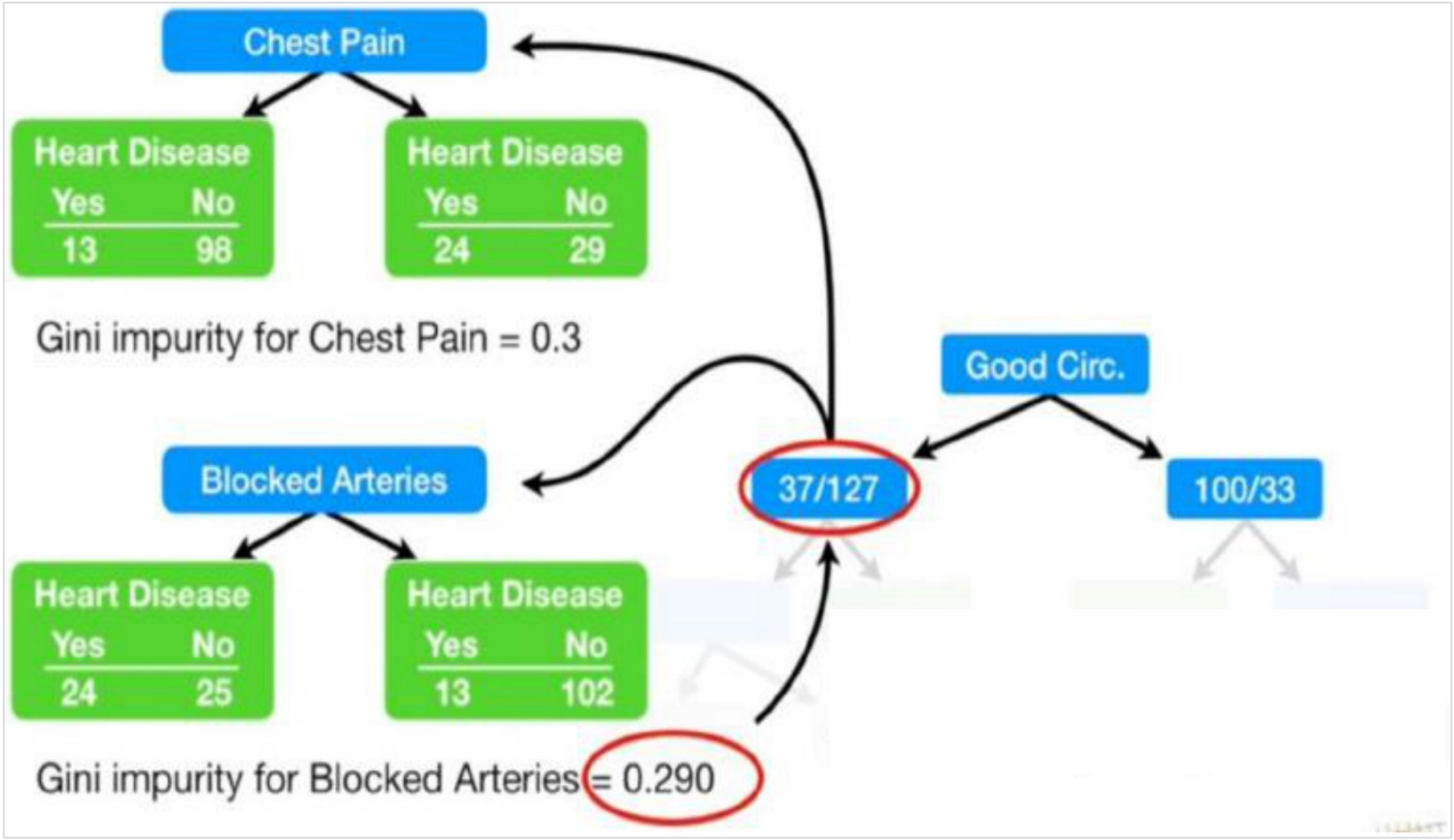


2단계 분기 — BC = True 쪽(37/127)

남은 후보 CP · BA를 같은 방식으로 비교한다

비교

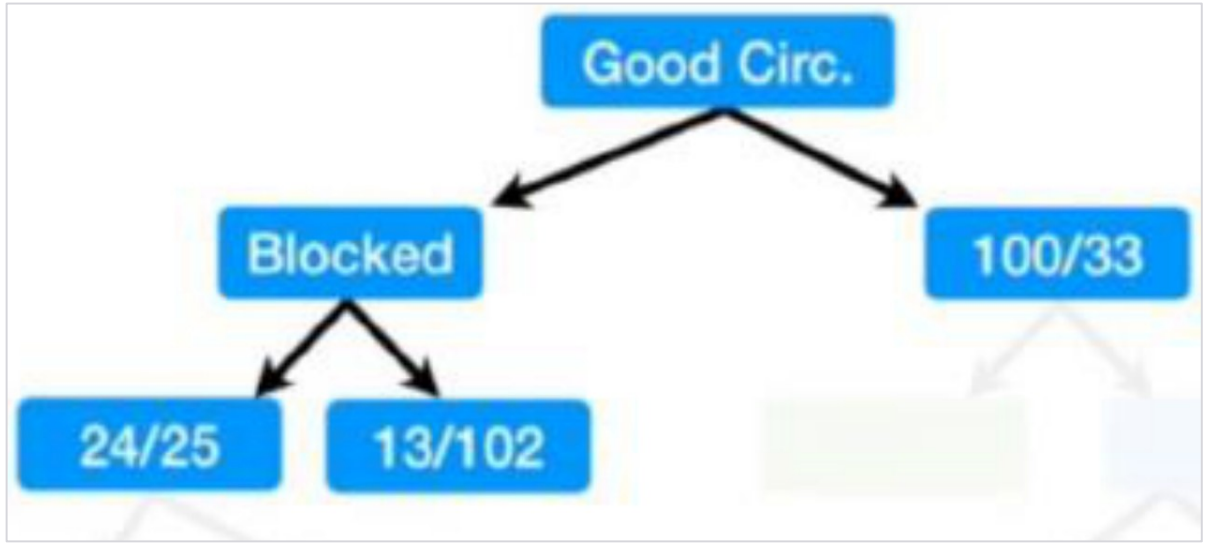
- CP로 나누면 지니 0.3
- BA로 나누면 지니 0.290 → 선택
- 하위 노드에서도 규칙은 똑같다: 가장 낮은 불순도



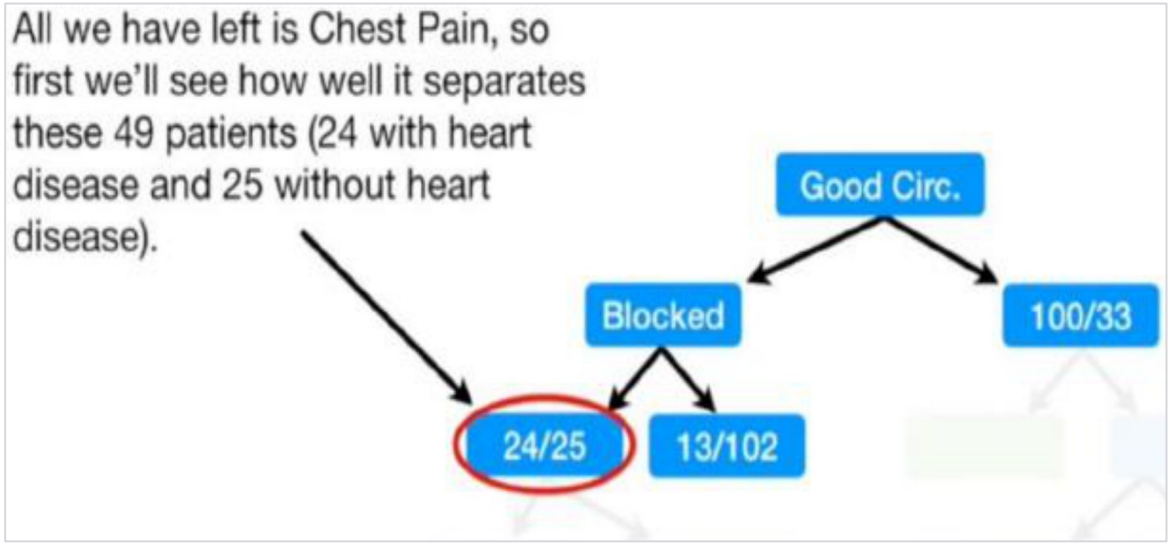
2단계 완료 → 3단계 분기 준비

왼쪽 24/25 노드에는 이제 흉부통(CP)만 남았다

2단계를 마친 트리



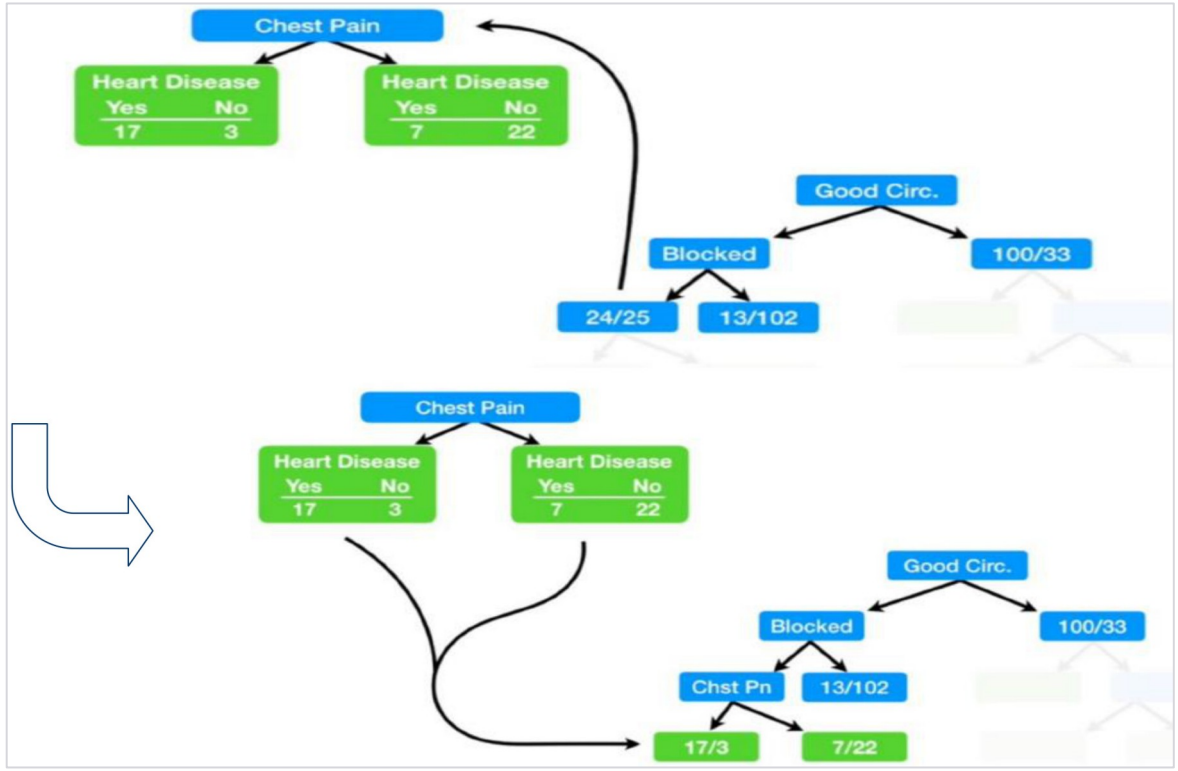
24/25 노드: 49명 (심장병 24 · 정상 25)



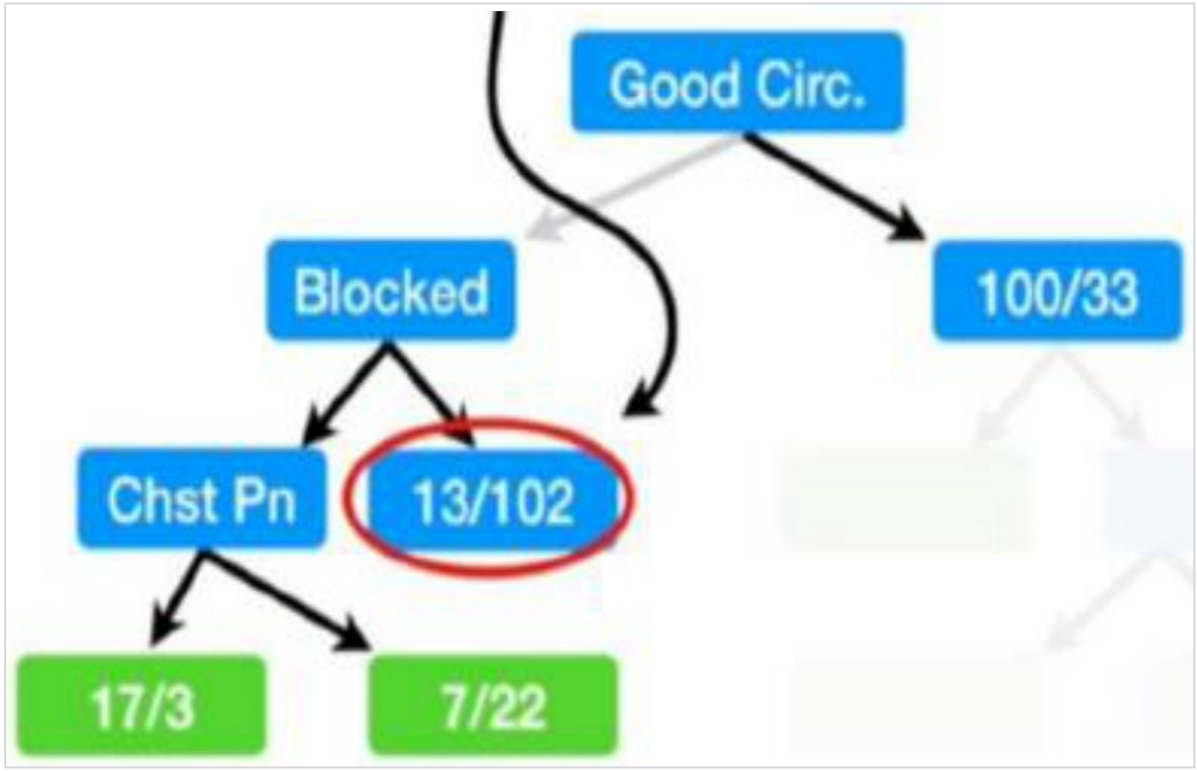
3단계 분기 — 남은 특성으로 계속 나눈다

24/25 노드는 CP로 17/3 · 7/22로 갈라진다. 이제 13/102 노드 차례다

24/25 노드를 CP로 분기



13/102 노드: 115명 중 13명 심장병

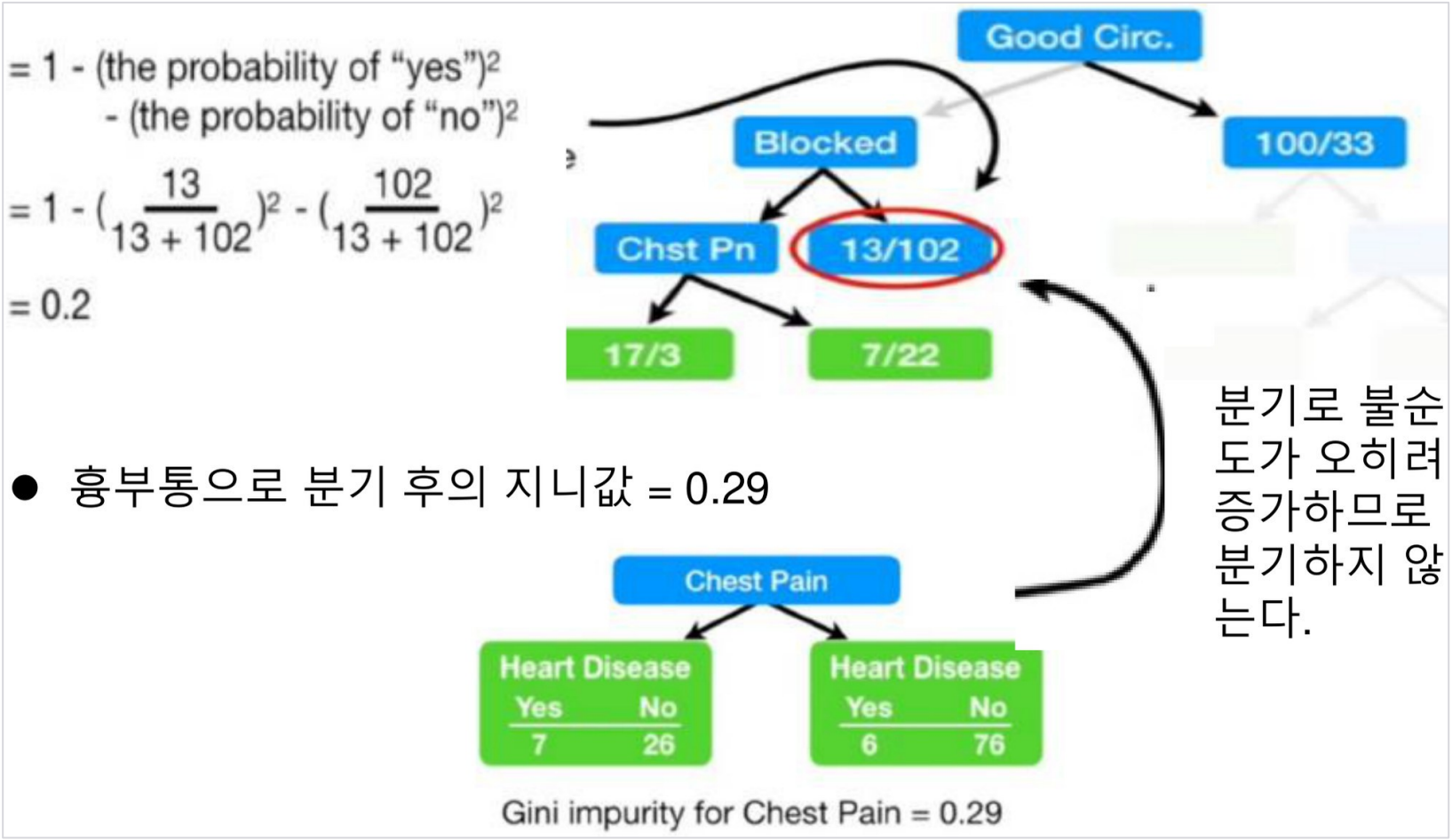


분기할까, 멈출까 — 분기 전후 지니 비교

13/102 노드: 분기 전 지니 0.2 vs CP로 분기 후 0.29

멈춤 규칙

- 분기하면 불순도가 오히려 커진다 (0.2 → 0.29)
- → 분기하지 않고 리프 노드로 둔다
- "더 나눌 수 있다"와 "나눌 가치가 있다"는 다르다

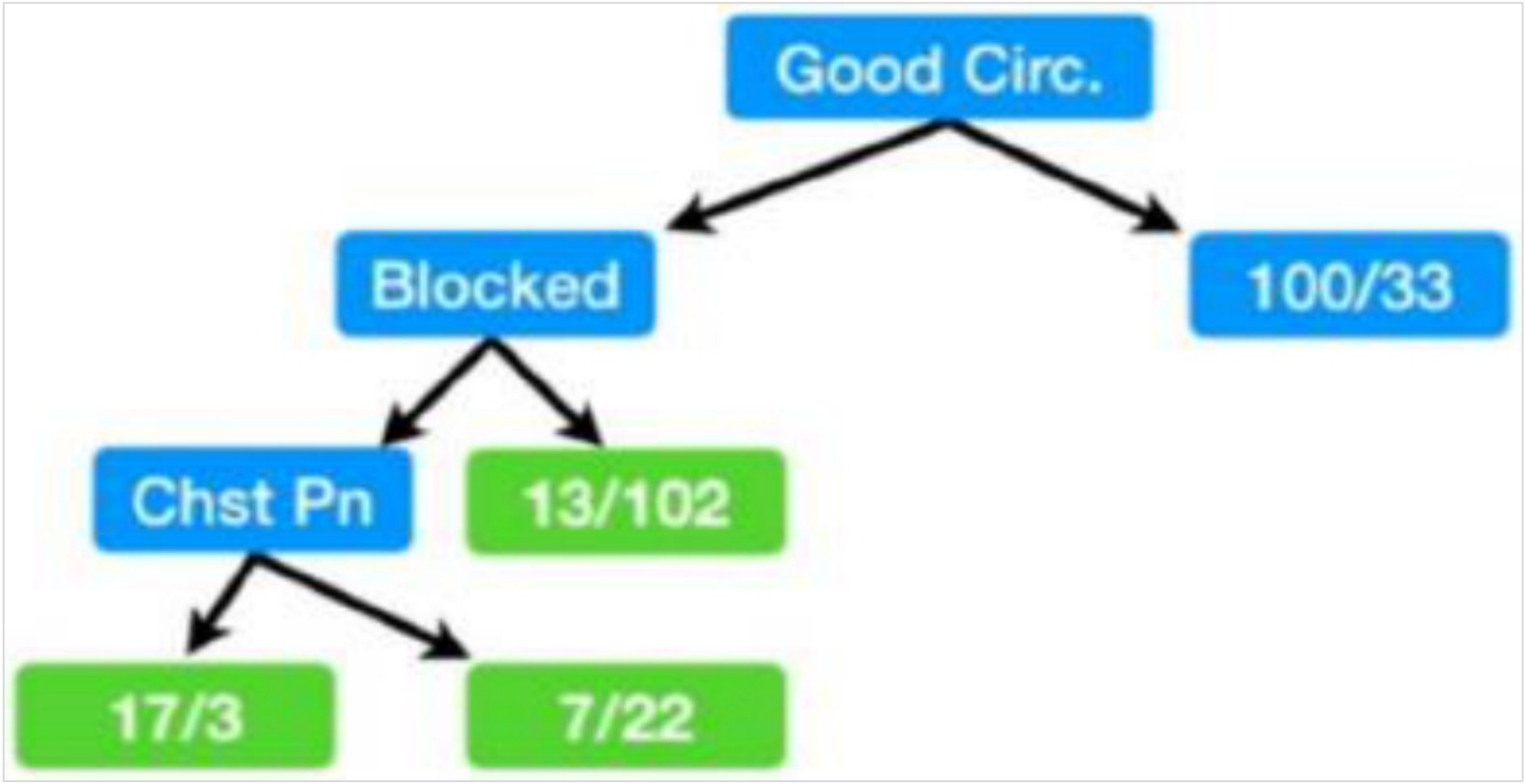


오른쪽 가지를 마저 완성한다

모든 노드에 같은 세 가지 규칙을 적용한다

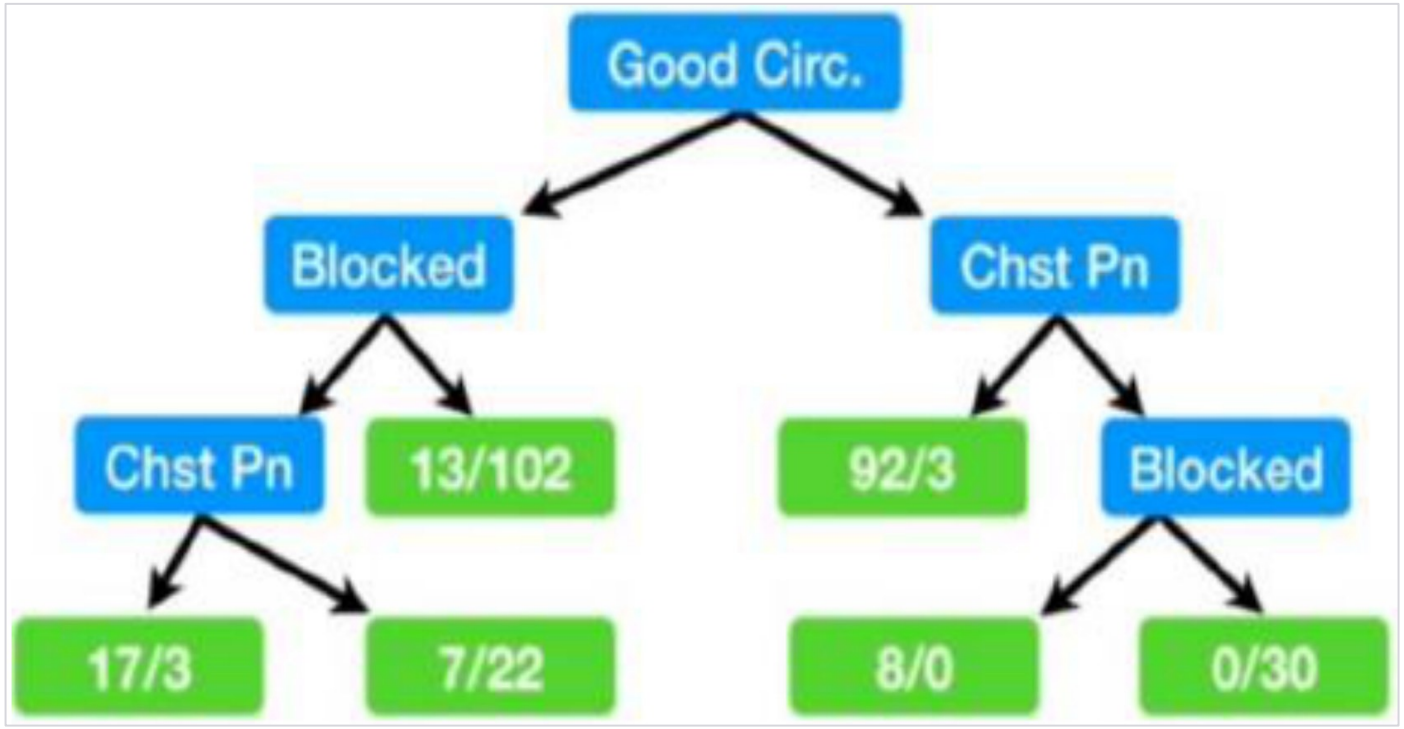
세 가지 규칙

- 1. 가능한 모든 분기에 대해 지니를 계산
- 2. 노드 자신의 지니가 가장 낮으면 → 리프
- 3. 분기가 불순도를 줄이면 → 가장 낮은 분기 선택



완성된 트리

루트 BC → 좌: BA / 우: CP. 리프 숫자는 심장병 Yes/No 환자 수



리프 8/0, 0/30처럼 완전히 순수한 리프도 생긴다. 새 환자는 질문을 따라 내려가 리프의 다수 클래스로 분류된다.

수치형 특성 — 체중(Weight)으로 나누려면

① 정렬 → ② 인접값 평균을 임계값 후보로 → ③ 후보마다 지니 계산

원 데이터

Weight	Heart Disease
220	Yes
180	Yes
225	Yes
190	No
155	No

① 체중순 정렬

	Weight	Heart Disease
Lowest	155	No
	180	Yes
	190	No
	220	Yes
Highest	225	Yes

② 인접 평균 계산

Weight	Heart Disease
155	No
167.5	
180	Yes
185	
190	No
205	
220	Yes
222.5	
225	Yes

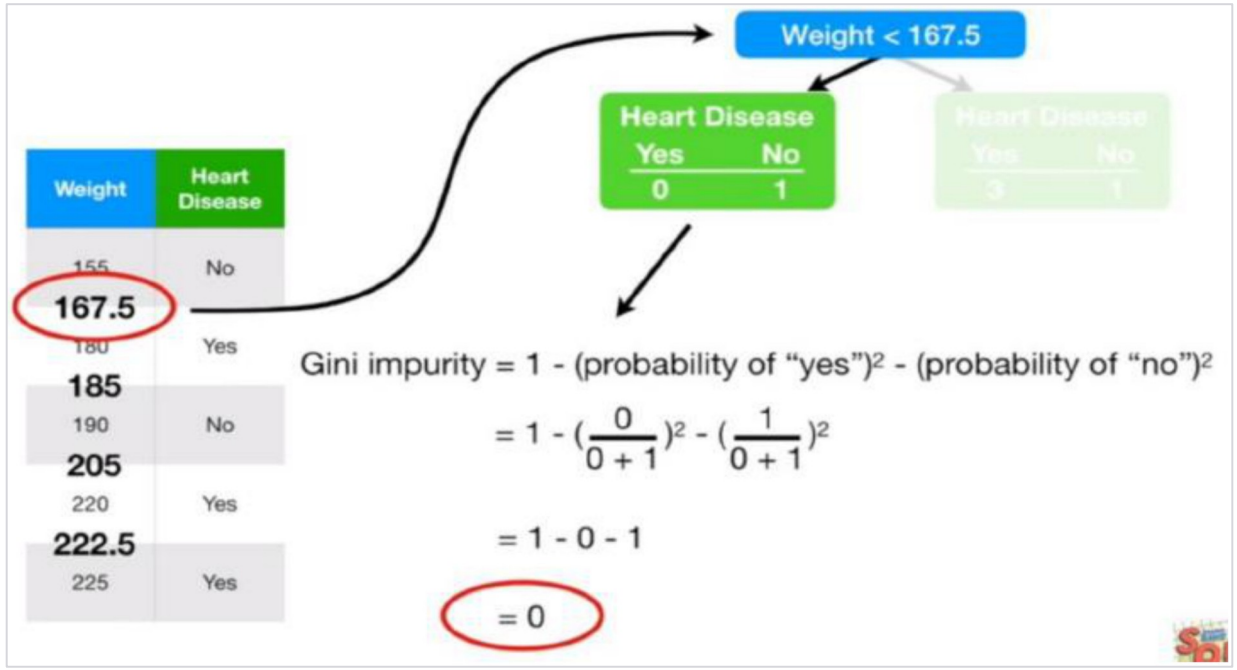
③ 후보별 지니

Weight	Heart Disease	
155	No	
167.5		Gini impurity = ?
180	Yes	
185		Gini impurity = ?
190	No	
205		Gini impurity = ?
220	Yes	
222.5		Gini impurity = ?
225	Yes	

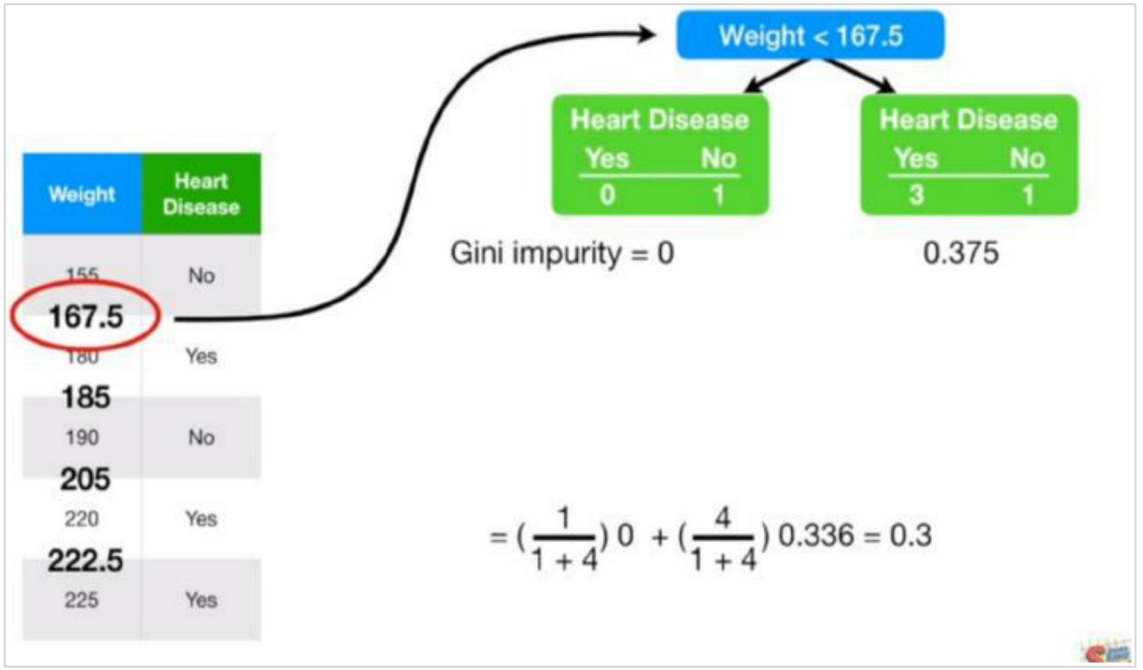
후보 임계값 "체중 < 167.5"의 지니

왼쪽 리프(0/1)는 지니 0, 오른쪽 리프(3/1)는 0.375 → 가중평균 0.3

왼쪽 리프: 순수 (지니 0)



가중평균: (1/5)·0 + (4/5)·0.375 = 0.3

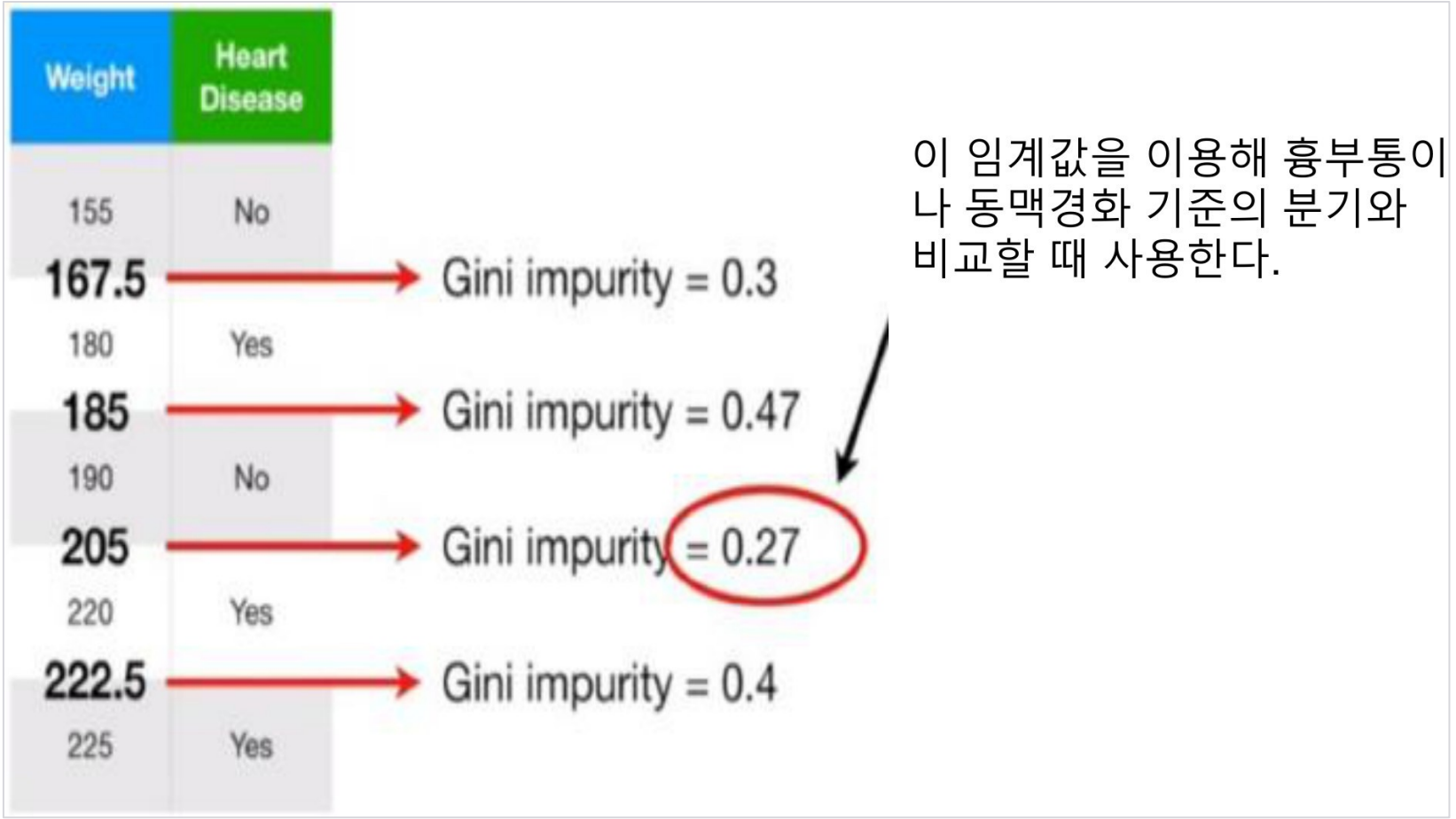


가장 불순도가 낮은 분기는 "체중 < 205"

후보 네 개: 0.3 · 0.47 · 0.27 · 0.4

다음 단계

- 체중의 대표 임계값 = 205 (지니 0.27)
- 이 값을 CP · BA 같은 다른 특성의 지니와 비교해 노드 질문을 정한다
- 정렬 비용 때문에 대규모 데이터에선 히스토그램 구간화(HistGB · LightGBM)를 쓴다



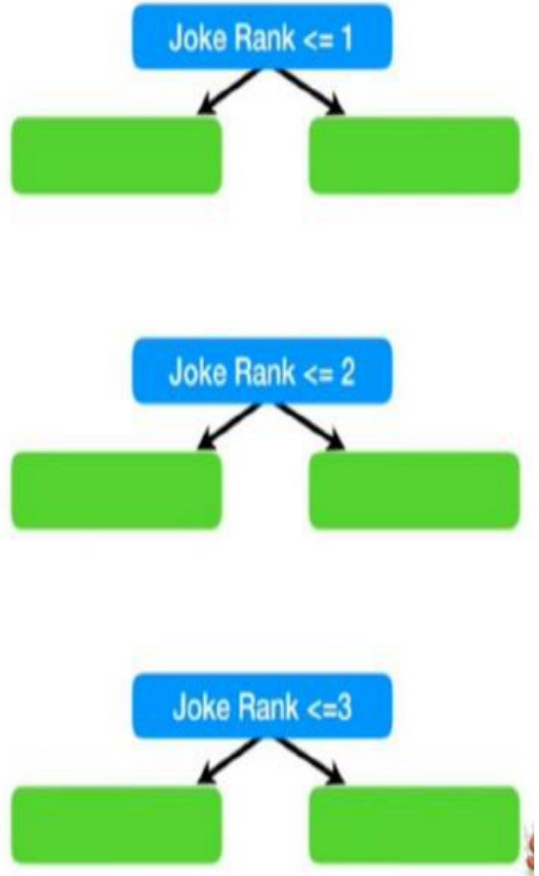
순서형 특성 — 순위 기준으로 자른다

예: "내 농담 순위(1~4)"와 StatQuest 선호 여부

방법

- 수치형과 같되, 가능한 순위 경계마다 지니를 계산
- Rank ≤ 1, ≤ 2, ≤ 3 까지만 본다
- Rank ≤ 4는 모든 샘플을 포함하므로 분할이 아니다

Rank my jokes...	Likes StatQuest
1	Yes
1	No
3	Yes
1	Yes
etc...	etc...

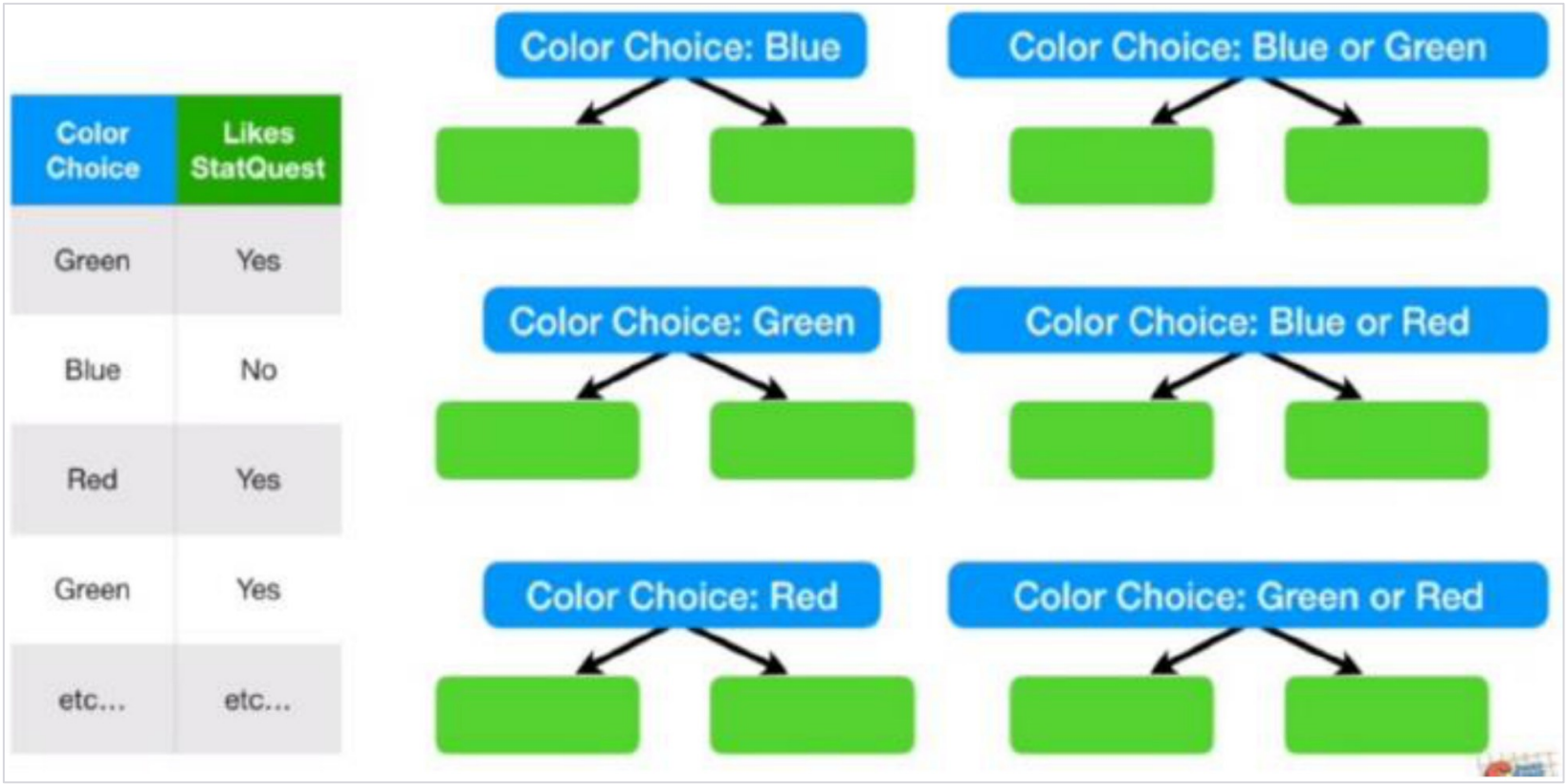


범주형 특성 — 조합마다 계산한다

예: 좋아하는 색이 파랑 · 초록 · 빨강 중 하나

방법

- 개별 범주와 가능한 모든 조합(Blue or Green 등)의 지니를 계산
- 범주가 늘면 조합 수가 지수적으로 급증
- 실무: 원-핫 인코딩, 또는 CatBoost · HistGB의 범주형 전용 처리



과대적합과 복잡도 제어

모든 리프가 순수해질 때까지 키우면 훈련 데이터를 100% 맞추지만 일반화는 낮다

하이퍼파라미터	의미	효과
max_depth	트리의 최대 깊이	가장 직접적인 복잡도 제한
max_leaf_nodes	리프 노드의 최대 개수	리프 수로 크기 제한
min_samples_leaf	리프가 되기 위한 최소 샘플 수	작은 리프(외우기) 방지
min_samples_split	노드를 분할하기 위한 최소 샘플 수	작은 노드의 분할 억제
ccp_alpha	비용-복잡도 가지치기 강도	다 키운 뒤 잘라내기 (사후 가지치기)

사전 가지치기(pre-pruning): 위 네 개로 성장을 미리 멈춤 · 사후 가지치기(post-pruning): ccp_alpha로 다 키운 트리를 잘라냄

구현 — 깊이 제한의 효과

유방암 데이터: 무제한 트리 vs max_depth = 4

결과 해석

- 무제한: 훈련 1.000 · 테스트 0.937 → 과대적합
- 깊이 4: 훈련 0.988 · 테스트 0.951
- 훈련 정확도를 조금 내주고 일반화를 얻었다
- scikit-learn 1.8에서 재실행해도 같은 값

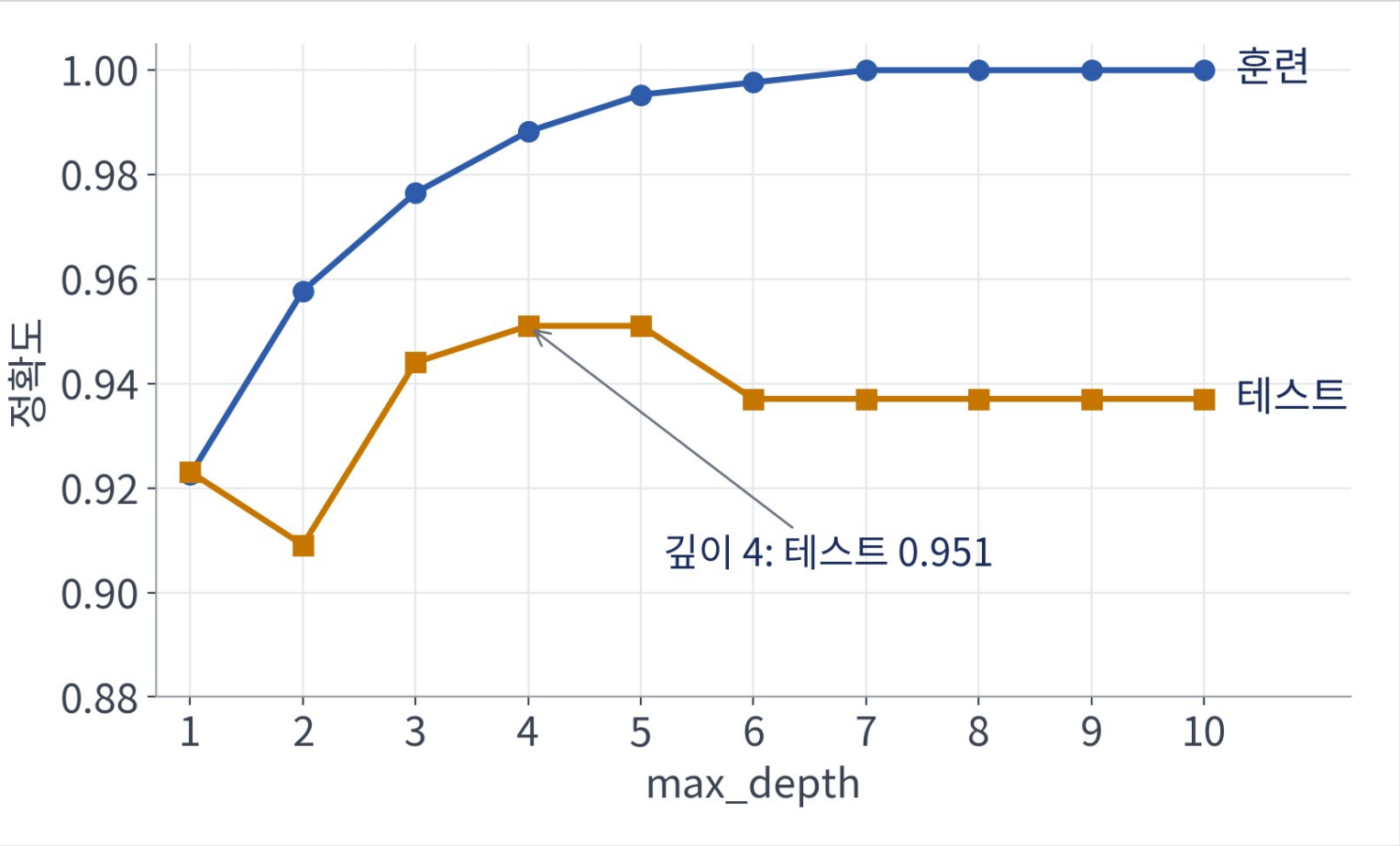


보강 — 깊이에 따른 훈련/테스트 정확도

같은 데이터·분할에서 max_depth를 1~10으로 바꿔 본 결과 (scikit-learn 1.8 재현)

읽는 법

- 훈련 정확도는 깊을수록 1.0으로 오른다
- 테스트 정확도는 깊이 4~5에서 가장 높고 이후 떨어진다
- 실무에서는 교차검증으로 깊이를 고른다



보강 — 비용-복잡도 가지치기 (CART)

트리의 오차와 리프 수 사이의 균형을 α 하나로 조절한다

$$R_\alpha(T) = R(T) + \alpha\,|\tilde{T}|$$

기호

- R(T): 트리 T의 훈련 오차(불순도 합)
- |T̃|: 리프 개수
- α: 리프 하나당 벌점 — 클수록 작은 트리

실무

- cost_complexity_pruning_path()로 α 후보를 얻는다
- α를 교차검증으로 고른다
- Breiman 외(1984) CART 책의 방법 그대로다

특성 중요도 (feature importance)

0(사용 안 됨) ~ 1(단독으로 완벽 예측) 사이, 전체 합은 1

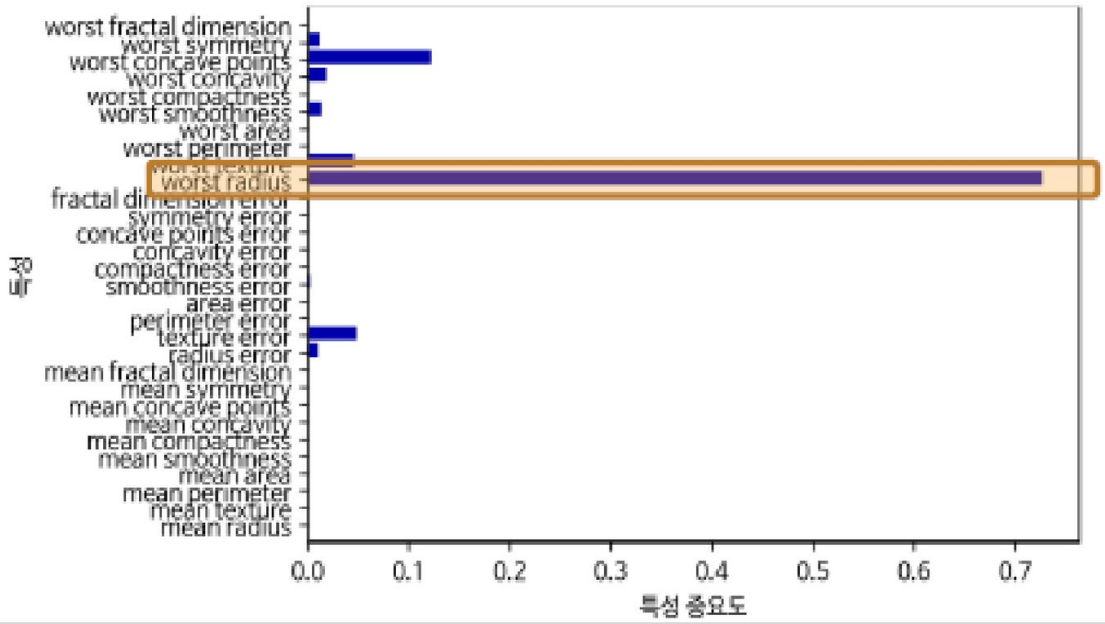
주의할 점

- "worst radius" 하나가 0.727 — 루트 근처 분할이 크게 잡힌다
- 불순도 감소 기반(MDI)은 값이 많은 연속형 특성을 과대평가하는 경향
- 상관된 특성끼리 중요도를 나눠 갖는다
- 교차 확인: permutation importance · SHAP

0(예측에 사용되지 않음)과 1(예측과 완벽하게 동일) 사이의 값, 전체 합은 1

```
In [63]: print("특성 중요도:\n{}".format(tree.feature_importances_))
```

```
특성 중요도:
[ 0.     0.     0.     0.     0.     0.     0.     0.     0.     0.     0.01
  0.048  0.     0.     0.002  0.     0.     0.     0.     0.     0.727
  0.046  0.     0.     0.014  0.     0.018  0.122  0.012  0.     ]
```

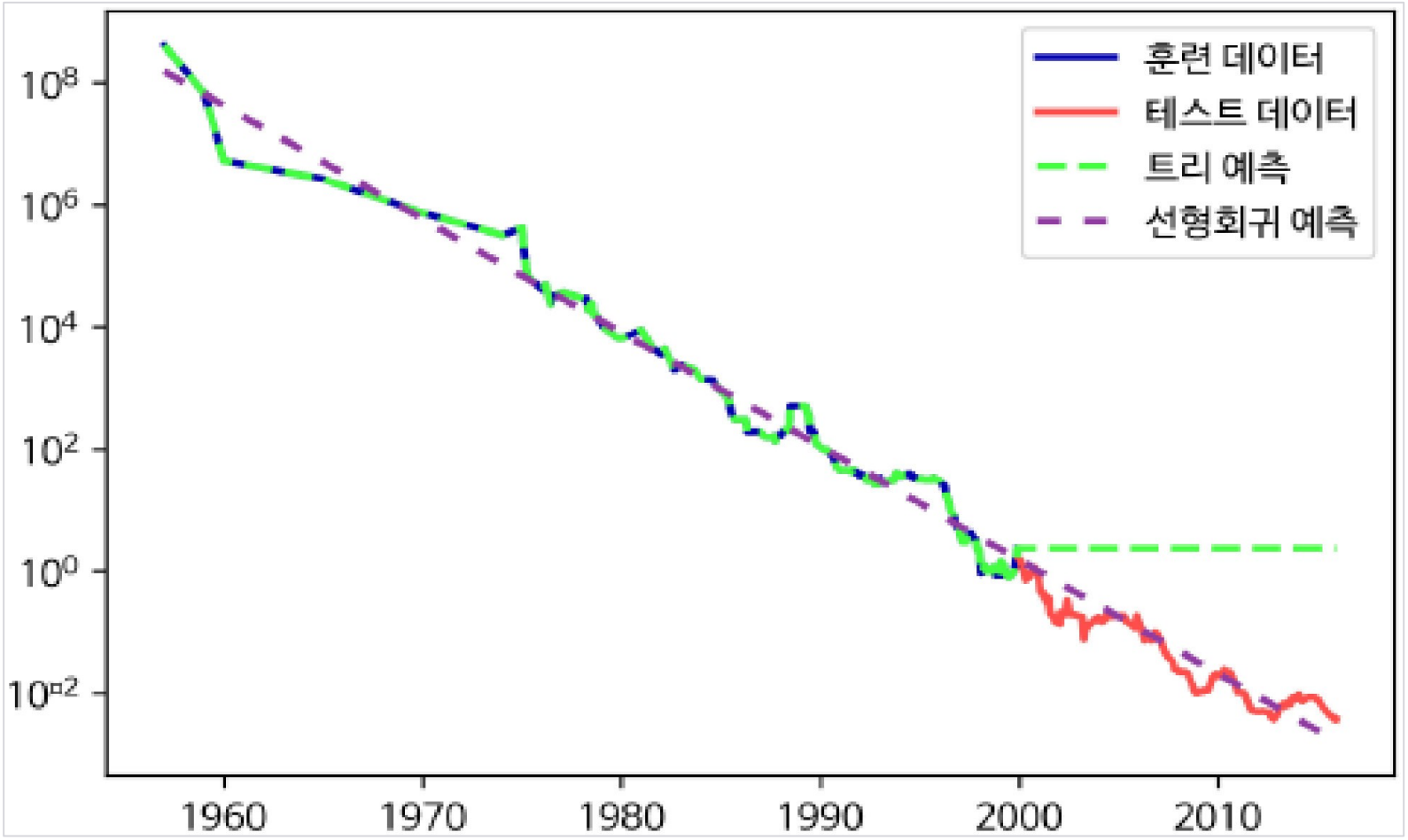


트리는 외삽(extrapolation)을 못 한다

훈련 범위 밖(2000년 이후)에서 트리 예측은 마지막 리프 값에 머문다

금융 데이터에 주는 함의

- 가격 · 지수 레벨처럼 추세가 있는 타깃을 트리로 직접 예측하면 위험
- 수익률 · 변화율 · 비율로 바꿔 정상성을 맞춘다
- 선형 모델은 추세를 이어가지만 트리는 평평해진다 (그림: 로그 스케일)



의사결정트리의 장단점

장점

- 작은 트리는 시각화가 쉬워 설명하기 좋다
- 특성을 하나씩 처리하므로 스케일에 무관 → 정규화 · 표준화 불필요
- 이진 · 연속형 특성이 섞여도 문제없다

단점

- 과대적합 — 가장 큰 약점
- 데이터가 조금 바뀌어도 구조가 크게 바뀐다 (높은 분산)
- 축에 평행한 경계 · 외삽 불가

분산은 트리 평균(2부 RF)으로, 편향은 순차 학습(4·5부 부스팅)으로 줄인다

02

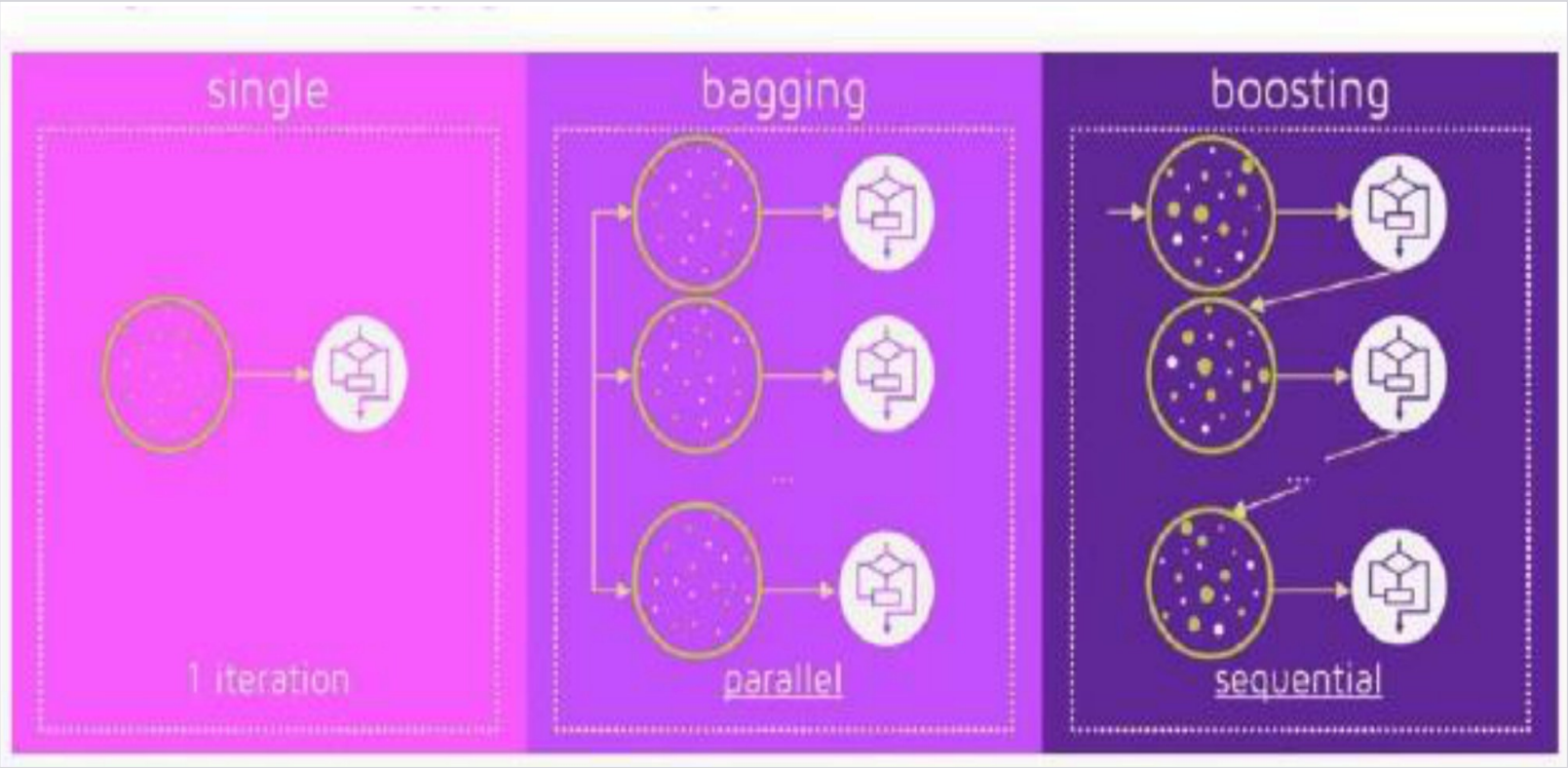
2부 · 랜덤 포레스트

랜덤 포레스트 — 서로 다른 트리를 모아 평균한다

앙상블 · 배깅 · 부트스트랩 · 특성 무작위 선택 · Out-of-Bag 검증

단일 모델 vs 배깅 vs 부스팅

배깅은 병렬로 독립 학습 후 결합, 부스팅은 앞 모델의 결과를 받아 순차로 학습



원 강의에서는 AdaBoost 파트 첫 장에 있던 그림이다. 두 갈래를 먼저 보여 주려고 여기로 옮겼다.

앙상블 · 배깅 · 부스팅

앙상블 (ensemble)

여러 ML 알고리즘의 결과를 결합해 하나의 추정치를 얻는다. 약학습기를 묶어 강학습기를 만들 수 있고, 얼마나 좋아지는지는 학습기 간 상관에 달렸다.

배깅 (bagging)

Bootstrap AGGregatING

복원추출로 새 데이터셋을 여러 개 만들어 각각 학습하고, 투표(분류) 또는 평균(회귀)으로 최종 추정치를 얻는다.

부스팅 (boosting)

예측을 순서대로 하며 이전 오차를 교정한다. AdaBoost는 오분류 관측치의 가중치를 올리고, GBM은 이전 예측기의 잔차에 새 예측기를 적합한다.

랜덤 포레스트란

많은 트리를 만들되, 트리끼리 서로 달라지도록 두 겹의 무작위성을 넣는다

무작위성 두 겹

- 행: 원 데이터에서 부트스트랩 표본을 뽑아 트리마다 다른 데이터
- 열: 각 노드에서 특성의 무작위 부분집합만 후보로 사용
- (Extra-Trees는 임계값까지 무작위화)

최종 결정

- 분류: 다수결 투표 (또는 확률 평균)
- 회귀: 트리 예측의 평균
- 가중 투표도 가능 — 가중치는 홀드아웃 정확도 등에서

$$\hat{f}_{\mathrm{RF}}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x) \qquad \hat{y} = \mathrm{mode}\left\{T_1(x), \ldots, T_B(x)\right\}$$

만드는 법 ① 부트스트랩 데이터

원 데이터와 같은 크기로 복원추출(중복 허용)한다

Original Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	167	Yes

Bootstrapped Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	Yes
No	No	No	125	No
Yes	No	Yes	167	Yes
Yes	No	Yes	167	Yes

원 데이터 4명 → 부트스트랩: 3번 환자가 두 번 뽑히고 4번 환자는 빠졌다.

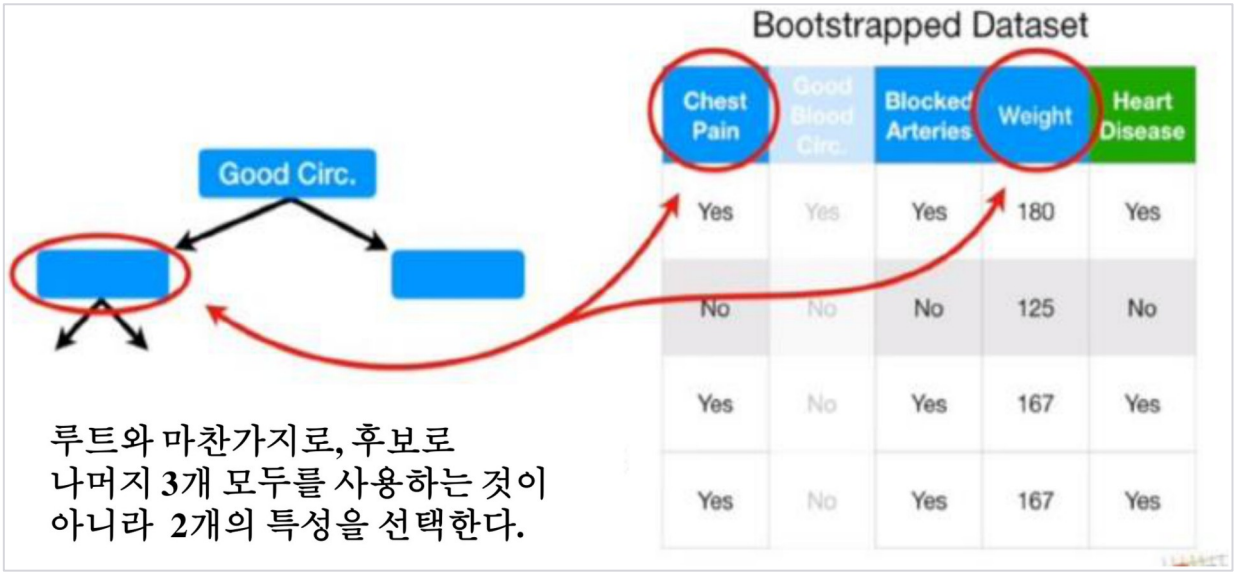
만드는 법 ② 노드마다 특성 일부만 후보로

루트에서는 무작위로 2개(BC · BA)만 골라 그중 최선을 택한다

루트: 무작위 2개 중 선택



다음 노드: 남은 3개 중 다시 2개

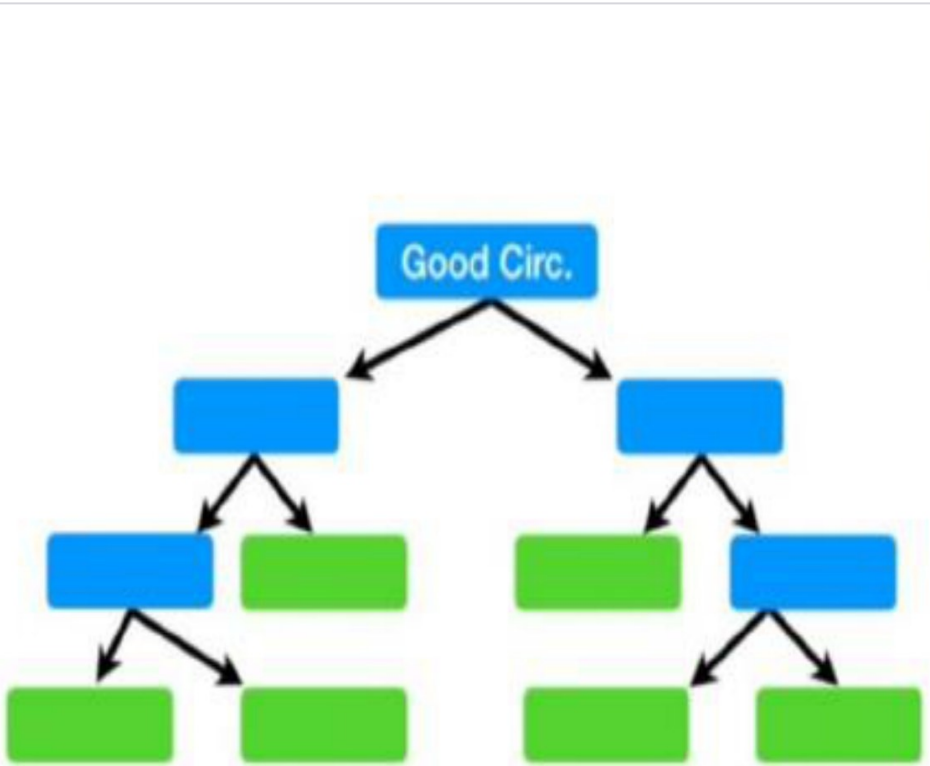


만드는 법 ② (계속) — 트리 완성

부트스트랩 데이터로, 매 단계 무작위 부분집합만 보며 트리를 끝까지 키운다

max_features

- 분류 기본값: 특성 수의 제곱근 (scikit-learn "sqrt")
- 회귀 권장: 전체 특성(1.0) — scikit-learn 사용자 가이드
- 작을수록 트리 간 상관 ↓, 개별 트리 정확도도 ↓

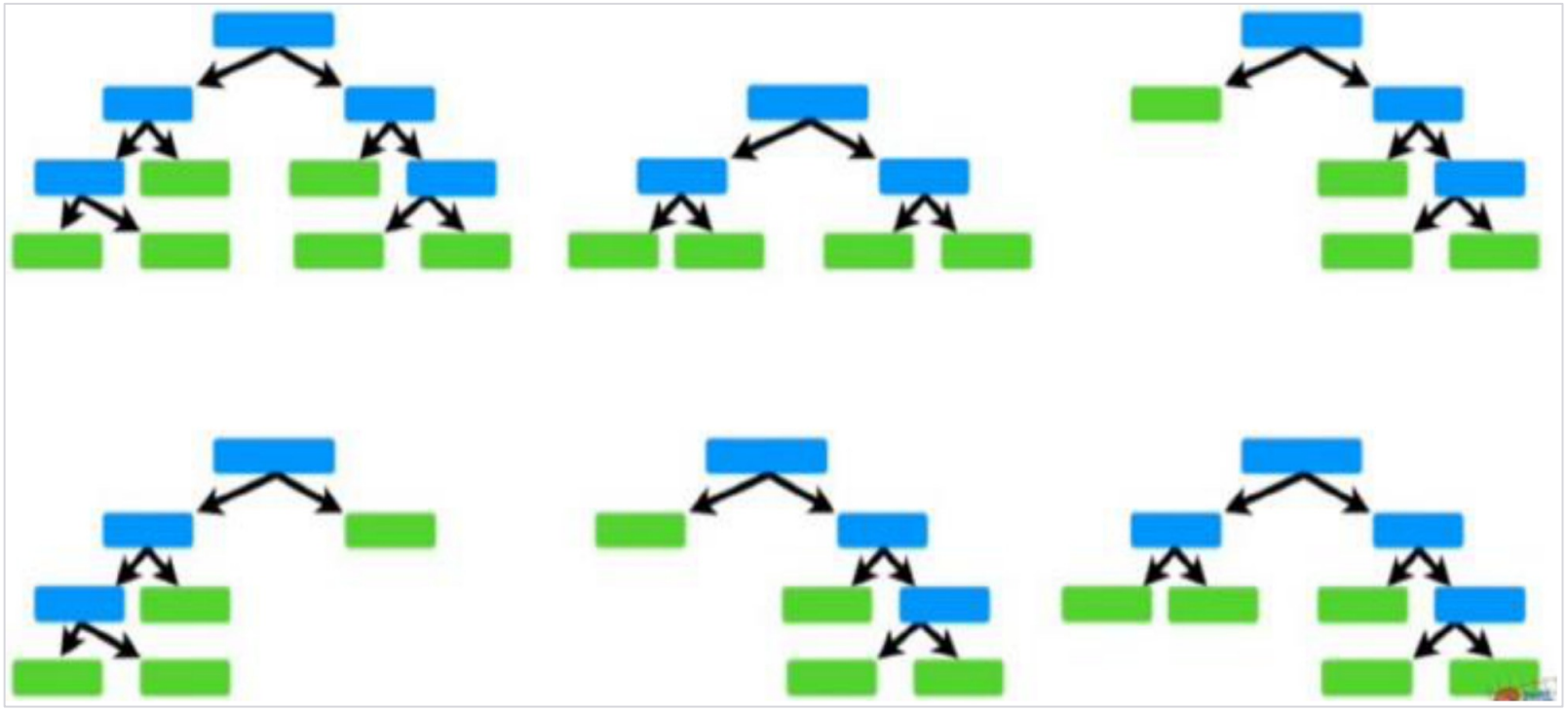


Bootstrapped Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	Yes
No	No	No	125	No
Yes	No	Yes	167	Yes
Yes	No	Yes	167	Yes

만드는 법 ③ 반복해서 원하는 수의 트리를

부트스트랩과 특성 선택이 매번 달라서 다양한 모양의 트리가 만들어진다



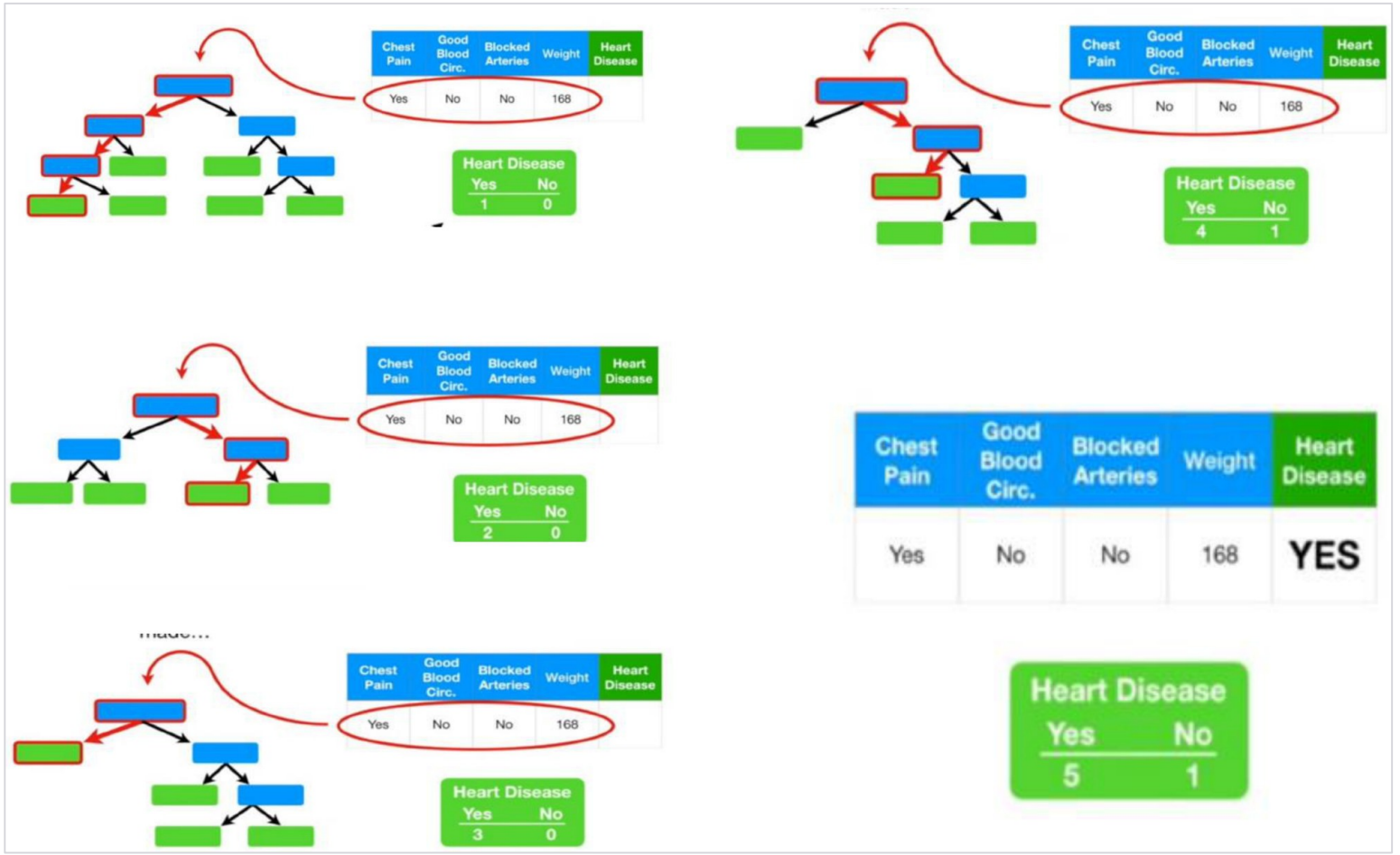
scikit-learn 기본 n_estimators = 100. 트리를 늘려도 과대적합이 심해지지는 않고, 성능이 수렴한다.

배깅으로 새 환자를 진료한다

새 환자 데이터를 모든 트리에 통과시켜 투표한다

결과

- 투표 결과 Yes 5 : No 1
- → 다수결로 "심장병 Yes"
- Bagging = Bootstrapping + Aggregate



Out-of-Bag(OOB) 테스트 — 뽑히지 않은 데이터

부트스트랩은 중복을 허용하므로 일부 샘플은 한 번도 뽑히지 않는다

핵심

- 보통 약 1/3이 부트스트랩에서 빠진다
- 빠진 샘플 = Out-of-Bag 데이터
- 이 트리의 학습에는 쓰이지 않았으므로 검증용으로 쓸 수 있다



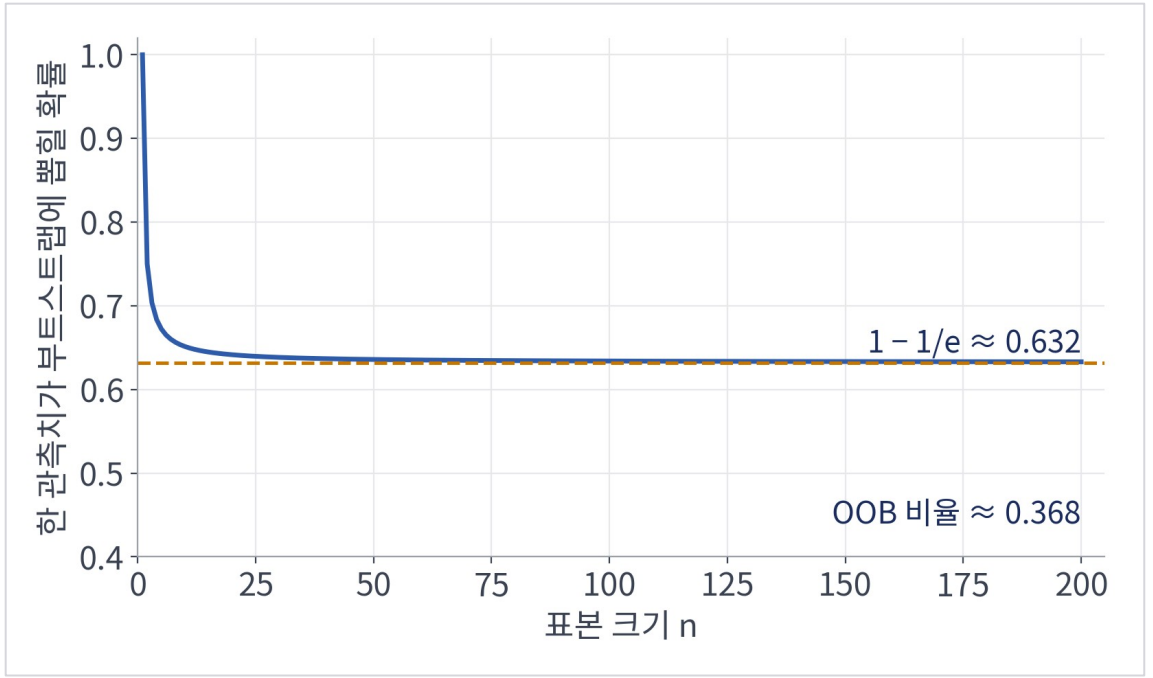
보강 — 왜 "약 1/3"인가

n개 중 복원추출 n번: 특정 샘플이 한 번도 안 뽑힐 확률은 $(1 - 1/n)^n \to 1/e$

$$\Pr(i \in \text{bootstrap}) = 1 - \left(1 - \frac{1}{n}\right)^n \longrightarrow 1 - e^{-1} \approx 0.632$$

정리

- 뽑히는 비율 ≈ 63.2%
- OOB 비율 ≈ 36.8% — 원 강의의 "약 1/3"
- n이 수십만 넘으면 거의 정확히 1/e



OOB 데이터로 트리를 검증한다

부트스트랩에 포함되지 않은 데이터를 "Out-of-Bag 데이터셋"이라 한다

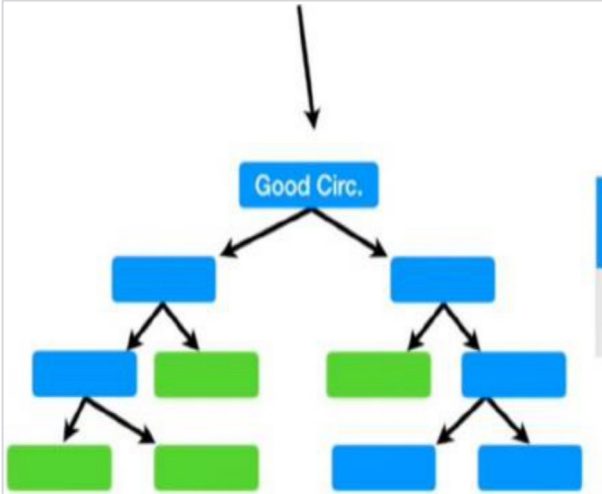
4번째 환자가 OOB

Original Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	167	Yes

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	No	210	No

이 트리로 OOB 샘플을 분류해 본다

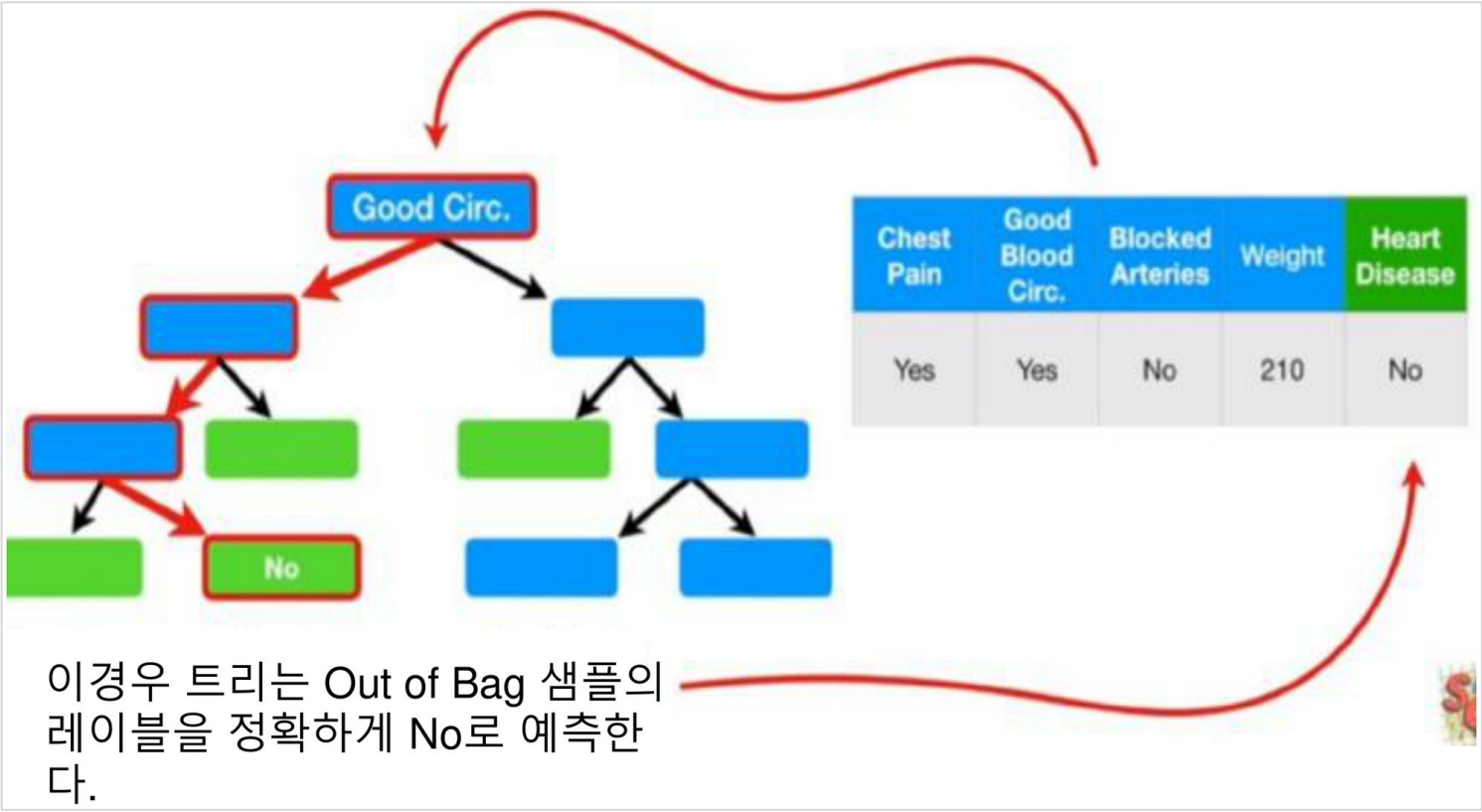


Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	No	210	No

OOB 샘플을 트리마다 분류한다

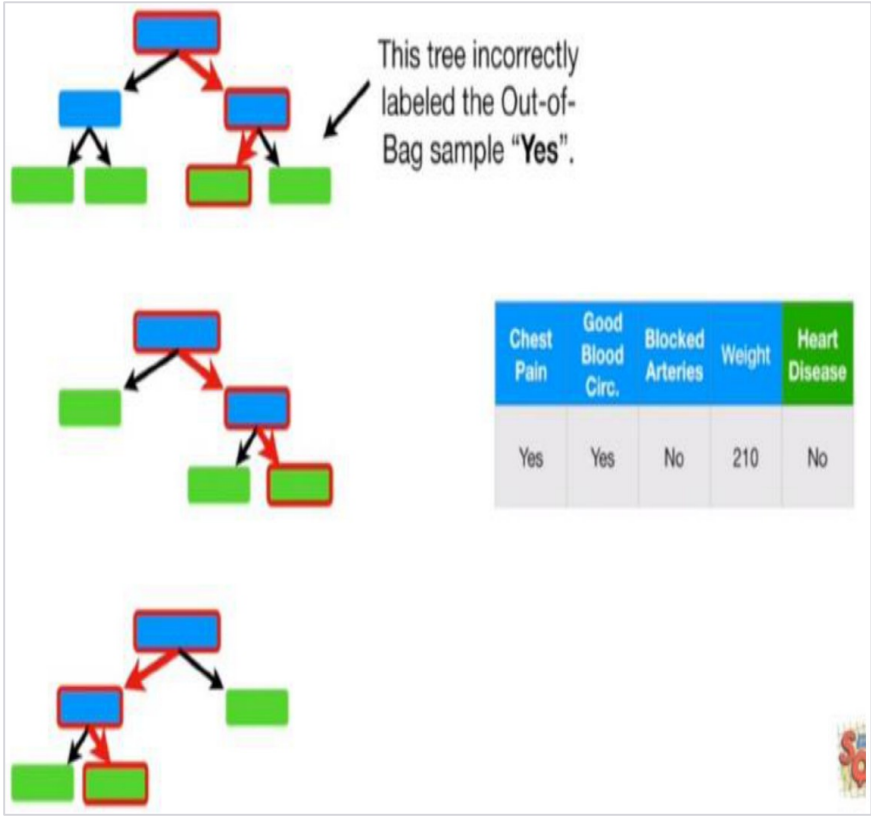
이 샘플을 학습에 쓰지 않은 트리들만 투표에 참여한다

트리 1: 정답 "No" 예측



이경우 트리는 Out of Bag 샘플의 레이블을 정확하게 No로 예측한다.

나머지 트리: 한 트리는 "Yes"로 오분류

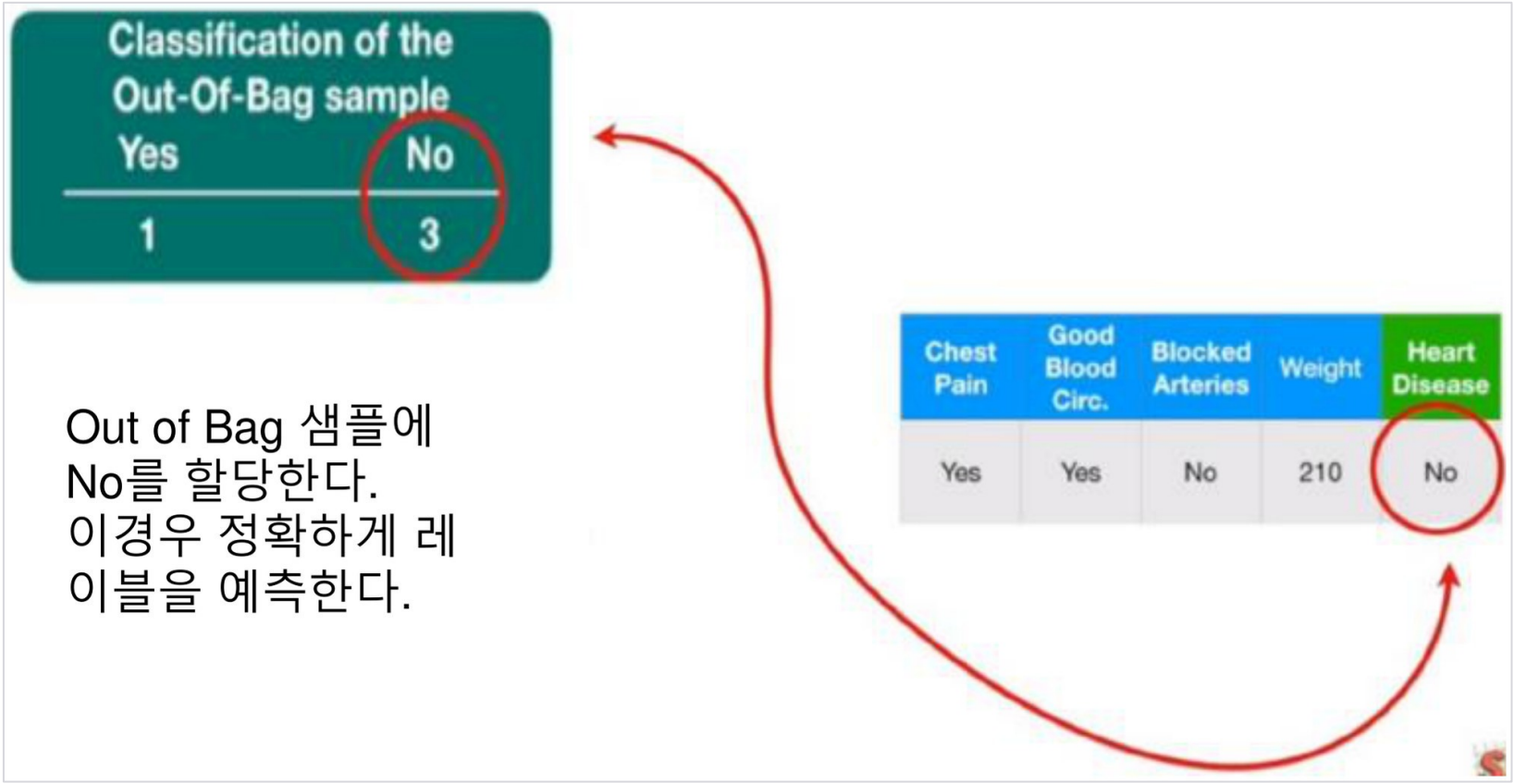


OOB 샘플의 최종 분류

투표 결과 No 3 : Yes 1 → "No" — 정답과 일치

결과

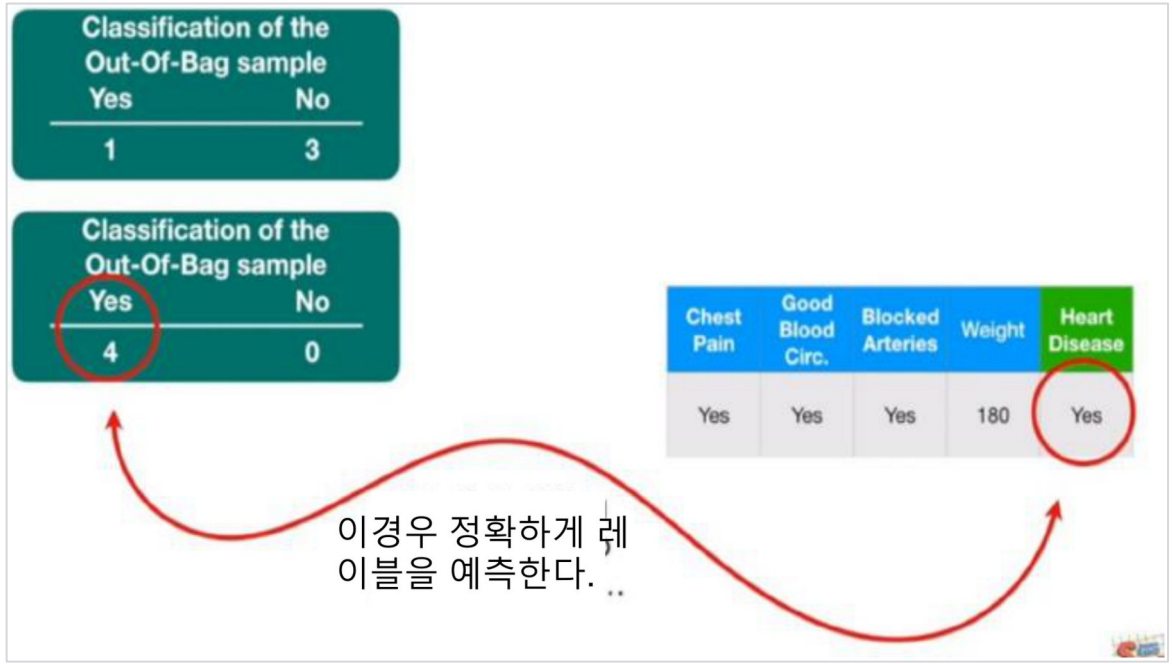
- OOB 샘플을 뺀 트리들의 다수결
- 이 경우 정확하게 레이블을 예측
- 같은 일을 모든 OOB 샘플에 반복



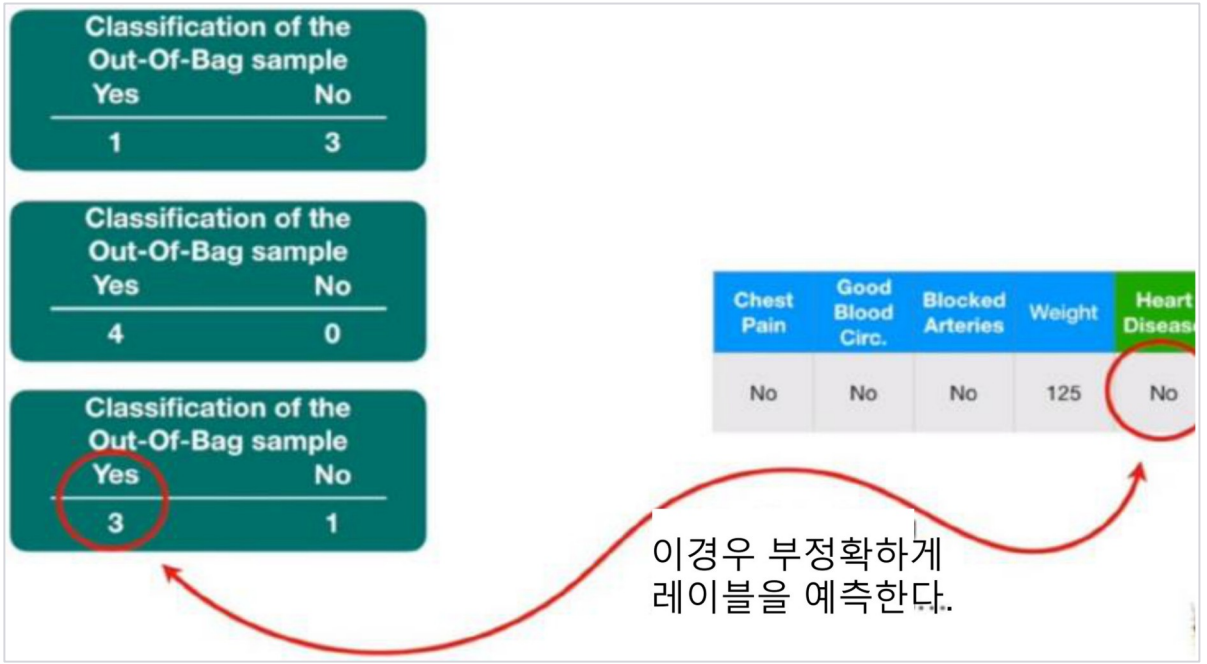
모든 OOB 샘플에 대해 반복한다

맞힐 때도, 틀릴 때도 있다

Yes 4 : No 0 → 정답 Yes (정확)



Yes 3 : No 1 → 정답 No (부정확)

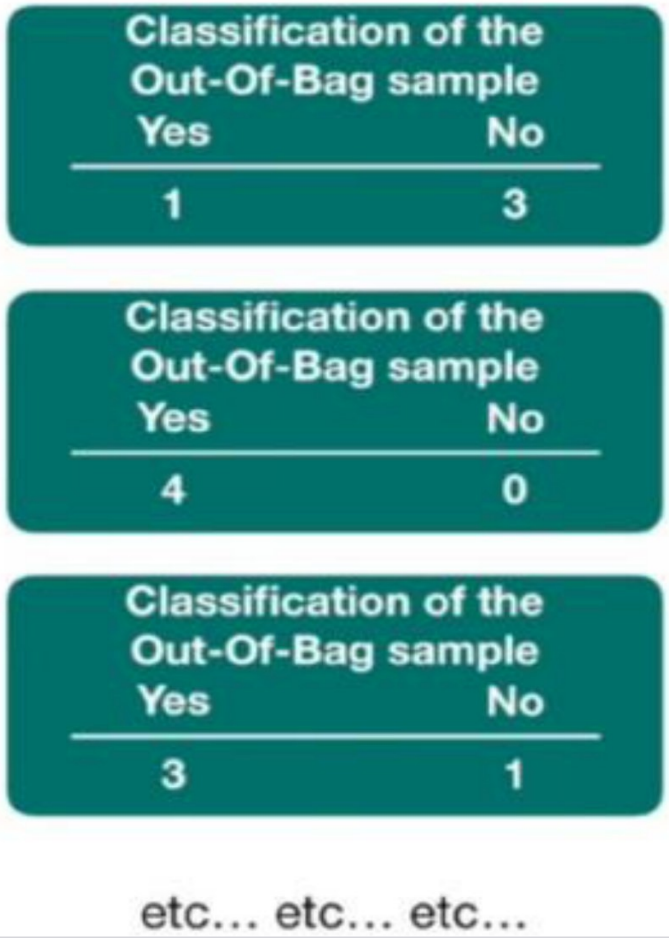


OOB 오차 = 오분류된 OOB 샘플의 비율

별도의 검증 세트 없이 RF의 정확도를 추정한다

정의와 활용

- OOB 정확도 = 정확히 분류된 OOB 샘플 비율
- OOB 오차 = 1 − OOB 정확도
- 트리 수 · max_features를 고를 때 쓴다
- OOB는 비관적인 편 → 가능하면 교차검증도



랜덤포레스트에 의해 정확하게 분류된 Out-of-Bag 샘플의 비율로 랜덤포레스트의 정확도를 측정할 수 있다.

오분류된 Out-of-Bag 샘플의 비율은 “Out-of-Bag 오차”라고 한다.

보강 — scikit-learn으로 RF 한 번에

유방암 데이터, 앞 트리 예제와 같은 분할 (scikit-learn 1.8 실행 결과)

Python · scikit-learn

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,      # 트리 수 (기본값)
    max_features="sqrt",   # 노드별 후보 특성 (기본값)
    oob_score=True,        # OOB 정확도 계산
    random_state=0, n_jobs=-1)
rf.fit(X_train, y_train)

rf.score(X_train, y_train)   # 1.000
rf.score(X_test, y_test)     # 0.958
rf.oob_score_                # 0.955
```

단일 트리와 비교

- 단일 트리 테스트 0.937 → RF 0.958
- OOB 0.955는 테스트 정확도와 거의 같다
- 트리를 끝까지 키워도 평균이 분산을 눌러 준다

03

3부 · 결측치 처리

결측치 — 트리와 랜덤 포레스트는 어떻게 다루나

단순 대체 · 상관 열 활용 · 회귀 대체 · RF 근접도(proximity) 반복 대체 · 새 샘플 처리

기본 지식 — 결측은 왜, 어떻게 생기나

결측 메커니즘에 따라 "대체해도 되는지"가 달라진다

유형	뜻	예	단순 대체
MCAR	완전 무작위 결측	입력 오류로 무작위 누락	편향 적음
MAR	관측된 다른 변수로 설명되는 결측	소형주일수록 애널리스트 추정치 없음	다른 열을 활용하면 보정 가능
MNAR	결측 자체가 값과 관련	실적 나쁜 기업이 공시 지연	편향 위험 — 결측 여부도 정보

금융 사례 — Bryzgalova · Lerner · Lettau · Pelger, "Missing Financial Data" (RFS 2025)

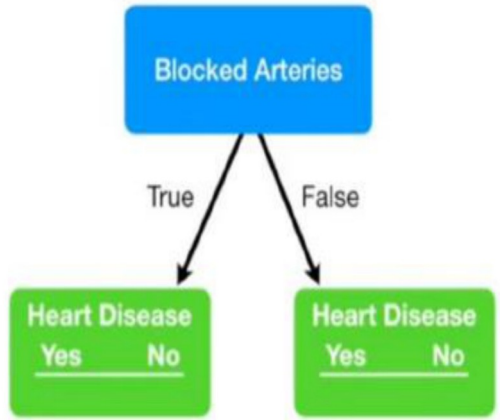
- 기업 특성 데이터의 결측은 시가총액 절반을 차지하는 70% 넘는 기업에 영향을 준다
- 결측 패턴이 체계적이라 평균 대체 같은 전통적 방법은 편향을 만든다

의사결정트리와 결측치

동맥경화(BA) 값이 "???"인 환자 — 앞에서는 집계에서 건너뛰었다

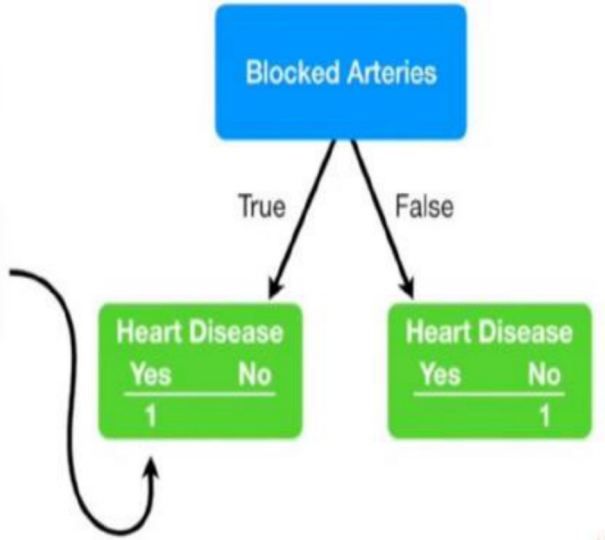
BA 기준 트리

Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...



값이 있는 환자는 정상 집계

Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...

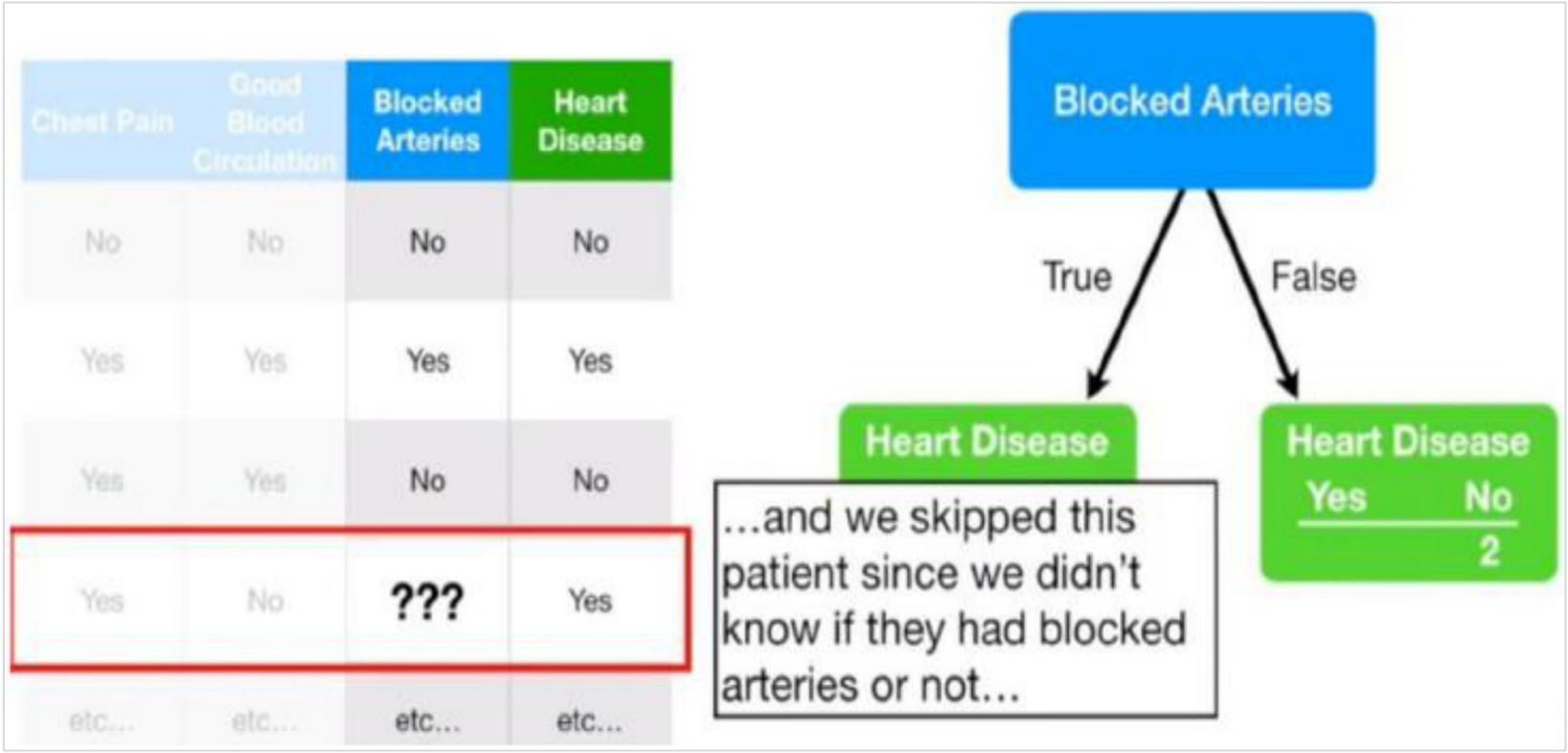


건너뛰는 것이 최선일까

"???" 환자를 빼면 그 환자의 다른 정보(CP, 심장병 등)도 버린다

대안

- 범주형: ① 최빈값 대체 ② 상관 높은 다른 열 활용
- 수치형: ① 평균 · 중위값 대체 ② 회귀로 추정
- 그림: StatQuest 강의 슬라이드 인용



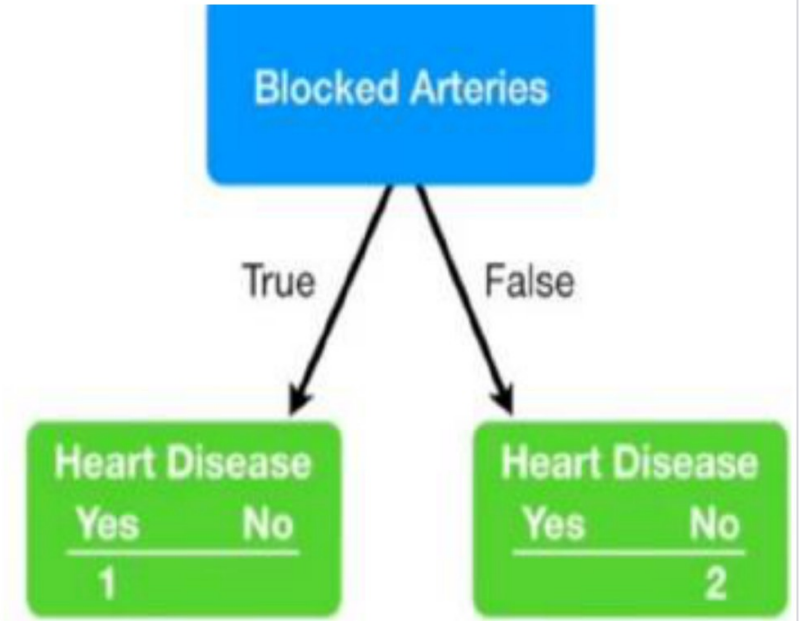
범주형 결측 ① 가장 빈도가 높은 값으로

동맥경화(BA) 열에서 가장 많이 나온 값으로 채운다

특징

- 가장 간단하다
- 다른 열의 정보를 쓰지 않는다
- 결측이 많으면 한 범주로 쏠린다

Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...



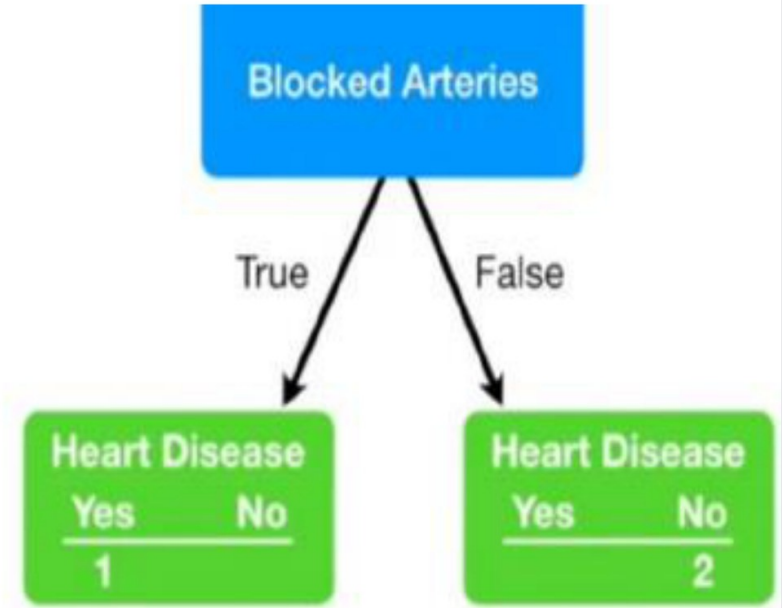
범주형 결측 ② 상관 높은 열을 가이드로

BA와 가장 상관이 높은 흉부통(CP)을 찾아 값을 따라간다

아이디어

- CP와 BA가 비슷하게 움직인다면
- CP = Yes인 환자의 BA는 Yes일 가능성이 높다
- → 다른 열(MAR)의 정보를 활용

Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
No	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...



범주형 결측 ② 적용 — CP 값을 따라 채우기

CP와 BA가 모두 No인 행, 모두 Yes인 행을 보고 규칙을 세운 뒤 "???"에 적용한다

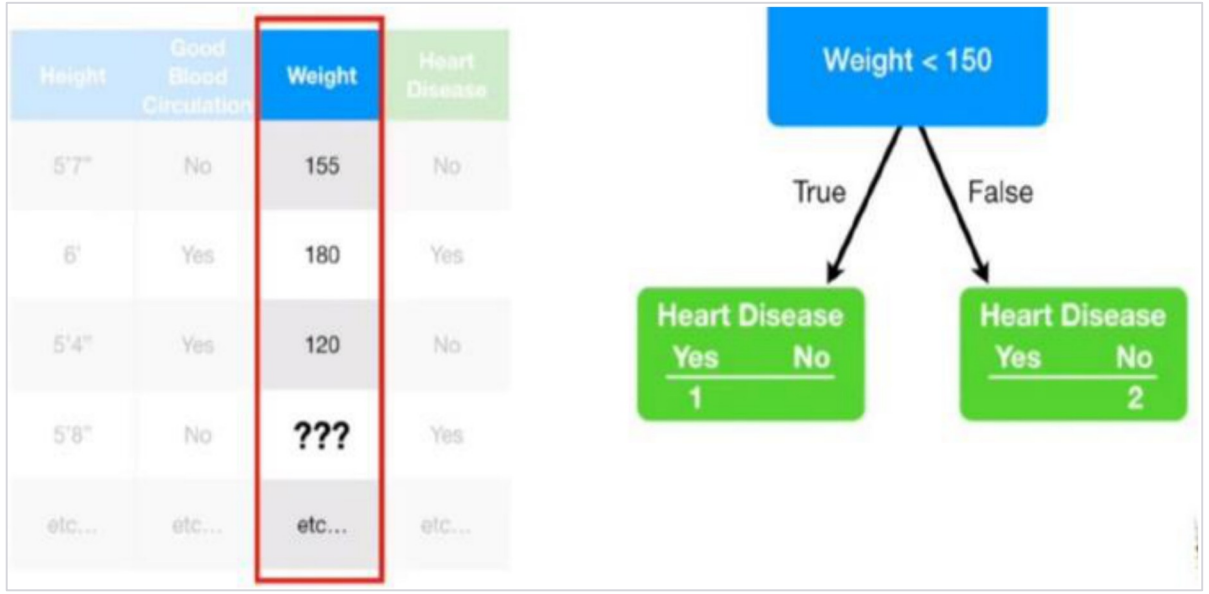


결측 환자의 CP = Yes → BA도 "Yes"로 채운다.

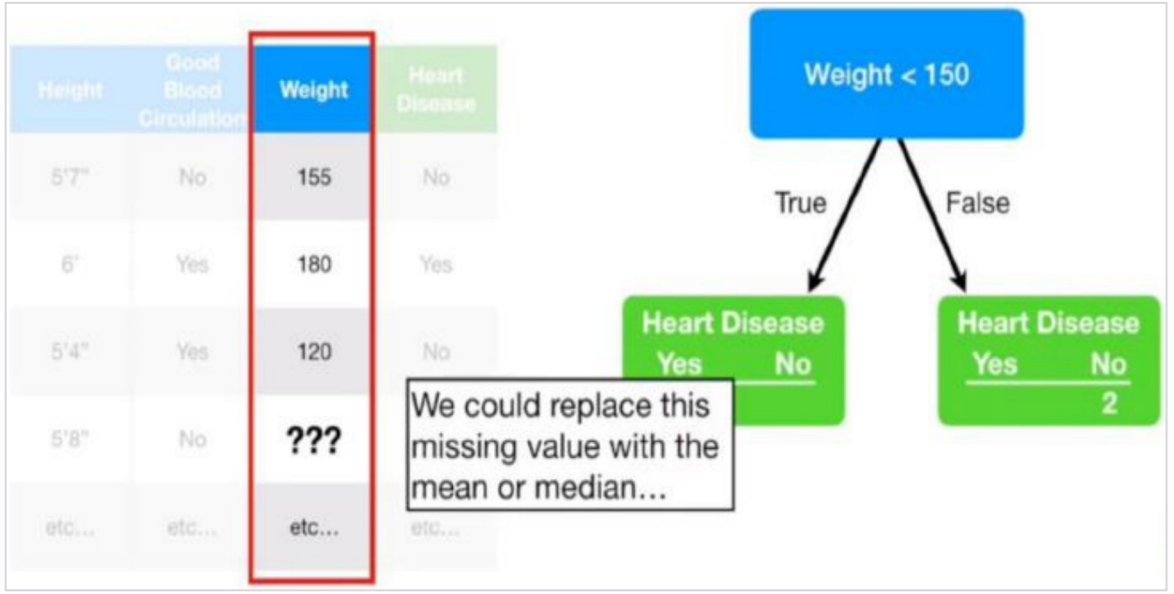
수치형 결측 ① 평균 또는 중위값

체중(Weight) 열의 "???"

체중 결측



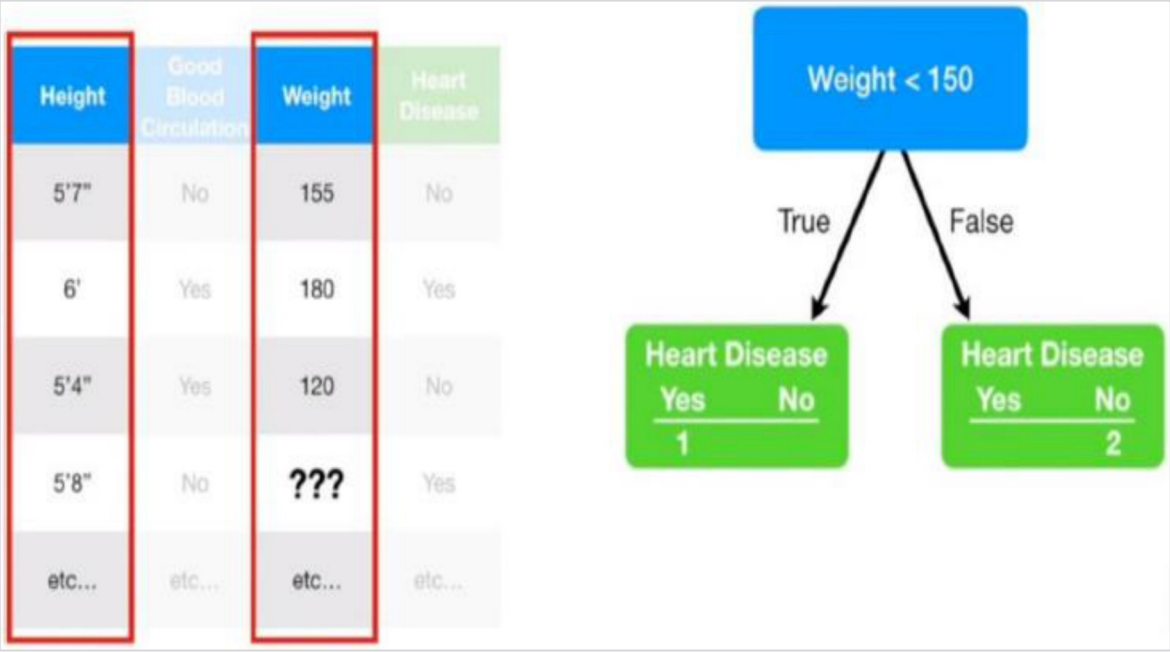
평균 · 중위값으로 대체



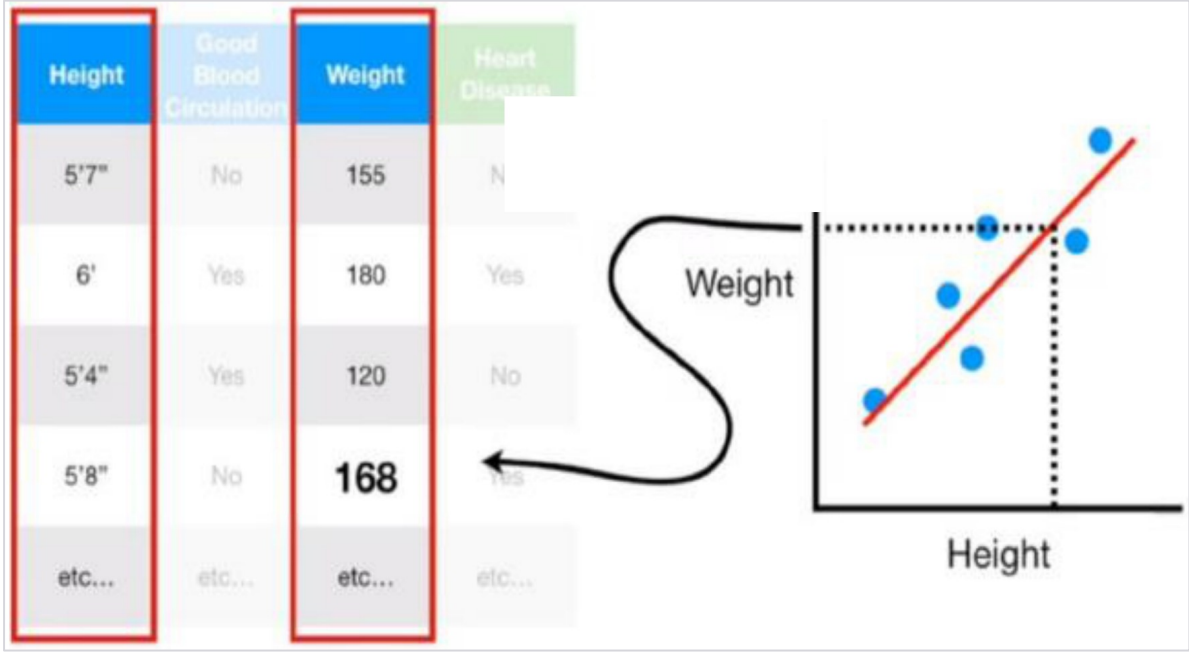
수치형 결측 ② 상관된 열로 회귀 추정

키(Height)와 체중이 상관 → 키로 체중을 회귀해 결측을 추정한다

키와 체중을 함께 본다



회귀선으로 168 추정



랜덤 포레스트의 결측치 — 두 가지 경우

원 데이터의 결측 · 새로 분류할 샘플의 결측

두 가지 경우

- ① RF를 만들 원 데이터에 결측이 있다 → 4번 환자의 BA · 체중
- ② 분류하려는 새 샘플에 결측이 있다
- ①은 근접도(proximity) 반복 대체, ②는 후보값 대입 후 투표
- 그림: StatQuest 강의 슬라이드 인용

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	???	???	No

Random Forests consider 2 types of missing data…

1) Missing data in the original dataset used to create the random forest.

2) Missing data in a new sample that you want to categorize.

New Sample

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	???	

① 원 데이터 — 먼저 대충 채운다

같은 클래스(심장병 No) 안에서 범주형은 최빈값, 수치형은 중위값

초기 추정

- BA: 심장병 No인 환자들의 최빈값 = "No"
- 체중: 같은 환자들의 중위값 = 167.5
- 이것은 출발점일 뿐 — 다음 단계에서 다듬는다



1단계 — 채운 데이터로 RF를 만든다

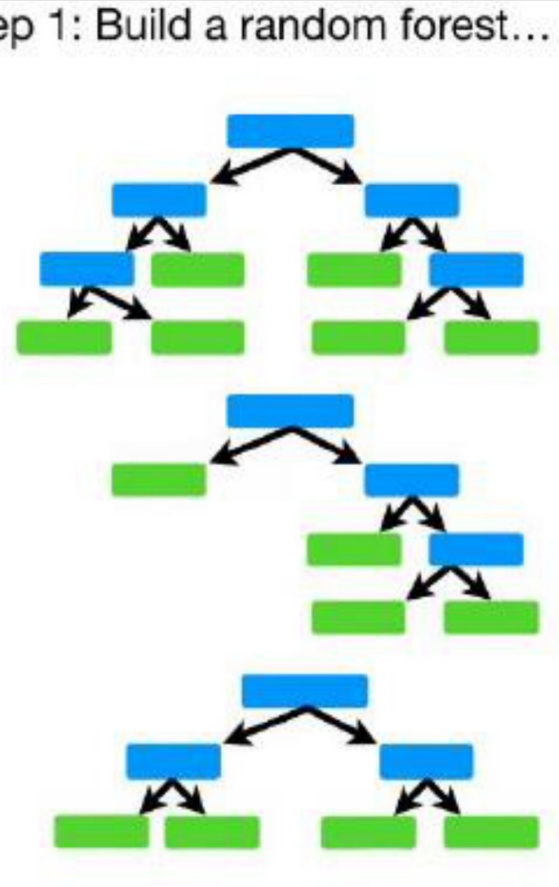
초기 추정값을 넣은 데이터셋으로 랜덤 포레스트를 학습

다음

- 목적은 예측이 아니라 "샘플 간 유사도" 측정
- 트리가 비슷한 샘플을 같은 리프로 보낸다는 성질을 이용

Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**167.5**	No



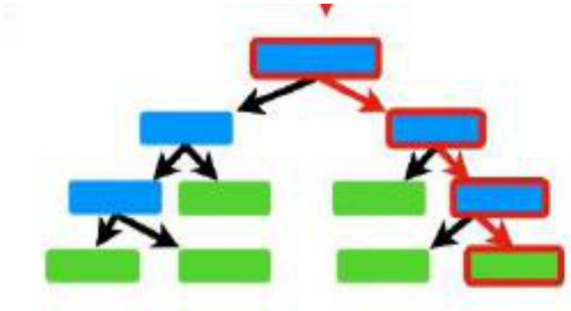
2단계 — 모든 데이터를 모든 트리에 통과시킨다

각 샘플이 어느 리프에 도착하는지 기록한다

샘플 1 · 2를 첫 트리에

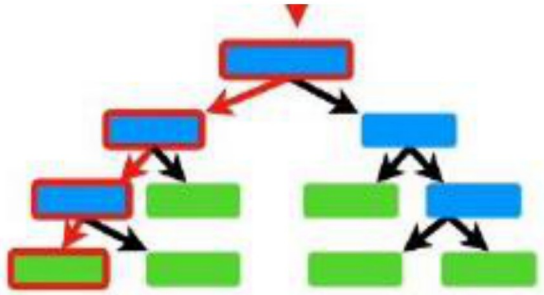
Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**167.5**	No



Filled-in Missing Values

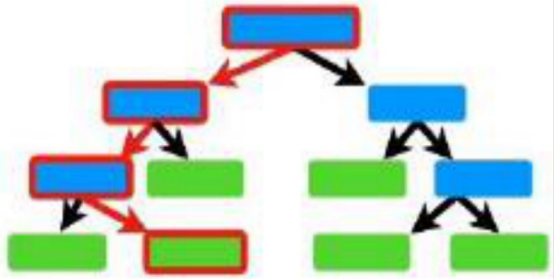
Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**167.5**	No



샘플 3 · 4도 같은 트리에

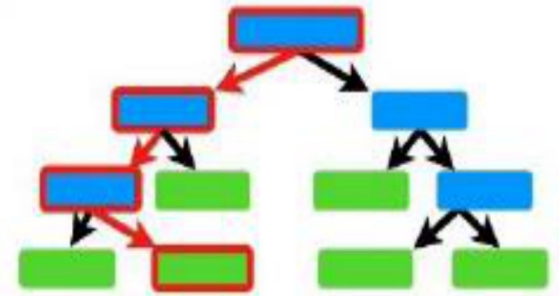
Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**167.5**	No



Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**167.5**	No

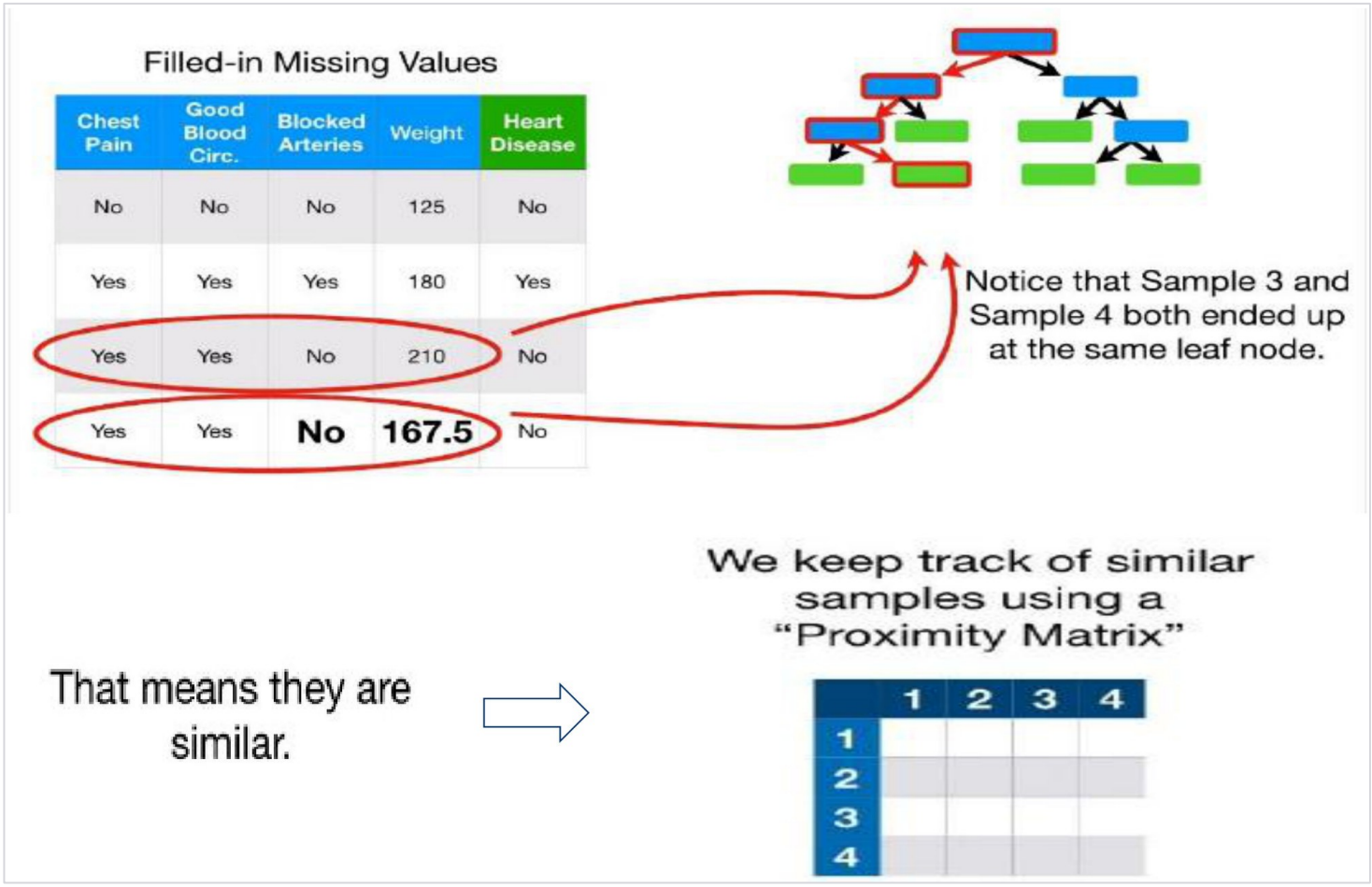


같은 리프에 도착한 샘플 = 비슷한 샘플

샘플 3과 4가 같은 리프 → 근접도 행렬(proximity matrix)에 기록

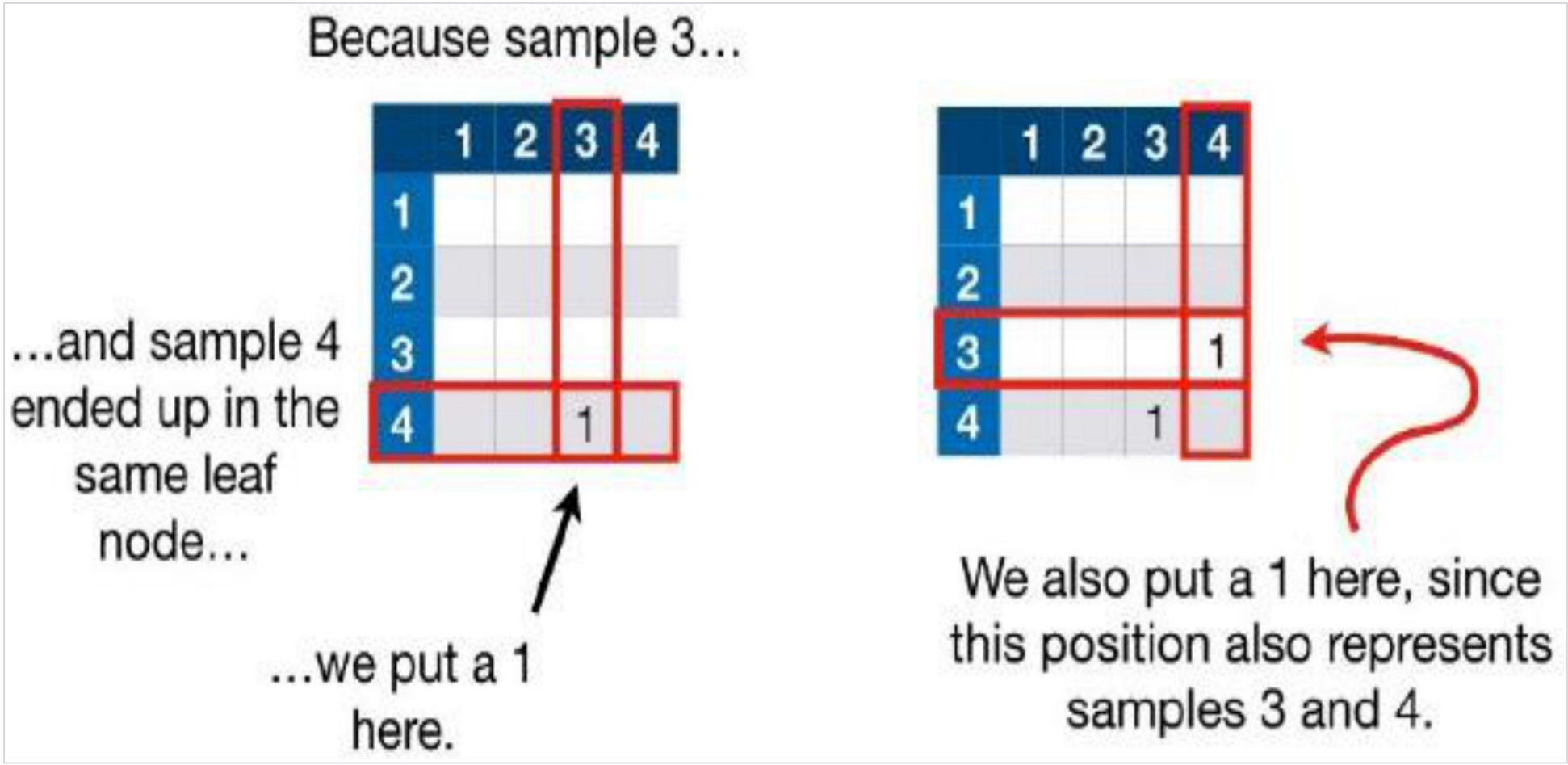
근접도 행렬

- 행 · 열 = 샘플 번호
- 같은 리프에 도착하면 해당 칸에 +1
- 트리를 지날 때마다 누적한다



근접도 행렬에 1을 더한다

(3, 4)와 (4, 3) 두 칸 모두 같은 쌍을 뜻한다



두 번째 · 세 번째 트리에서도 누적

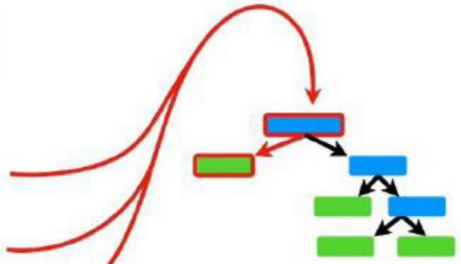
트리를 지날 때마다 같은 리프에 도착한 쌍에 1씩 더한다

트리 2 이후: 샘플 2 · 3 · 4

Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**167.5**	No

NOTE: Samples 2, 3 and 4 all ended up in the same leaf node.



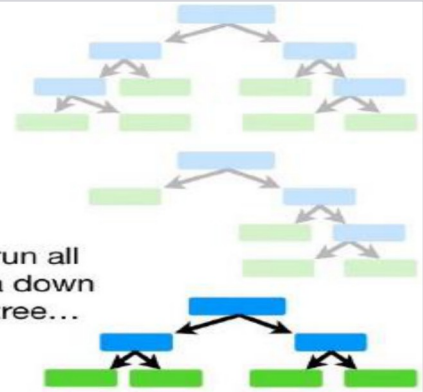
...after the second tree, we add 1 to any pair of samples that ended up in the same leaf node.

	1	2	3	4
1				
2			1	1
3		1		2
4		1	2	

트리 3 이후: (3, 4) = 3

Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**180**	No



Now we run all of the data down the third tree...

...and here's the updated proximity matrix.

	1	2	3	4
1				
2			1	1
3		1		3
4		1	3	

Only samples 3 and 4 ended up in the same leaf node.

모든 트리를 지나면 → 트리 수로 나눈다

예: 트리 10개 → (3, 4) = 8/10 = 0.8

근접도의 정의

- 0 ~ 1 사이 값
- 1에 가까울수록 두 샘플은 트리 입장에서 비슷하다

Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**167.5**	No

Ultimately, we run the data down all the trees and the proximity matrix fills in.

	1	2	3	4
1		2	1	1
2	2		1	1
3	1	1		8
4	1	1	8	

	1	2	3	4
1		0.2	0.1	0.1
2	0.2		0.1	0.1
3	0.1	0.1		0.8
4	0.1	0.1	0.8	

Then we divide each proximity value by the total number of trees. In this example, assume we had 10 trees.

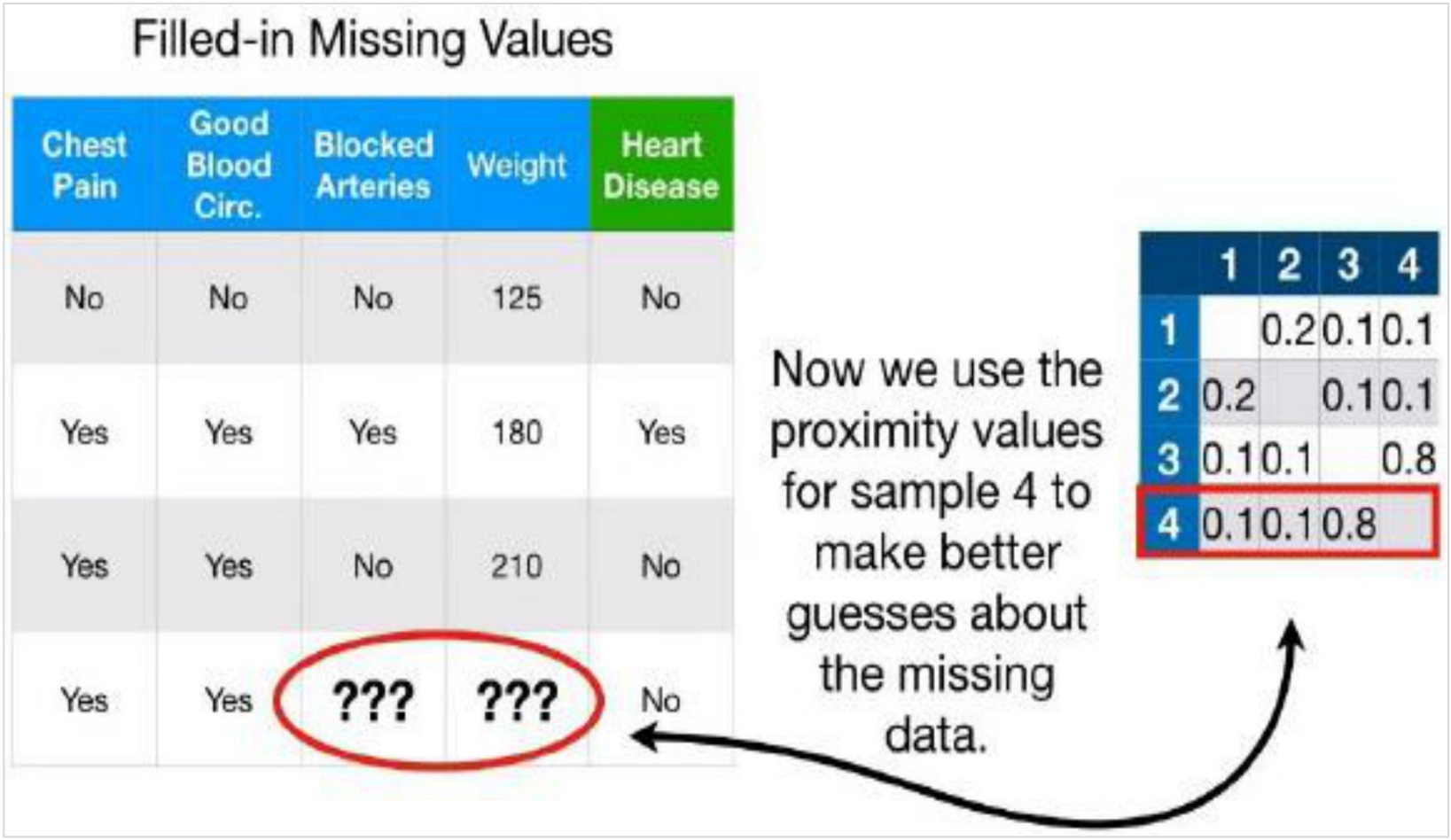
$$\text{prox}(i,j) = \frac{1}{B}\sum_{b=1}^{B} \mathbf{1}\{\text{leaf}_b(i) = \text{leaf}_b(j)\}$$

근접도로 결측값을 다시 추정한다

샘플 4의 행: (1) 0.1 · (2) 0.1 · (3) 0.8

아이디어

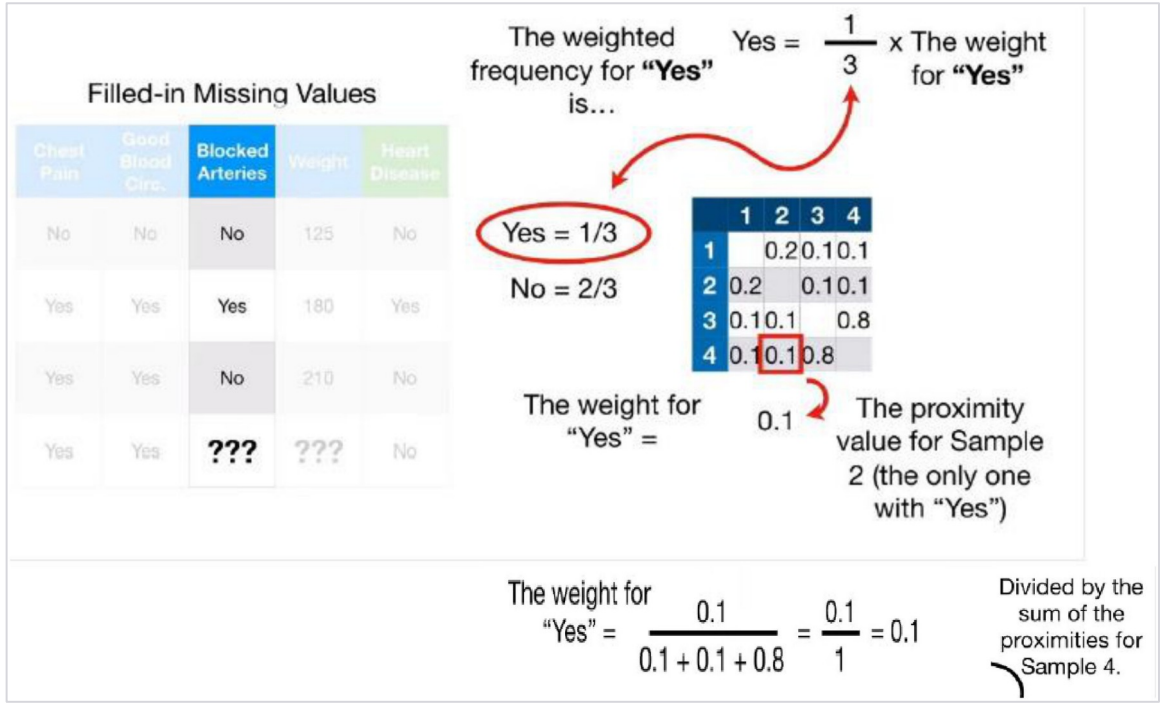
- 결측이 있는 샘플 4와 가까운 샘플의 값을 더 믿는다
- = 근접도로 가중한 투표(범주형) · 가중평균(수치형)



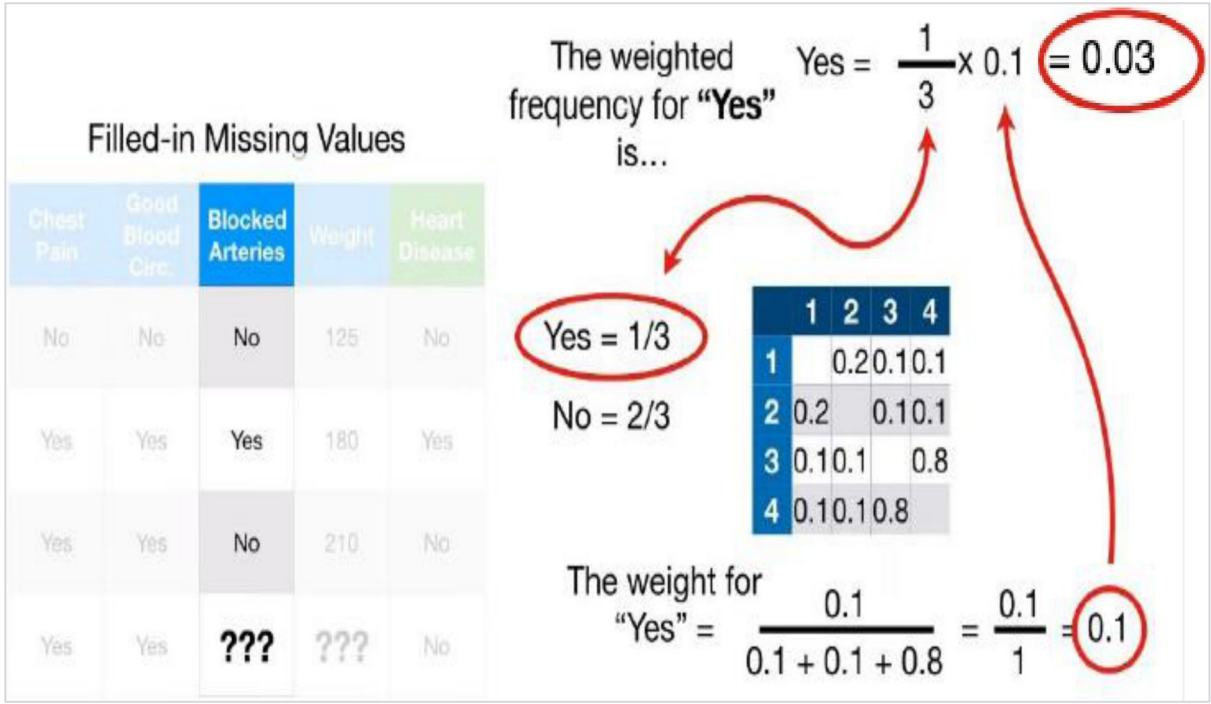
범주형(BA): 근접도로 가중한 빈도

Yes의 가중빈도 = Yes 비율 1/3 × Yes 가중치 0.1

Yes 가중치 = 0.1 / (0.1 + 0.1 + 0.8)

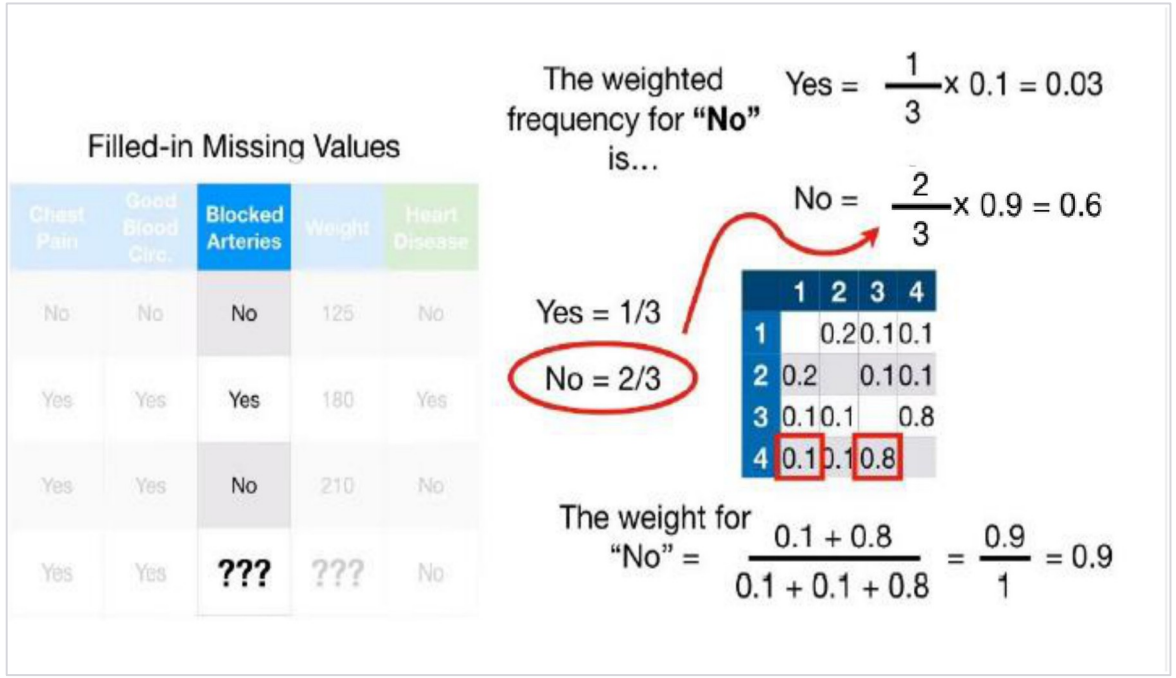


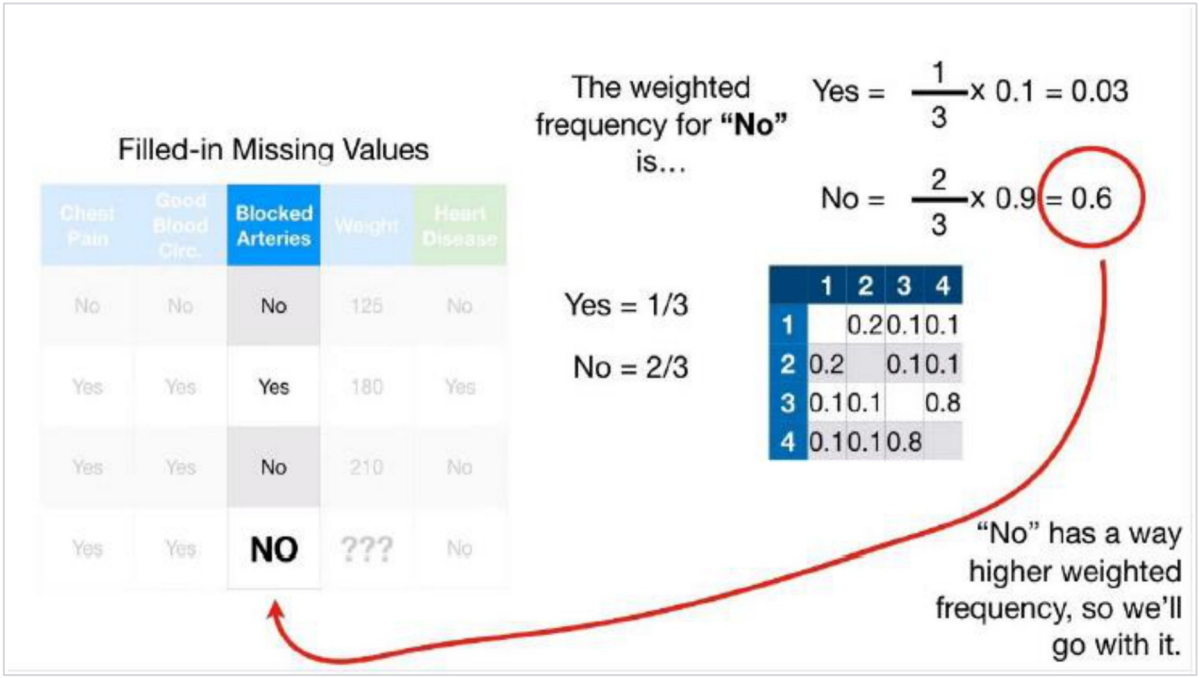
Yes 가중빈도 = 0.03



No의 가중빈도가 훨씬 크다 → "No"

No 가중빈도 = 2/3 × 0.9 = 0.6





$$\text{Yes}: \frac{1}{3} \times \frac{0.1}{1.0} = 0.03 \qquad \text{No}: \frac{2}{3} \times \frac{0.1+0.8}{1.0} = 0.6$$

보강 — 근접도 가중빈도 · 가중평균의 식

범주형은 가중 빈도가 가장 큰 값, 수치형은 근접도 가중평균

$$\tilde{f}_i(c) = \Pr(c) \times \frac{\sum_{j:\, x_j = c} \mathrm{prox}(i,j)}{\sum_{j \neq i} \mathrm{prox}(i,j)}$$

$$\hat{x}_i = \sum_{j \neq i} x_j \frac{\mathrm{prox}(i,j)}{\sum_{k \neq i} \mathrm{prox}(i,k)}$$

범주형 (BA)

- Pr(c): 해당 열에서 값 c의 비율
- 분수: 값이 c인 샘플들과의 근접도 합 / 전체 근접도 합
- 가장 큰 c를 채운다 → "No"

수치형 (체중)

- 다른 샘플 값에 근접도 비율을 곱해 더한다
- 샘플 3이 0.8로 가장 가까워 210이 크게 반영
- 결과: 198.5 (다음 장)

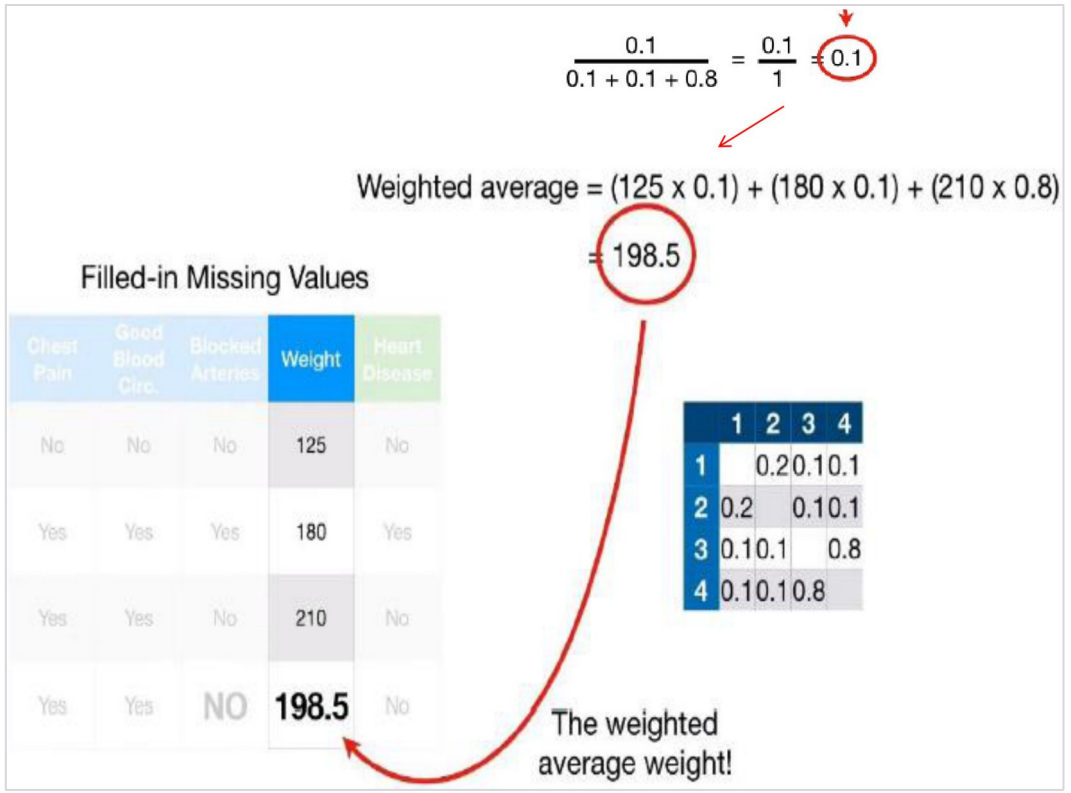
수치형(체중): 근접도 가중평균

125 × 0.1 + 180 × 0.1 + 210 × 0.8 = 198.5

근접도로 가중평균



가중치 0.1 / 0.1 / 0.8 → 198.5



이 과정을 수렴할 때까지 반복한다

RF를 다시 만들고 → 근접도 다시 계산 → 결측값 다시 추정

반복

- 보통 6~7번이면 값이 더 이상 바뀌지 않는다
- Breiman의 원 RF 구현(R: rfImpute)이 이 방식
- 비슷한 발상: missForest(2012) — 열마다 RF로 반복 대체

Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**NO**	**198.5**	No

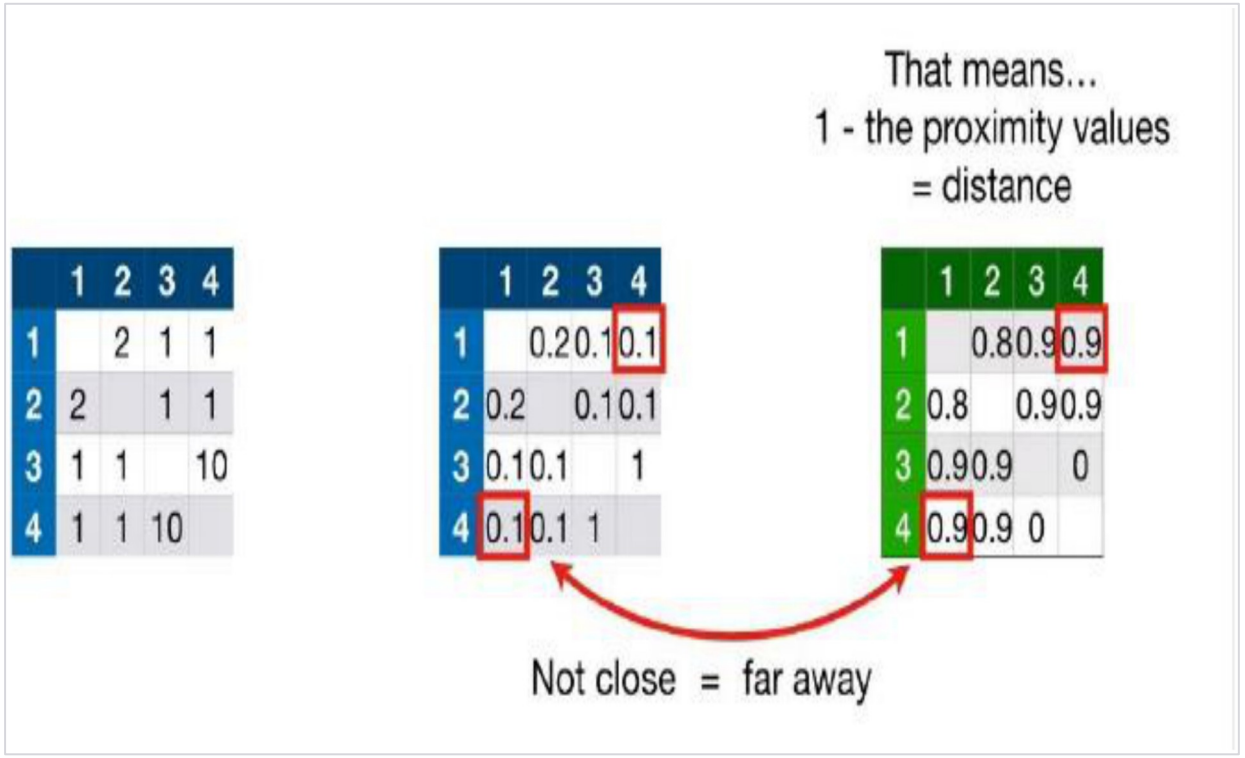
Now that we've revised our guesses a little bit, we do the whole thing over again...

We build a random forest, run the data through the trees, recalculate the proximities and recalculate the missing values.

We do this 6 or 7 times until the missing values converge (i.e. no longer change each time we recalculate).

덤 — 1 – 근접도 = 거리

근접도가 낮은 쌍은 멀리 떨어진 쌍이다

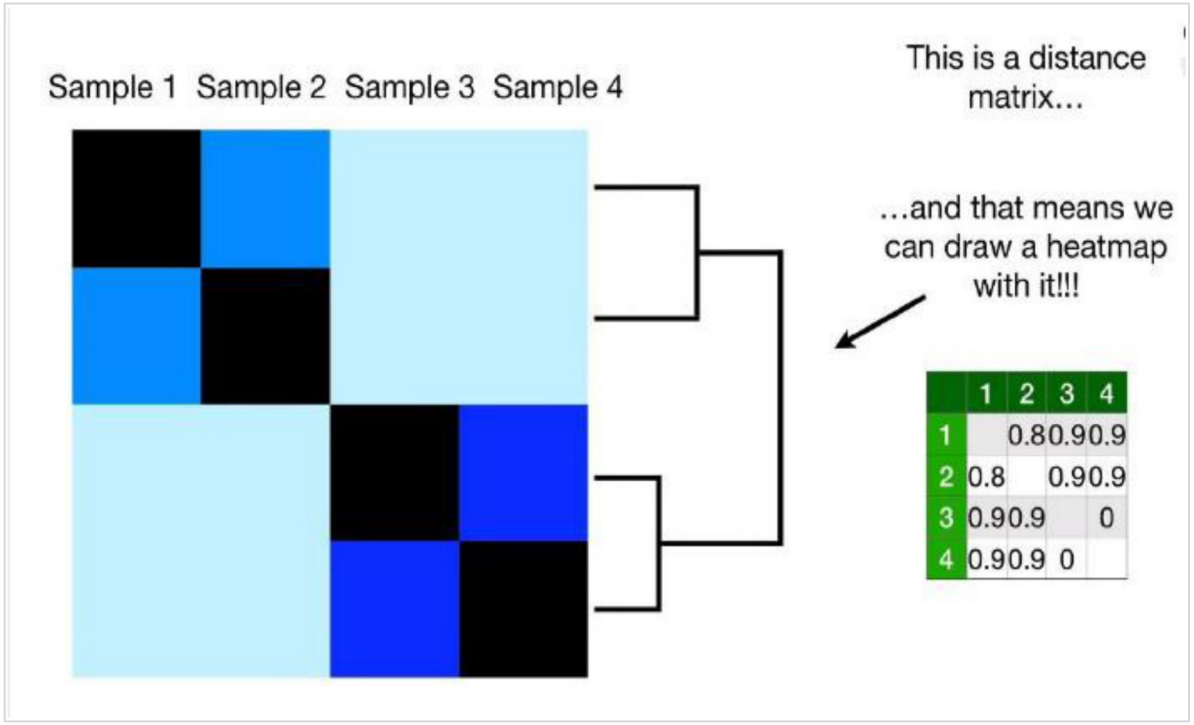


$$d(i,j) = 1 - \text{prox}(i,j)$$

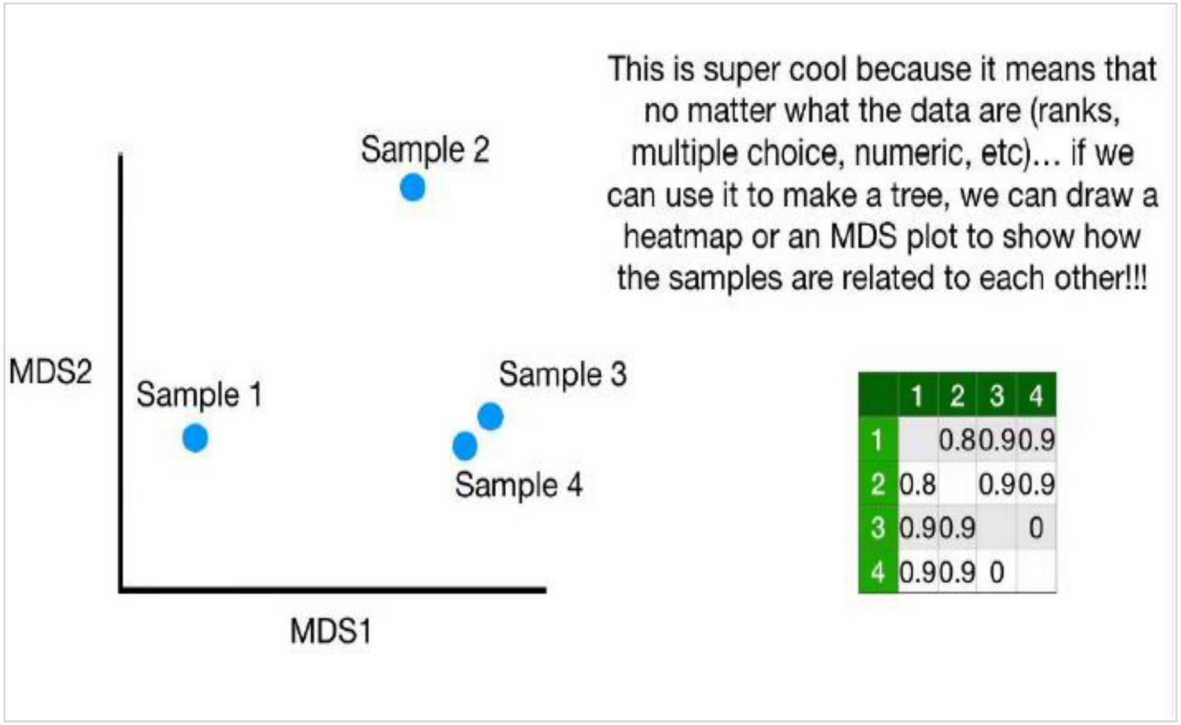
거리 행렬로 히트맵 · MDS 그림을 그린다

데이터가 순위든 범주든 수치든, 트리를 만들 수 있으면 샘플 간 관계를 시각화할 수 있다

거리 행렬 히트맵 + 계층 군집



MDS 평면: 샘플 3 · 4가 가깝다

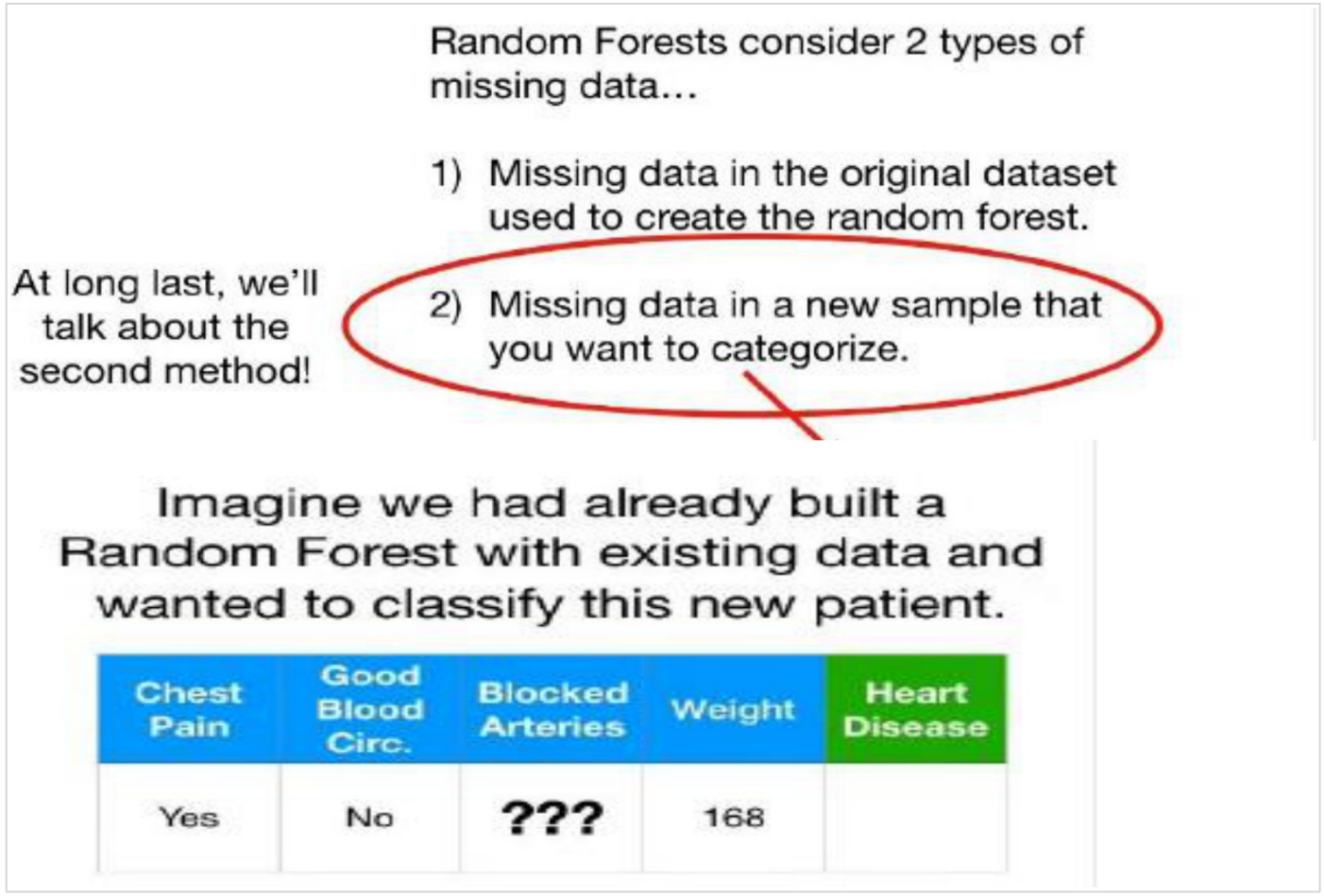


② 새 샘플에 결측이 있을 때

이미 만든 RF로 BA가 "???"인 새 환자를 분류하려 한다

상황

- RF는 기존 데이터로 이미 학습됨
- 새 환자: CP Yes · BC No · BA ??? · 체중 168
- 타깃(심장병)도 모른다

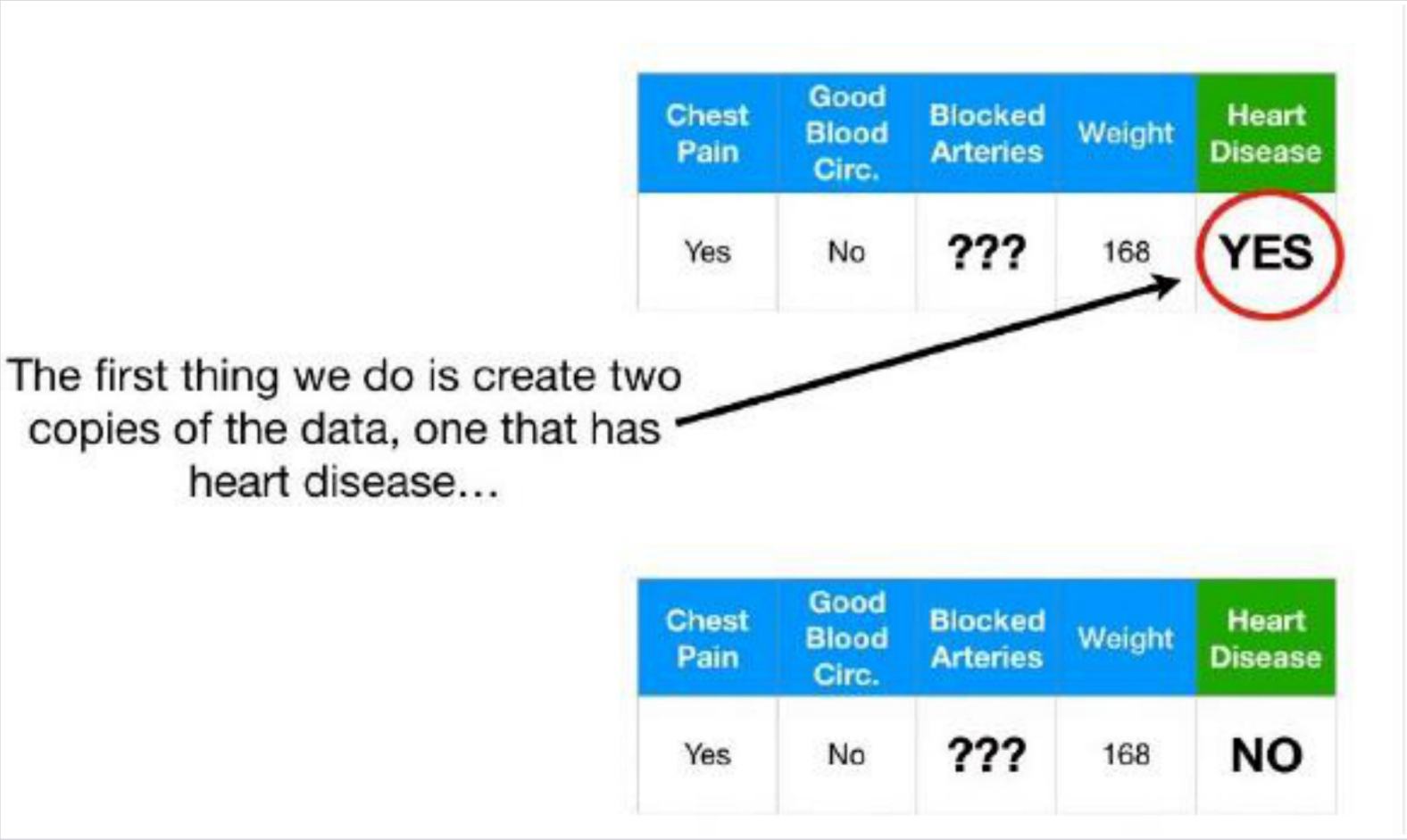


1. 타깃 후보별로 복사본을 만든다

심장병 = Yes인 복사본, 심장병 = No인 복사본

절차

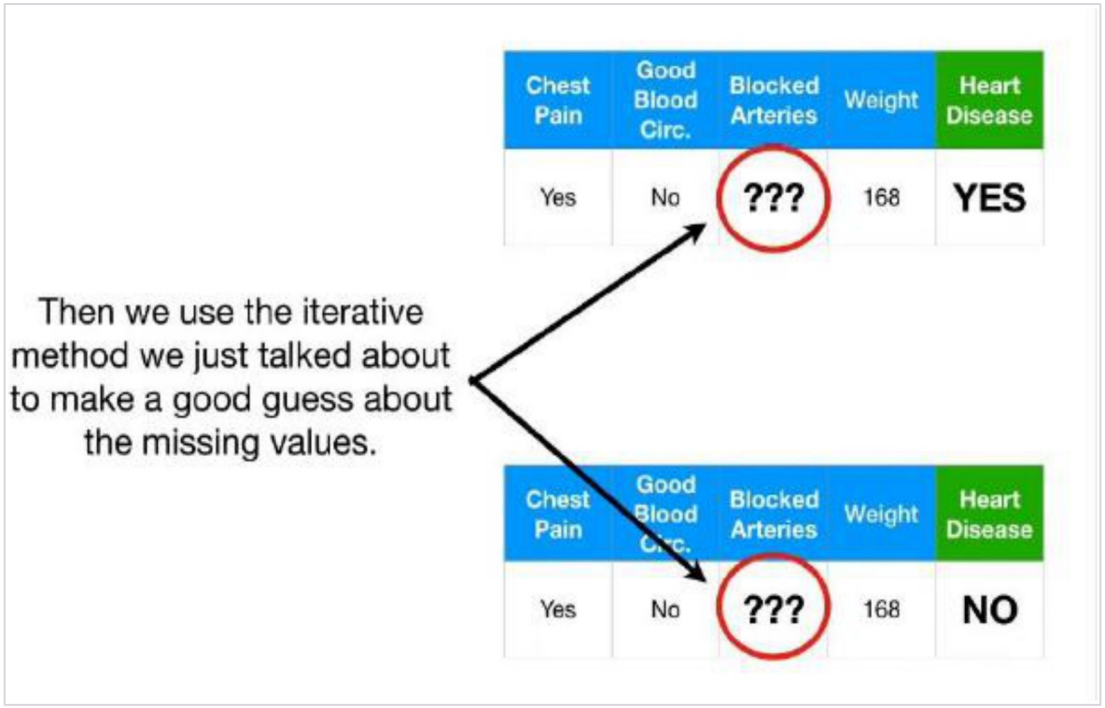
- 가능한 레이블마다 한 벌씩
- 각 복사본에서 결측 BA를 따로 추정한다



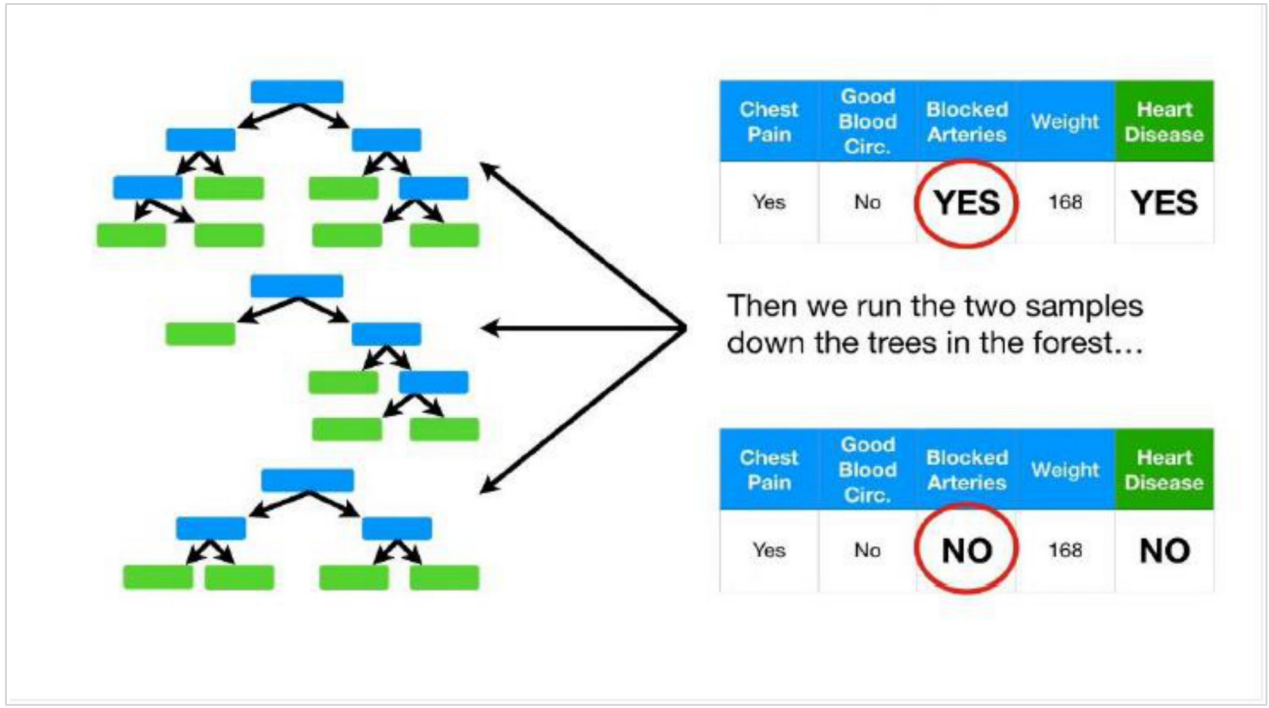
2. 각 복사본의 결측을 추정한다

앞의 반복 방법으로 추정 — 심장병 Yes인 경우 BA도 Yes일 가능성이 크다

반복 방법으로 추정



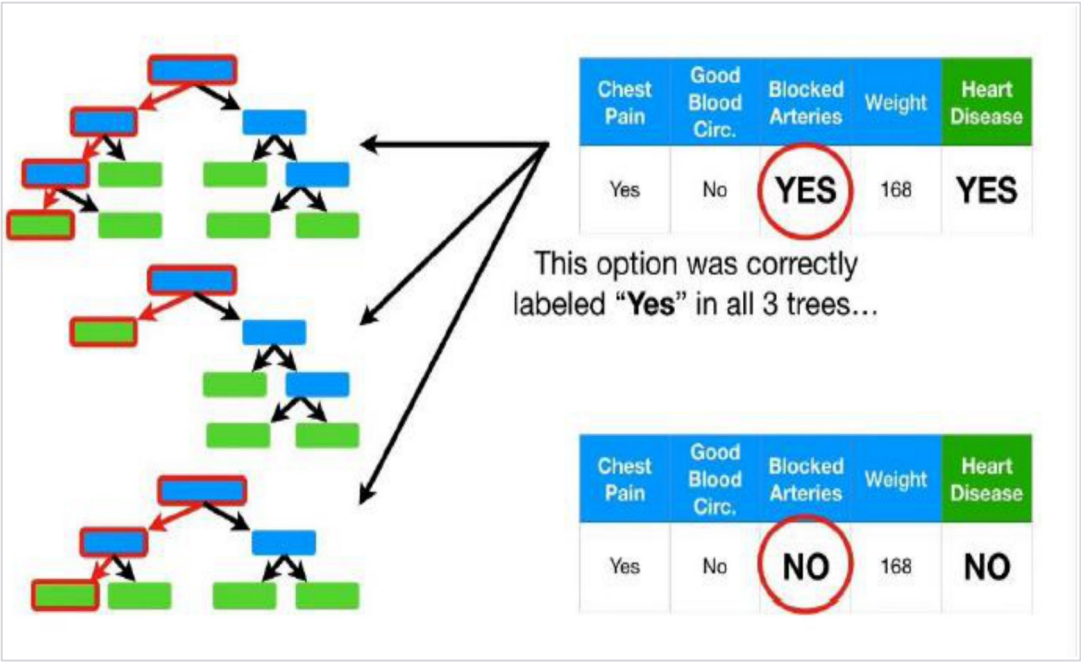
Yes 복사본 → BA Yes, No 복사본 → BA No



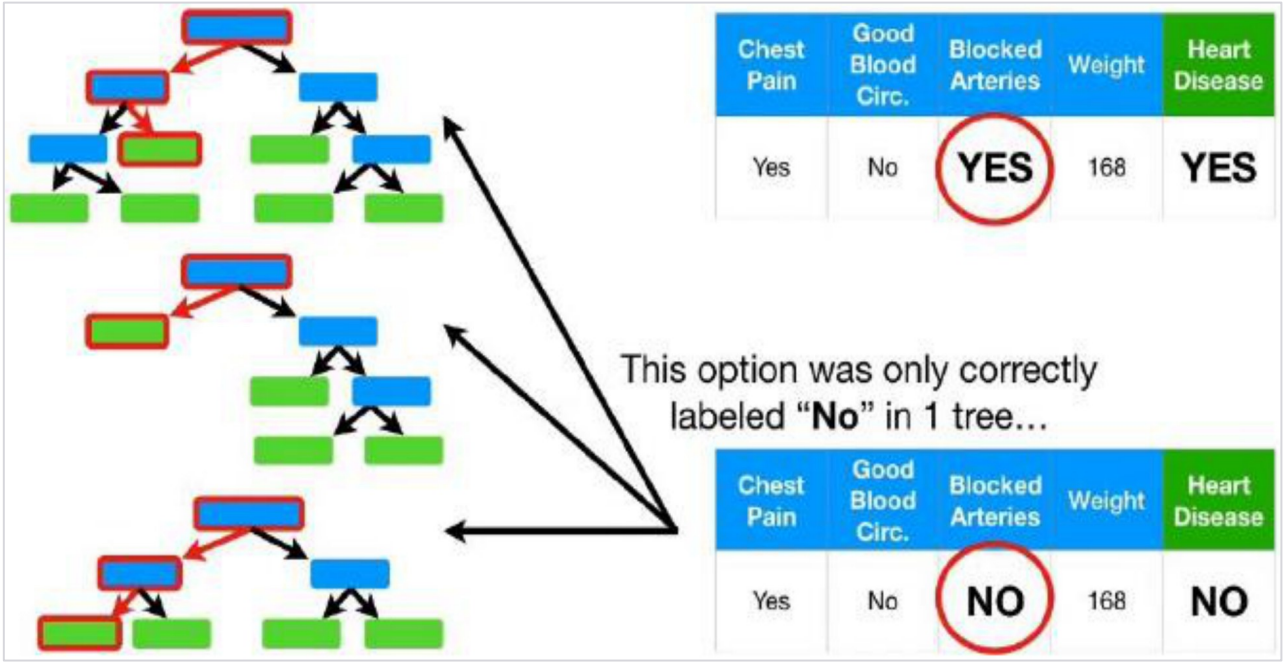
3. 두 복사본을 모든 트리에 통과시켜 비교

어느 복사본이 더 자주 "자기 레이블대로" 분류되는가

Yes 복사본: 3개 트리 모두 Yes



No 복사본: 1개 트리만 No

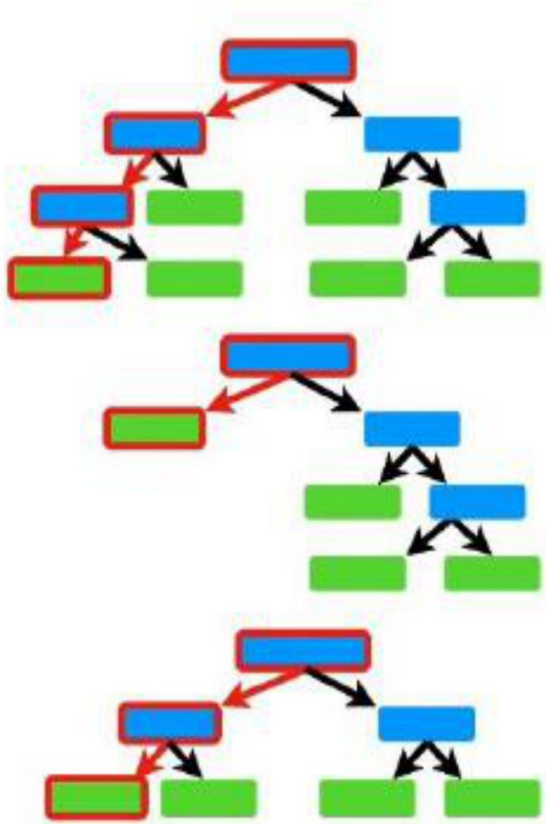


더 많이 맞힌 복사본이 이긴다

새 환자: 심장병 Yes, BA Yes로 결론

결론

- Yes 복사본이 더 많이 정확하게 분류됨
- → 이 옵션을 채택
- 결측을 채우는 동시에 분류까지 끝낸다



Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	No	YES	168	YES

This option wins because it was correctly labeled more than the other option.

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	No	NO	168	NO

보강 — 오늘날 라이브러리는 결측을 이렇게 다룬다

scikit-learn 트리 계열은 NaN을 받아 분할마다 결측이 갈 방향(왼쪽/오른쪽)을 학습한다

모델	결측 처리	도입
DecisionTree (scikit-learn)	분할마다 NaN 방향을 이득 기준으로 학습	1.3
RandomForest (scikit-learn)	트리마다 같은 방식, 학습 중 결측 없던 특성은 샘플 많은 쪽으로	1.4
ExtraTrees (scikit-learn)	결측 지원 추가	1.6
HistGradientBoosting	처음부터 NaN 기본 지원	—
XGBoost · LightGBM	기본 방향(default direction)을 학습 — sparsity-aware split	—

실무 주의

- 대체(imputation)는 교차검증 폴드 안에서 적합해야 누수(leakage)가 없다
- 결측 여부 자체를 지시변수(indicator)로 추가하는 방법도 흔하다

보강 — 결측 20% 주입 실험

유방암 데이터 훈련·테스트 특성에 무작위로 NaN 20%를 넣고 비교 (scikit-learn 1.8)

Python · scikit-learn

```
# X_train_m, X_test_m: 20%가 NaN
rf  = RandomForestClassifier(random_state=0)
rf.fit(X_train_m, y_train)              # NaN 그대로

imp = make_pipeline(SimpleImputer(strategy="median"),
                    RandomForestClassifier(random_state=0))
imp.fit(X_train_m, y_train)

hgb = HistGradientBoostingClassifier(random_state=0)
hgb.fit(X_train_m, y_train)
```

방법	테스트 정확도
RF (NaN 기본 지원)	0.951
중위값 대체 + RF	0.951
HistGB (NaN 기본 지원)	0.972

읽는 법

- 무작위 결측(MCAR)에서는 방법 간 차이가 작다
- 체계적 결측이면 결과가 달라질 수 있다

04

4부 · AdaBoost

AdaBoost — 틀린 샘플에 가중치를 몰아준다

약학습기(stump) · 가중 오차 ε · 발언권 α · 가중치 갱신 · 최종 가중 투표

기본 지식 — 부스팅과 약학습기

약학습기(weak learner): 무작위 추측보다 조금만 나은 모델. AdaBoost의 기본은 깊이 1 트리(stump)

① 같은 가중치로 시작

모든 샘플 가중치 1/N. 첫 stump를 학습한다.

② 틀린 샘플 가중치 ↑

오분류된 샘플은 가중치를 올리고 맞힌 샘플은 내린다.

③ 다음 stump는 어려운 샘플에 집중

가중치가 큰 샘플을 맞히는 방향으로 학습한다.

④ 가중 투표

잘 맞힌 stump일수록 큰 발언권 α를 받아 최종 결정에 참여한다.

RF의 트리는 서로 독립 · 동등한 한 표 / AdaBoost의 stump는 순차 · 성능에 따라 다른 발언권

보강 — AdaBoost 알고리즘 (이진 분류, y ∈ {−1, +1})

원 강의 그림의 ε와 α가 어디서 나오는지 식으로 정리

$$\varepsilon_t = \sum_{i=1}^{N} w_i^{(t)} \, \mathbf{1}\{h_t(x_i) \neq y_i\}$$

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}$$

$$w_i^{(t+1)} = \frac{w_i^{(t)} \, \exp\big(- \alpha_t \, y_i \, h_t(x_i) \big)}{Z_t}$$

$$H(x) = \operatorname{sign}\Big(\sum_{t=1}^{T} \alpha_t \, h_t(x) \Big)$$

가중 오차 → 발언권

- ε가 작을수록 α가 크다 (ε = 0.5면 α = 0)
- 틀린 샘플 가중치 ↑, 맞힌 샘플 ↓ → 합이 1이 되게 정규화

최종 분류기

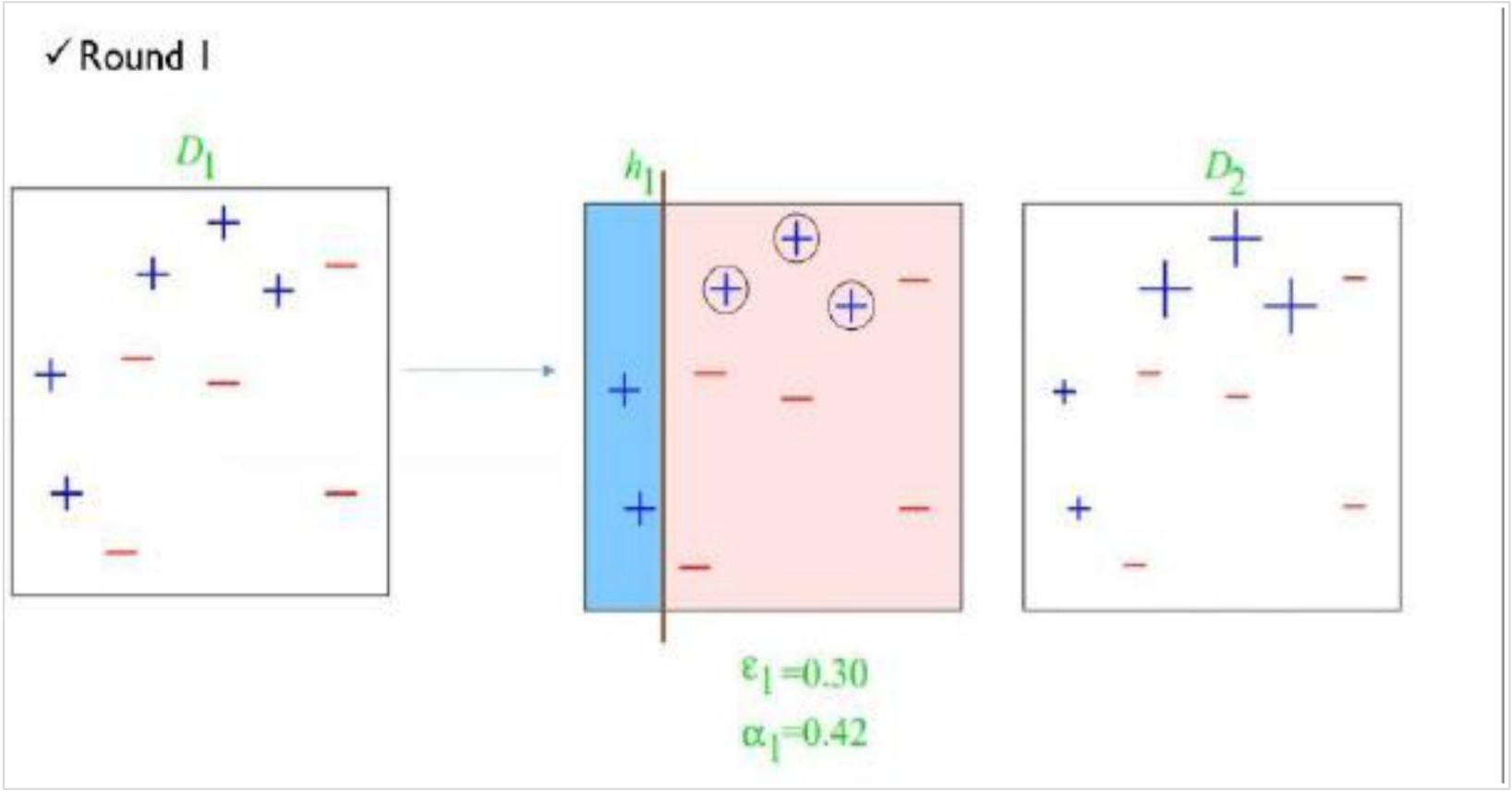
- stump 예측(±1) × α를 더해 부호로 판정
- Freund · Schapire(1997), 2003년 괴델상

Example 1 — Round 1

D_1: 10개 샘플 모두 같은 가중치 → 첫 stump h_1은 세로선 하나

읽는 법

- h_1은 + 세 개(동그라미)를 틀렸다
- $\varepsilon_1 = 0.30 \rightarrow \alpha_1 = 0.42$
- D_2: 틀린 + 세 개가 크게 그려짐 = 가중치 ↑

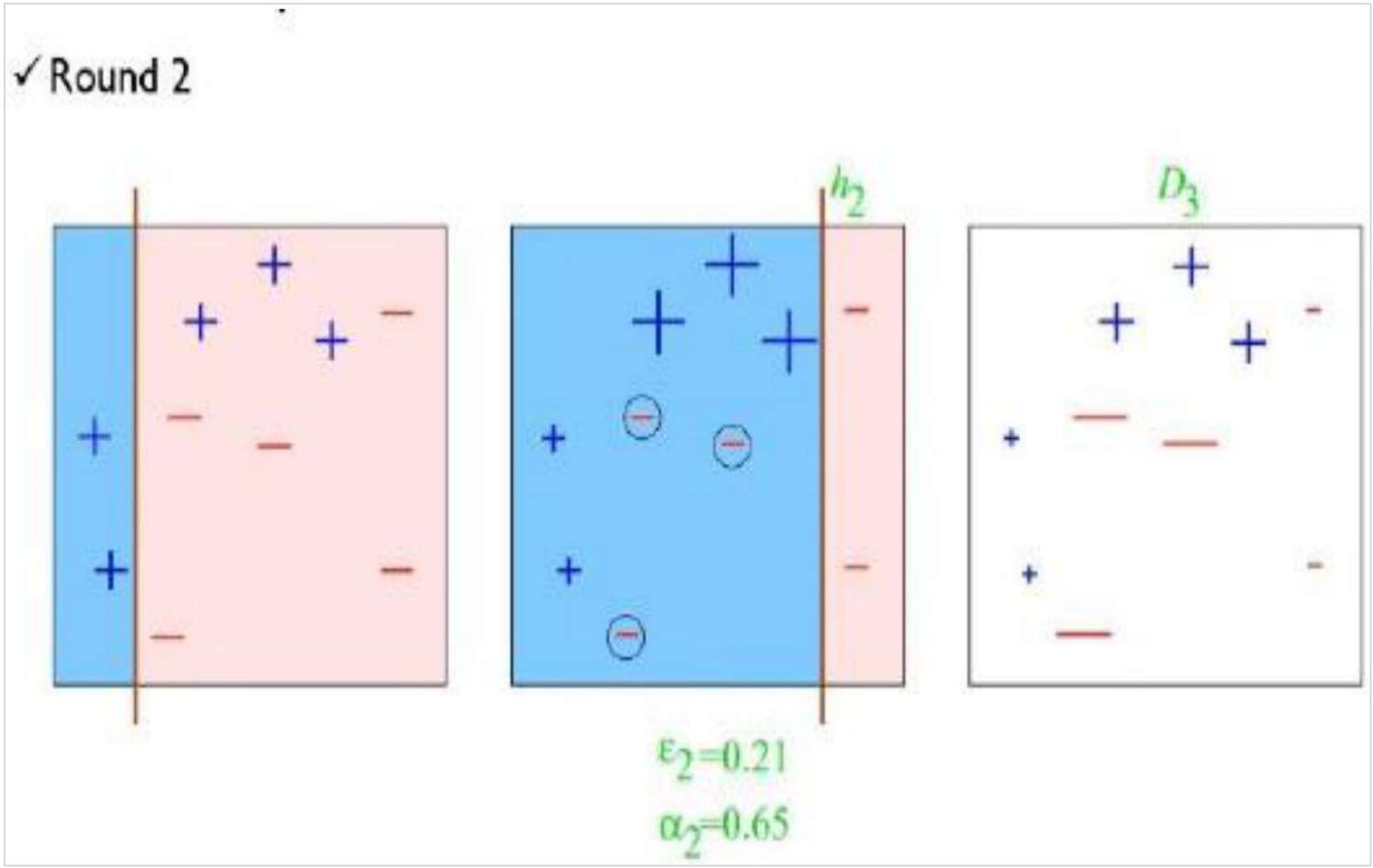


Example 1 — Round 2

가중치가 커진 + 들을 맞히도록 두 번째 stump h_2가 오른쪽 세로선을 긋는다

읽는 법

- h_2는 − 세 개(동그라미)를 틀렸다
- $\varepsilon_2 = 0.21 \rightarrow \alpha_2 = 0.65$
- D_3: 이번엔 그 − 세 개의 가중치 ↑

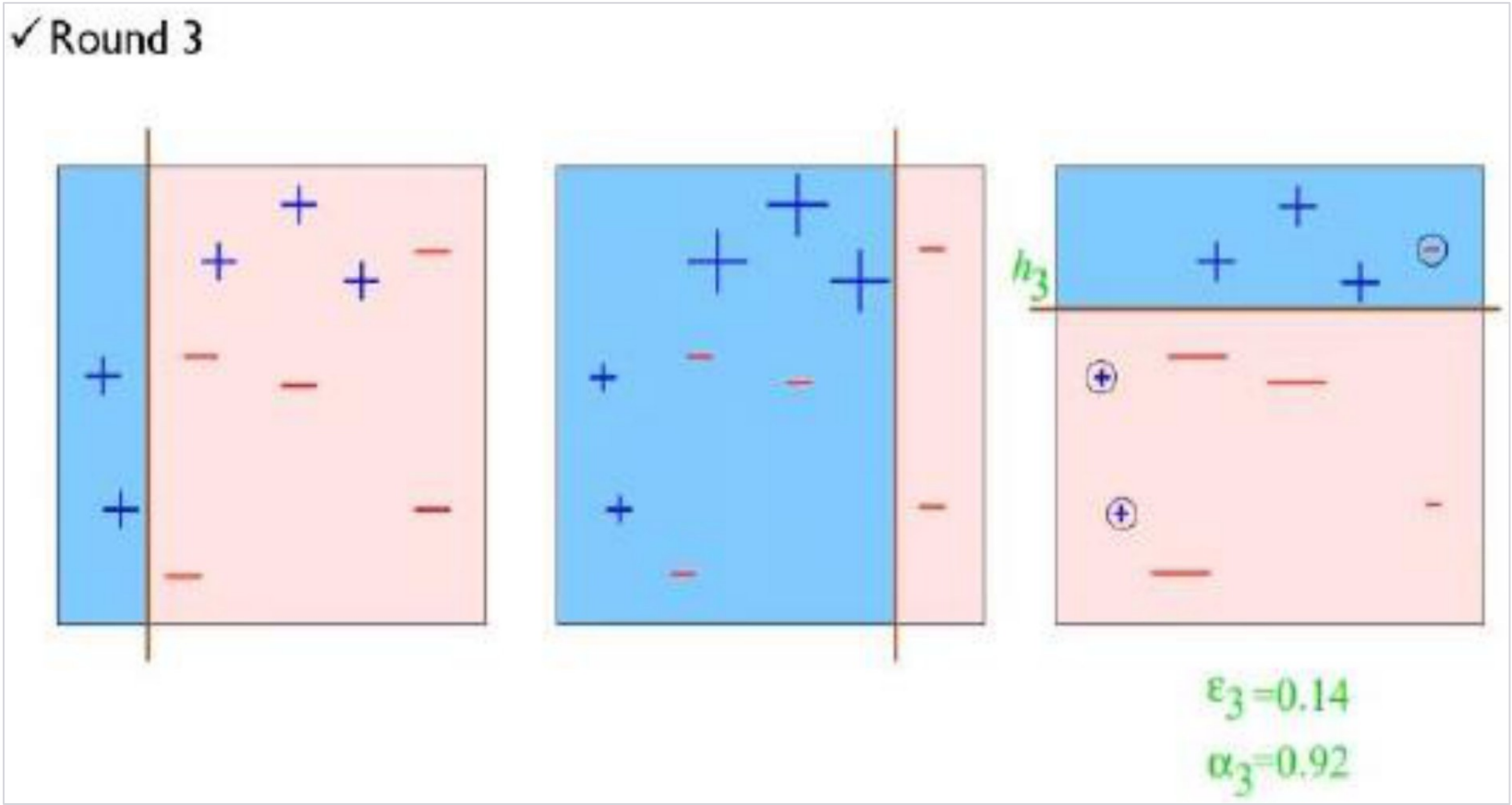


Example 1 — Round 3

세 번째 stump h_3는 가로선 — 앞의 두 stump가 놓친 영역을 겨냥한다

읽는 법

- $\varepsilon_3 = 0.14 \rightarrow \alpha_3 = 0.92$
- 라운드가 갈수록 ε가 작아지고 α는 커졌다
- 각 stump는 혼자서는 약하다

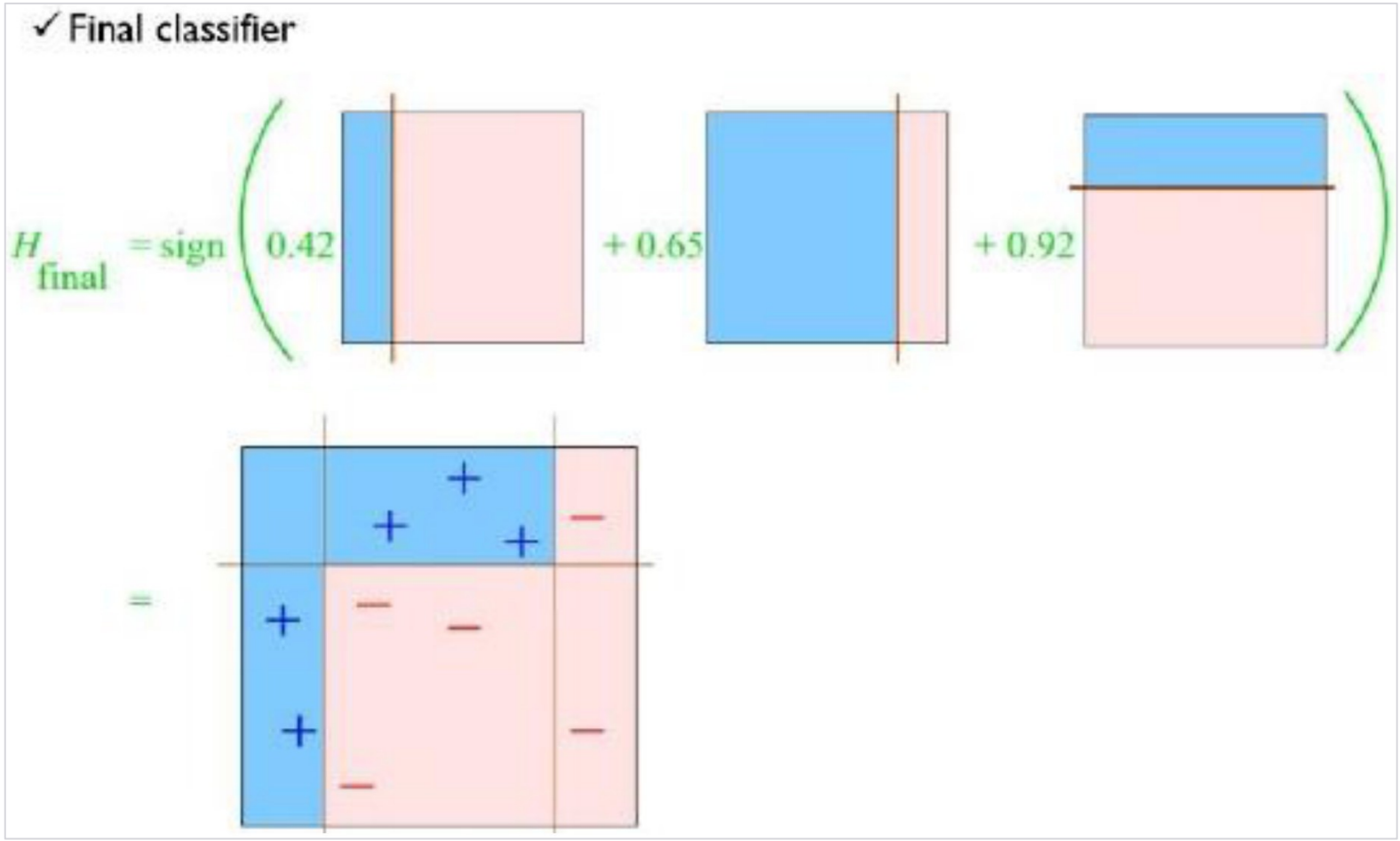


Example 1 — 최종 분류기

$H_final = sign(0.42 \cdot h_1 + 0.65 \cdot h_2 + 0.92 \cdot h_3)$

결과

- 직선 하나로는 못 가르던 +/−를
- 세 stump의 가중 합이 계단형 경계로 모두 맞힌다
- 약학습기의 결합 → 강학습기



보강 — Example 1의 α를 직접 검증

가중치를 정확히 따라가면 $\varepsilon_1 = 3/10$, $\varepsilon_2 = 3/14$, $\varepsilon_3 = 3/22$

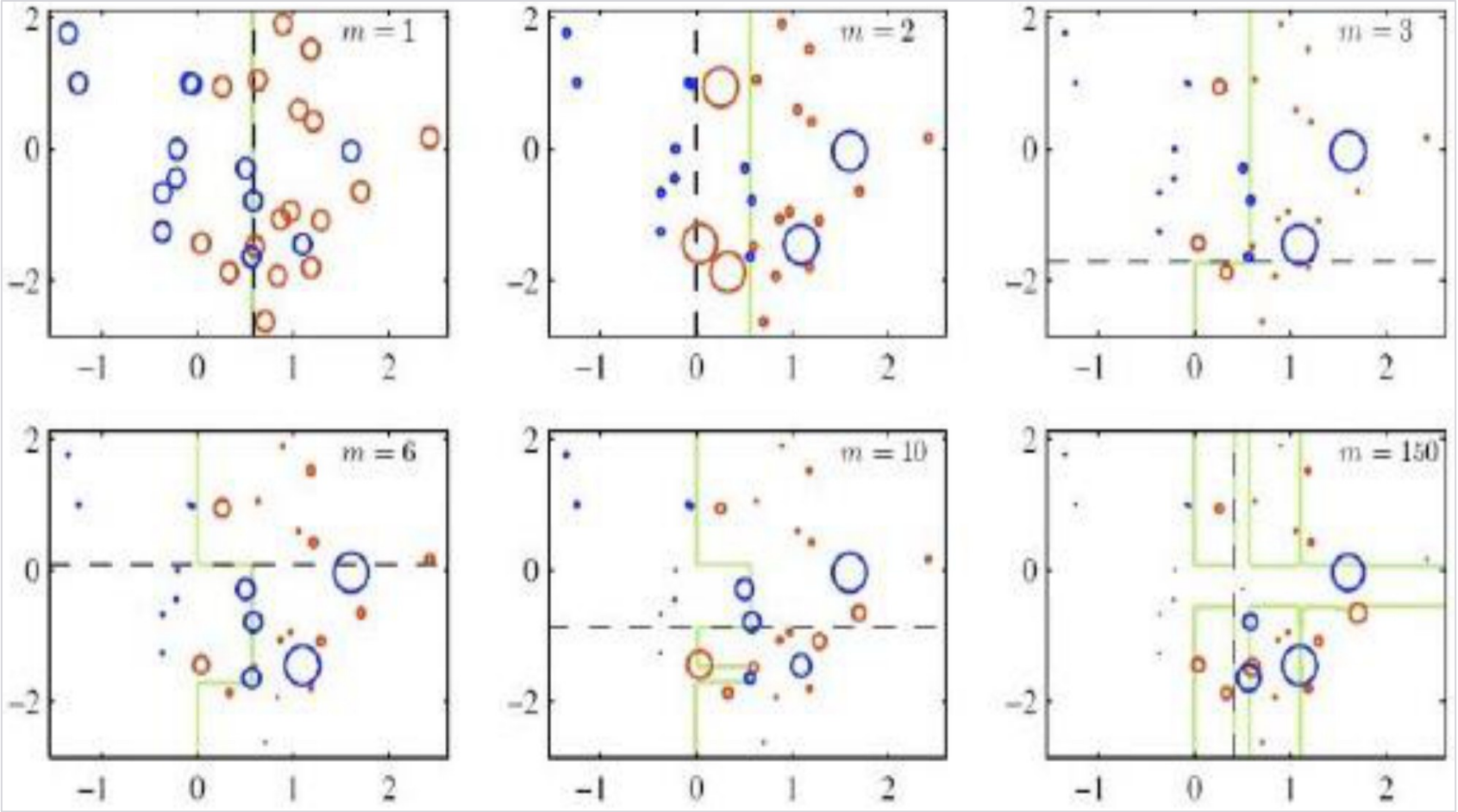
$$\alpha_1 = \tfrac{1}{2}\ln\tfrac{7/10}{3/10} = 0.42 \qquad \alpha_2 = \tfrac{1}{2}\ln\tfrac{11/14}{3/14} = 0.65 \qquad \alpha_3 = \tfrac{1}{2}\ln\tfrac{19/22}{3/22} = 0.92$$

라운드	틀린 샘플	그 샘플들의 가중치 합 ε	α = ½ ln((1−ε)/ε)
1	+ 3개 (각 1/10)	3/10 = 0.300	0.42
2	− 3개 (각 1/14)	3/14 = 0.214	0.65
3	3개 (각 1/22)	3/22 = 0.136	0.92

라운드 1 뒤 정규화: 틀린 3개가 합 1/2(각 1/6), 맞힌 7개가 합 1/2(각 1/14). 원 강의의 0.21 · 0.14는 3/14 · 3/22을 반올림한 값이다.

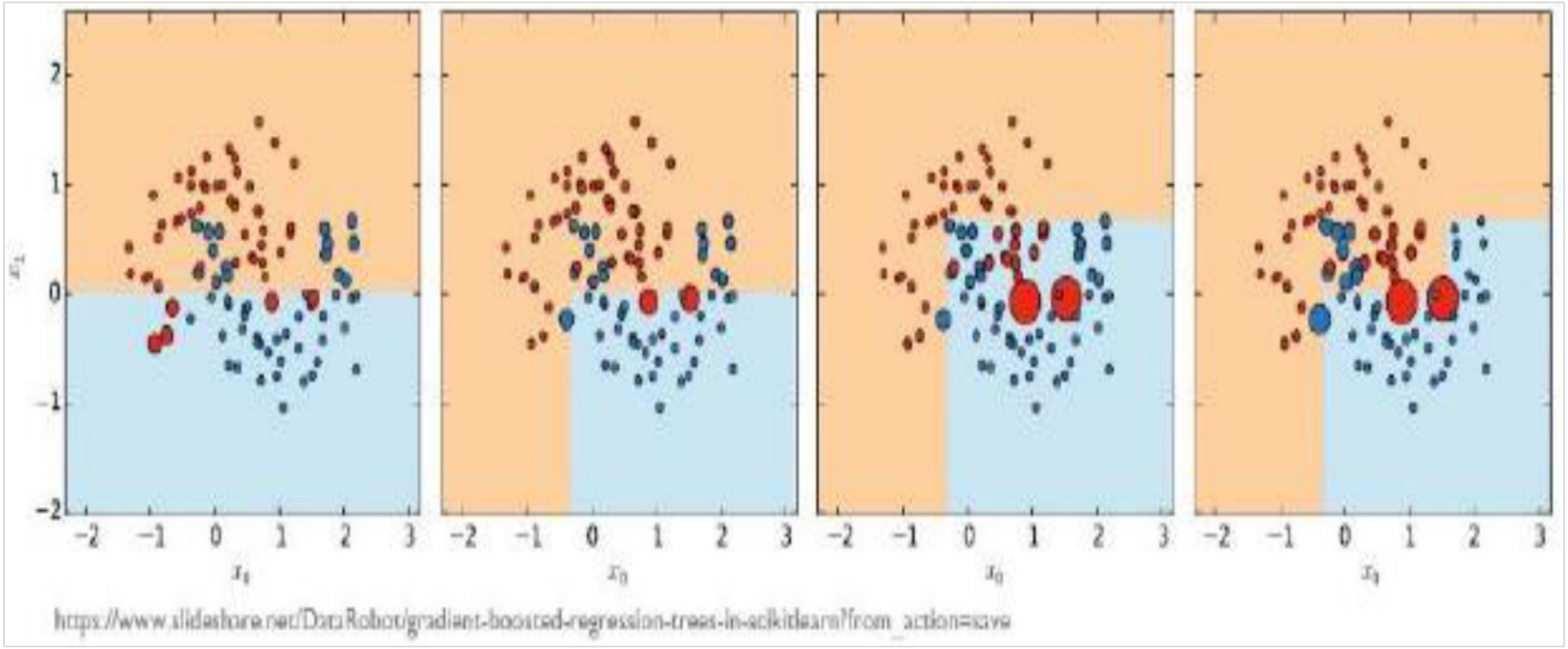
Example 2 — 라운드가 진행되며 경계가 정교해진다

m = 1, 2, 3, 6, 10, 150: 원의 크기 = 샘플 가중치, 점선 = 이번 stump, 초록선 = 누적 경계



Example 3 — 오분류 샘플이 커지는 모습

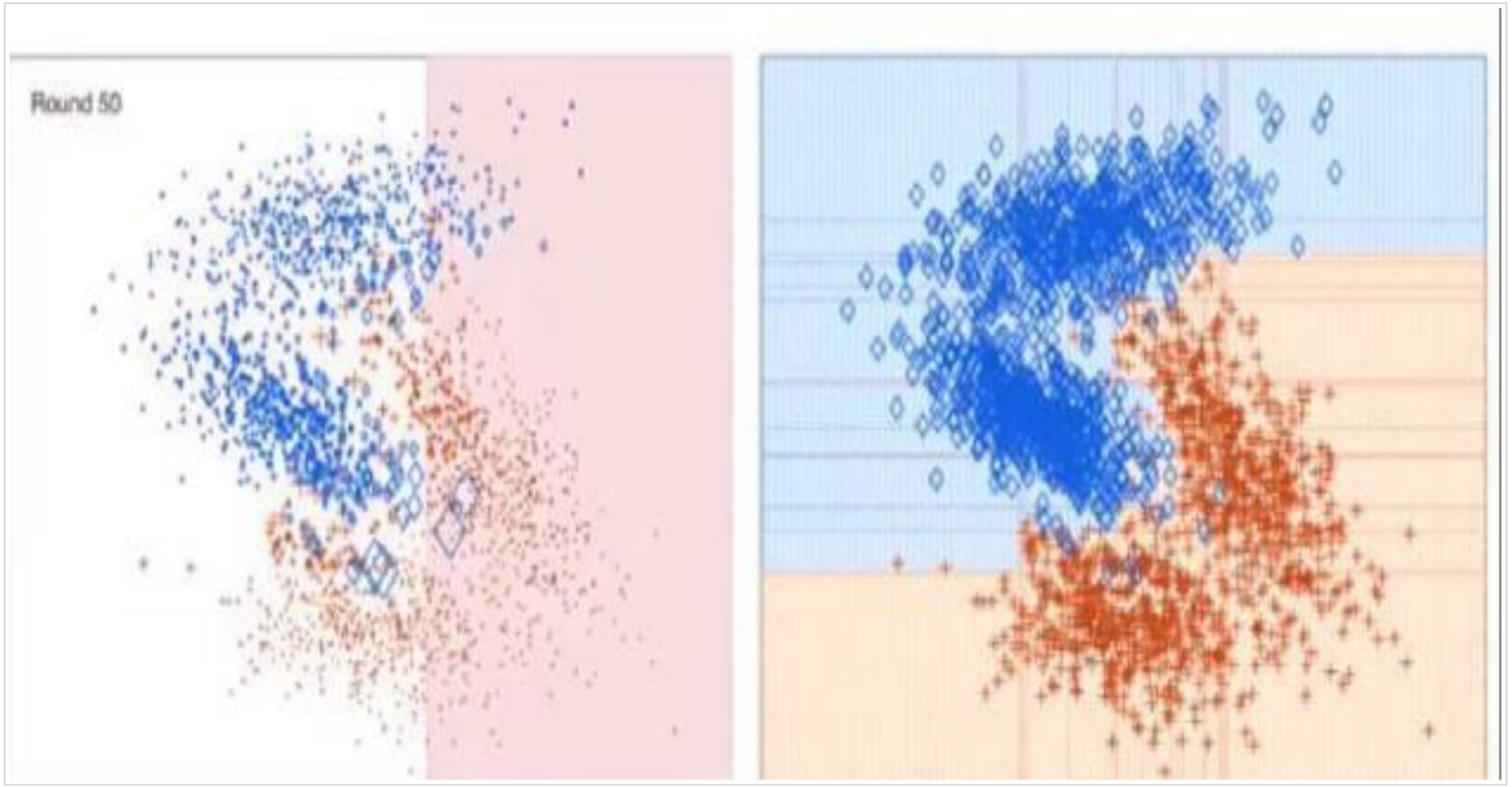
반복할수록 경계 근처의 틀린 샘플(큰 점)에 가중치가 몰린다



부스팅이 이상치(outlier)나 레이블 잡음에 약한 이유이기도 하다 — 잡음 샘플에 가중치가 계속 쌓인다.

Example 4 — 50라운드 후

왼쪽: 50번째 stump 한 개의 경계 / 오른쪽: 누적된 앙상블의 경계

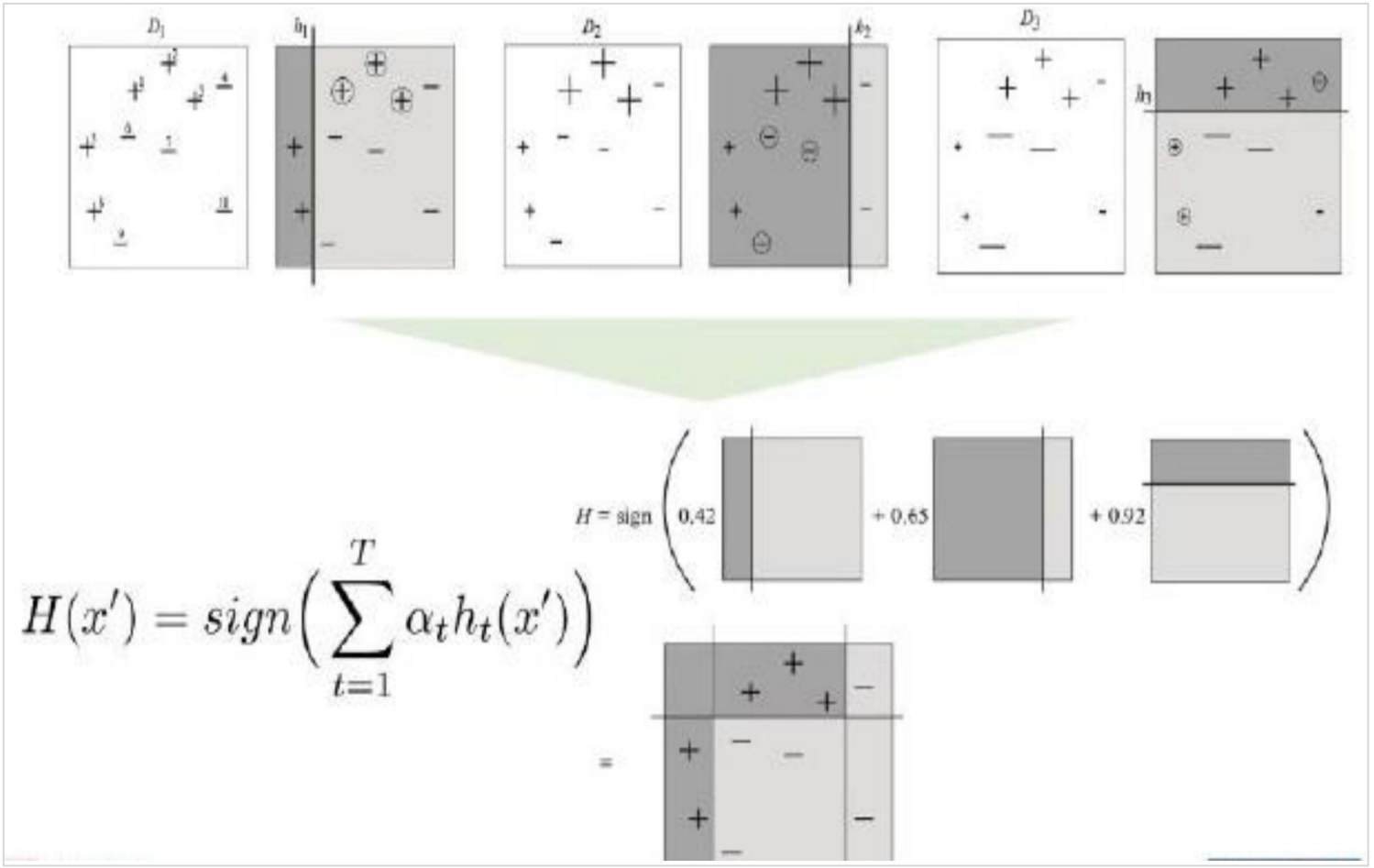


Example 5 — 전체 과정 요약

가중 재표본($D_1 \rightarrow D_2 \rightarrow D_3$)과 가중 투표를 한 장에

한 줄 요약

- 매 라운드 가중치 재조정
- stump마다 발언권 α 부여
- 최종: α로 가중한 투표의 부호



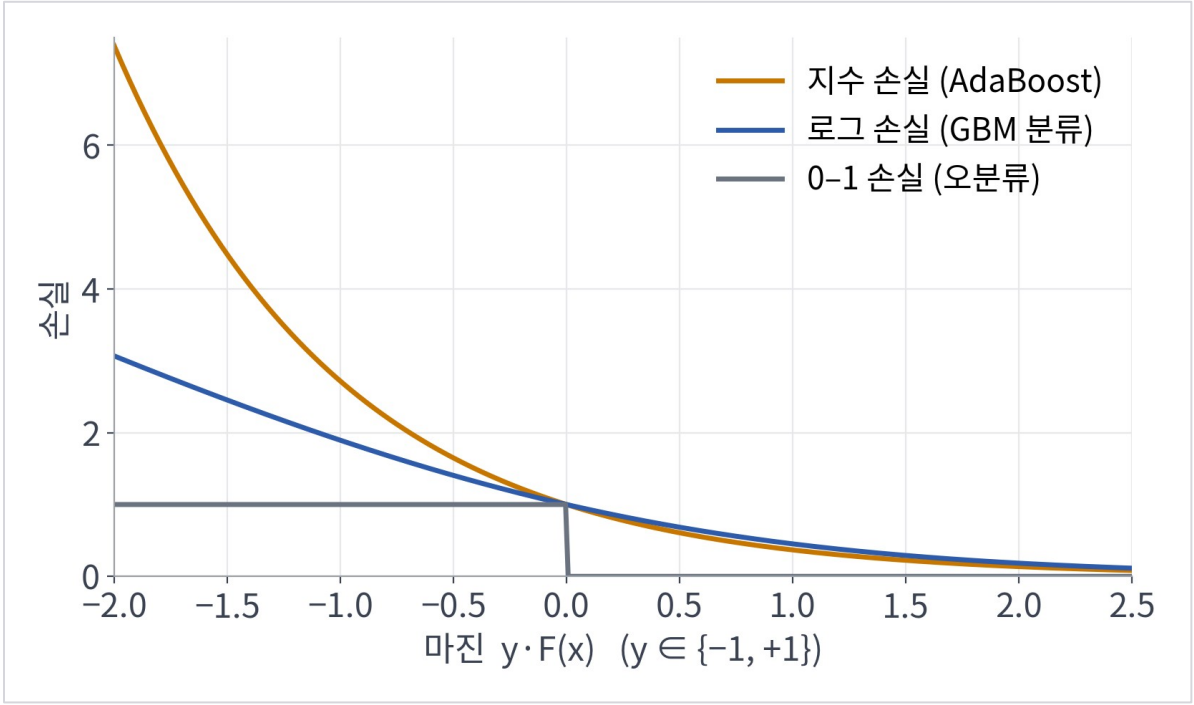
보강 — AdaBoost는 지수 손실을 줄이는 가법 모형이다

Friedman · Hastie · Tibshirani(2000): AdaBoost = 지수 손실의 전진 단계적 가법 모형 → GBM으로 가는 다리

$$L\big(y, F(x)\big) = e^{-yF(x)}, \qquad F_m(x) = F_{m-1}(x) + \alpha_m h_m(x)$$

왜 중요한가

- AdaBoost의 가중치 = 지수 손실의 기울기 크기
- "손실함수를 바꾸면?" → 5부 그래디언트 부스팅
- 지수 손실은 크게 틀린 샘플에 벌점이 폭증 → 잡음에 민감



보강 — scikit-learn AdaBoostClassifier

유방암 데이터, 같은 분할 (scikit-learn 1.8 실행 결과)

Python · scikit-learn

```
from sklearn.ensemble import AdaBoostClassifier

ada = AdaBoostClassifier(n_estimators=100,
                         random_state=0)
ada.fit(X_train, y_train)
ada.score(X_test, y_test)    # 0.965
```

기본값

- estimator: 깊이 1 트리 (stump)
- n_estimators = 50
- learning_rate = 1.0 (α에 곱하는 축소 계수)

버전 메모

- 예전의 algorithm="SAMME.R" 옵션은 폐지되어 현재는 SAMME 방식만 남았다
- 옛 코드를 옮길 때는 algorithm 인자를 지운다

에피소드 — AdaBoost, 카메라 속 얼굴을 찾다

Viola · Jones(2001): AdaBoost로 고른 단순한 특징을 계단식(cascade)으로 연결한 실시간 얼굴 검출기

무엇을 했나

- 수십만 개의 단순 사각 특징(Haar-like) 중 AdaBoost가 쓸 만한 것을 골라냄
- 쉬운 배경은 앞 단계에서 빨리 버리고 어려운 후보만 뒤로 넘김
- 당시 PC에서 실시간 검출 → 디지털카메라 · 휴대폰 얼굴 인식의 기반

강의와의 연결

- "약한 규칙 여러 개 + 가중 투표"라는 오늘의 구조 그대로
- 약학습기 = 사각 특징 하나로 만든 stump
- 이론(1995)이 산업 표준(2000년대)이 된 대표 사례

05

5부 · 그래디언트 부스팅

그래디언트 부스팅 — 잔차에 다음 트리를 적합한다

잔차 학습 · scikit-learn 구현 · 몸무게 예제 단계별 · 학습률 · Friedman 알고리즘

AdaBoost와 무엇이 다른가

샘플 가중치를 고치는 대신, 이전 예측기가 남긴 잔여 오차(residual)에 새 예측기를 학습시킨다

	AdaBoost	Gradient Boosting
다음 모델이 보는 것	가중치가 바뀐 같은 데이터	이전 모델의 잔차 (음의 기울기)
기본 학습기	stump (깊이 1)	얕은 회귀 트리 (분류에서도)
결합	α로 가중 투표	학습률 ν를 곱해 더함
손실함수	지수 손실 고정	미분 가능한 손실 (제곱 · 로그 · Huber)

Friedman(2001): 경사하강을 함수 공간에서 — "A Gradient Boosting Machine"

잔차에 순서대로 트리를 쌓는다

DecisionTreeRegressor(max_depth=2) 세 개를 손으로 이어 붙이기

코드의 흐름

- tree_reg1: 원래 y에 적합
- y2 = y − tree_reg1 예측 → tree_reg2 적합
- y3 = y2 − tree_reg2 예측 → tree_reg3 적합
- 예측 = 세 트리 예측의 합

```
from sklearn.tree import DecisionTreeRegressor

tree_reg1 = DecisionTreeRegressor(max_depth=2)
tree_reg1.fit(X, y)
```

- 첫번째 예측기에서 생긴 잔여 오차에 두번째 결정트리를 훈련시킨다.

```
y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2)
tree_reg2.fit(X, y2)
```

- 두번째 예측기에 생긴 잔여 오차에 세번째 회귀모델을 훈련시킨다.

```
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2)
tree_reg3.fit(X, y3)
```

개별 예측기 vs 앙상블 예측기

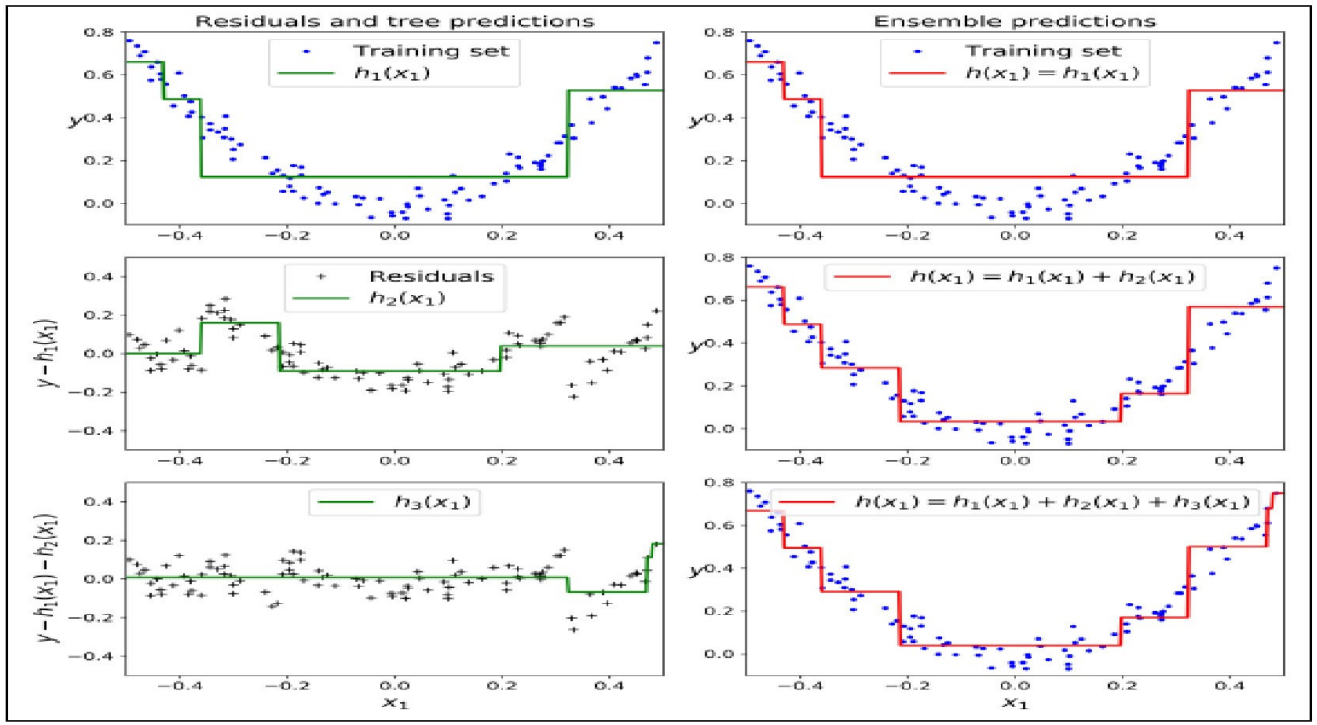
왼쪽: 각 트리가 적합한 잔차 / 오른쪽: 누적 앙상블 h_1 → h_1+h_2 → $h_1+h_2+h_3$

읽는 법

- y_pred = sum(tree.predict(X_new) for tree in trees)
- 트리를 더할수록 앙상블이 데이터에 가까워진다
- 남은 잔차(왼쪽 아래)는 점점 잡음처럼 평평해진다

```
y_pred = sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
```

- 개별 예측기와 앙상블 예측기의 비교



scikit-learn GradientBoostingRegressor

앞의 수동 과정 = max_depth=2, n_estimators=3, learning_rate=1.0

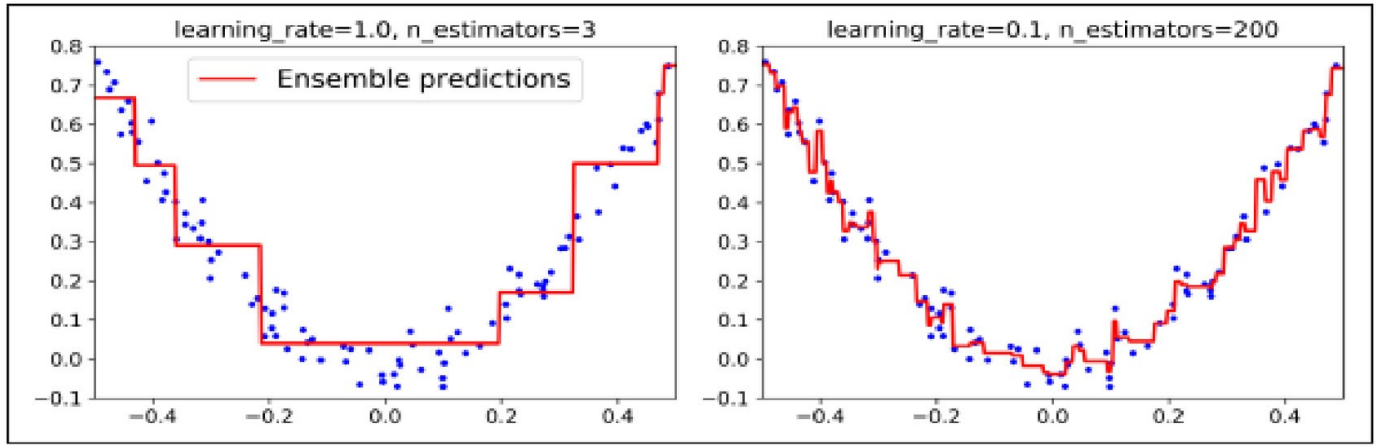
학습률과 트리 수

- learning_rate를 0.1처럼 낮추면 트리가 많이 필요하다
- 대신 예측 성능(일반화)은 좋아진다 — 축소(shrinkage)
- 왼쪽: 트리 부족 / 오른쪽: 트리 200개로 부드러운 적합

```
from sklearn.ensemble import GradientBoostingRegressor

gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3, learning_rate=1.0)
gbrt.fit(X, y)
```

- Learning rate 매개변수를 0.1처럼 낮게 설정하면 훈련을 위해 많은 트리가 필요하지만 예측의 성능은 좋아진다.

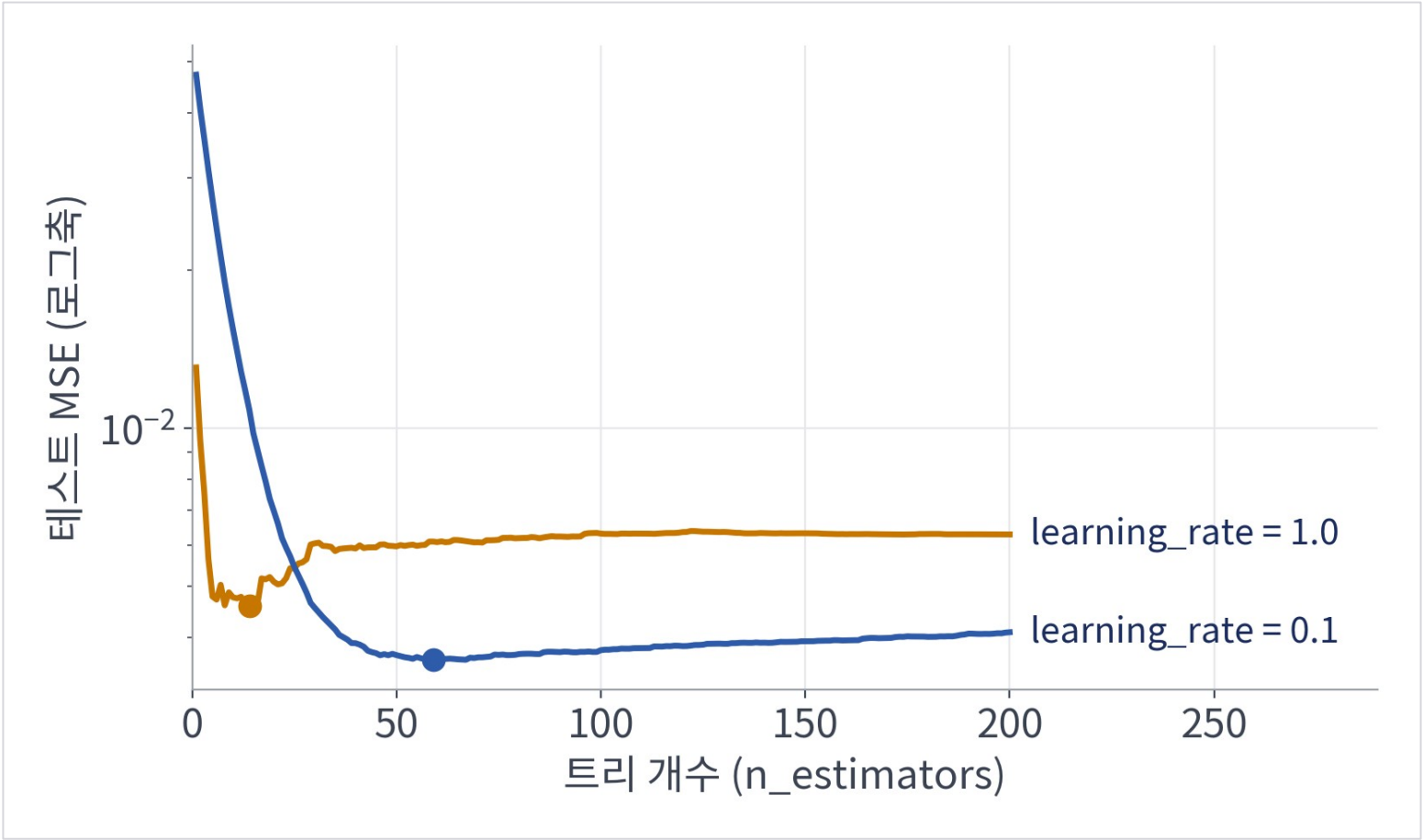


보강 — 학습률 1.0 vs 0.1, 테스트 오차로 확인

같은 2차 곡선 데이터(y = 3x² + 잡음), max_depth = 2 (scikit-learn 1.8 재현)

읽는 법

- 1.0: 14개에서 최저 후 오히려 나빠진다
- 0.1: 59개에서 최저, 최저 오차도 더 낮다
- 점 = 최저 지점 → early stopping으로 찾는다



GradientBoostingClassifier — 과대적합 다루기

유방암 데이터: 기본값 → 깊이 제한 → 학습률 감소

결과

- 기본(n_estimators=100, max_depth=3): 훈련 1.000 · 테스트 0.958 → 과대적합
- max_depth=1: 0.991 · 0.972
- learning_rate=0.01: 0.988 · 0.965

```
In [73]: from sklearn.ensemble import GradientBoostingClassifier

         X_train, X_test, y_train, y_test = train_test_split(
             cancer.data, cancer.target, random_state=0)

         gbrt = GradientBoostingClassifier(random_state=0)
         gbrt.fit(X_train, y_train)

         print("훈련 세트 정확도: {:.3f}".format(gbrt.score(X_train, y_train)))
         print("테스트 세트 정확도: {:.3f}".format(gbrt.score(X_test, y_test)))

         훈련 세트 정확도: 1.000
         테스트 세트 정확도: 0.958
```

n_estimators=100, max_depth=3

과대적합

```
In [74]: gbrt = GradientBoostingClassifier(random_state=0, max_depth=1)
         gbrt.fit(X_train, y_train)

         print("훈련 세트 정확도: {:.3f}".format(gbrt.score(X_train, y_train)))
         print("테스트 세트 정확도: {:.3f}".format(gbrt.score(X_test, y_test)))

         훈련 세트 정확도: 0.991
         테스트 세트 정확도: 0.972
```

트리 깊이 제한

```
In [75]: gbrt = GradientBoostingClassifier(random_state=0, learning_rate=0.01)
         gbrt.fit(X_train, y_train)

         print("훈련 세트 정확도: {:.3f}".format(gbrt.score(X_train, y_train)))
         print("테스트 세트 정확도: {:.3f}".format(gbrt.score(X_test, y_test)))

         훈련 세트 정확도: 0.988
         테스트 세트 정확도: 0.965
```

학습률 감소

82

몸무게 예제 — 키 · 좋아하는 색 · 성별로 몸무게 예측

6명의 데이터 (그림: StatQuest 강의 슬라이드 인용)

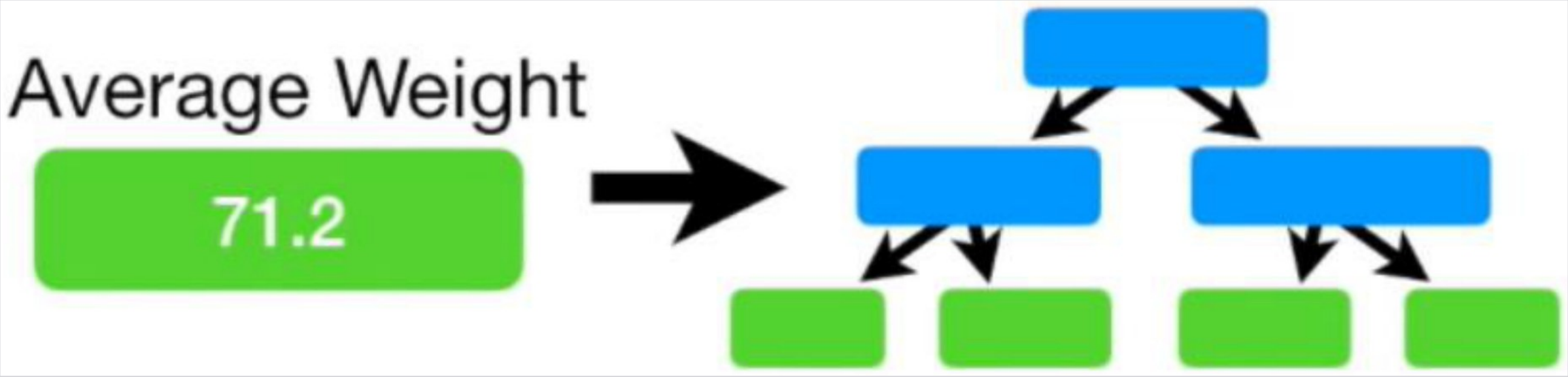
과제

- 타깃: 몸무게(kg) — 회귀 문제
- 특성: 키(m), 좋아하는 색, 성별
- GBM이 한 단계씩 어떻게 예측을 고치는지 따라간다

Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

1단계 — 싱글 리프(평균)로 출발

첫 예측은 모든 사람에게 평균 몸무게 71.2kg



$$F_0 = \bar{y} = \frac{88 + 76 + 56 + 73 + 77 + 57}{6} = 71.2$$

2단계 — 잔여 오차(Residual) 계산

잔차 = 실제값 − 예측값(71.2)

잔차

- 88 − 71.2 = 16.8
- 56 − 71.2 = −15.2
- 잔차가 곧 "아직 설명 못 한 부분"

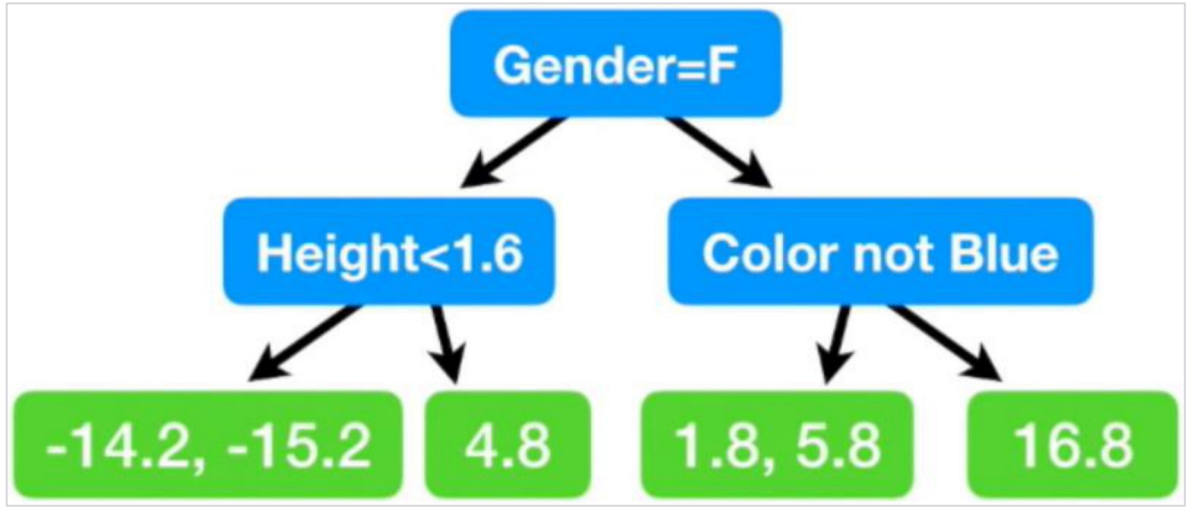
Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	16.8
1.6	Green	Female	76	4.8
1.5	Blue	Female	56	-15.2
1.8	Red	Male	73	1.8
1.5	Green	Male	77	5.8
1.4	Blue	Female	57	-14.2

$$r_i = y_i - F_0(x_i)$$

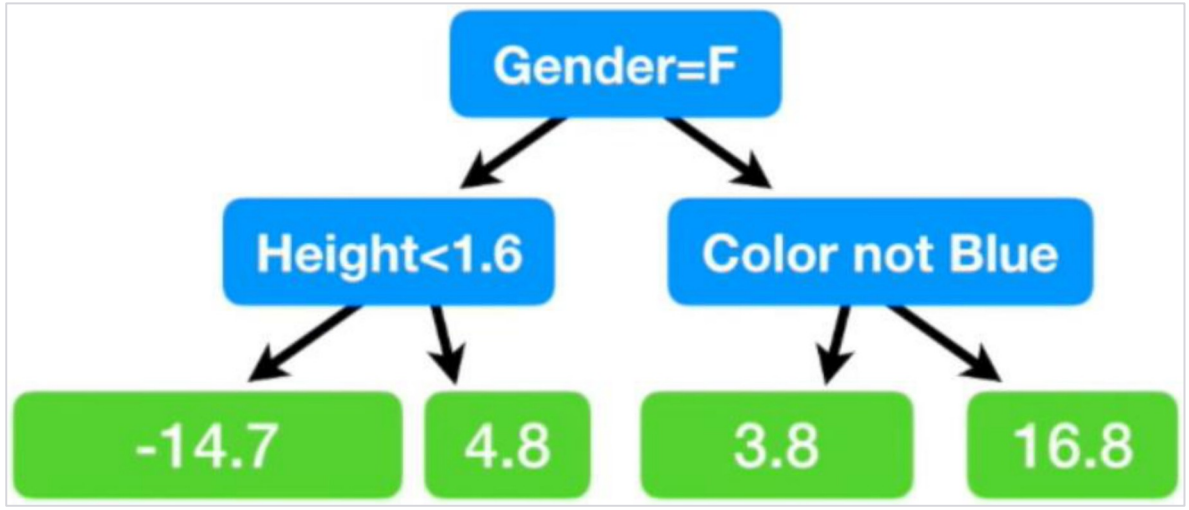
3단계 — 잔차를 예측하는 트리

키 · 색 · 성별로 몸무게가 아니라 잔차를 예측한다. 리프에 값이 둘이면 평균을 넣는다

리프에 잔차 배정

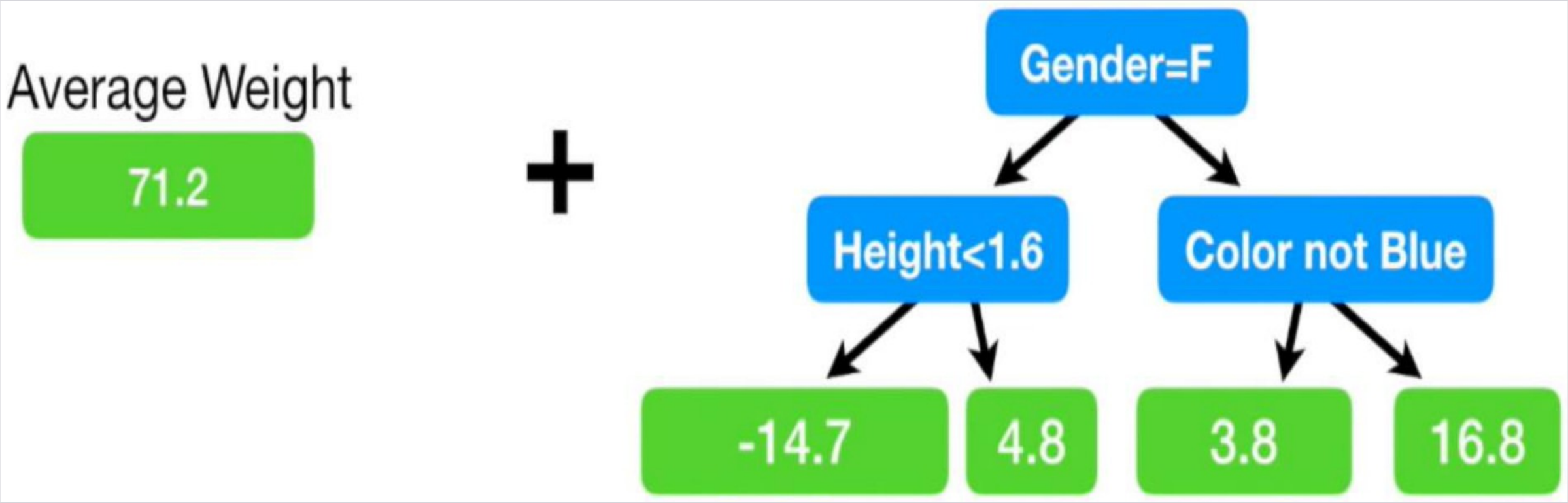


두 값의 리프 → 평균: (−14.2 − 15.2)/2 = −14.7



4단계 — 초기 리프와 잔차 트리를 조합

예측 = 평균 몸무게(71.2) + 잔차 트리의 출력



초기 트리(싱글 리프)와 두 번째로 구한 트리를 더해 몸무게를 예측한다.

4단계 — 남성 · 파란색이면?

두 번째 트리에서 성별이 남자이고 좋아하는 색이 파랑이면 잔차를 16.8로 예측한다



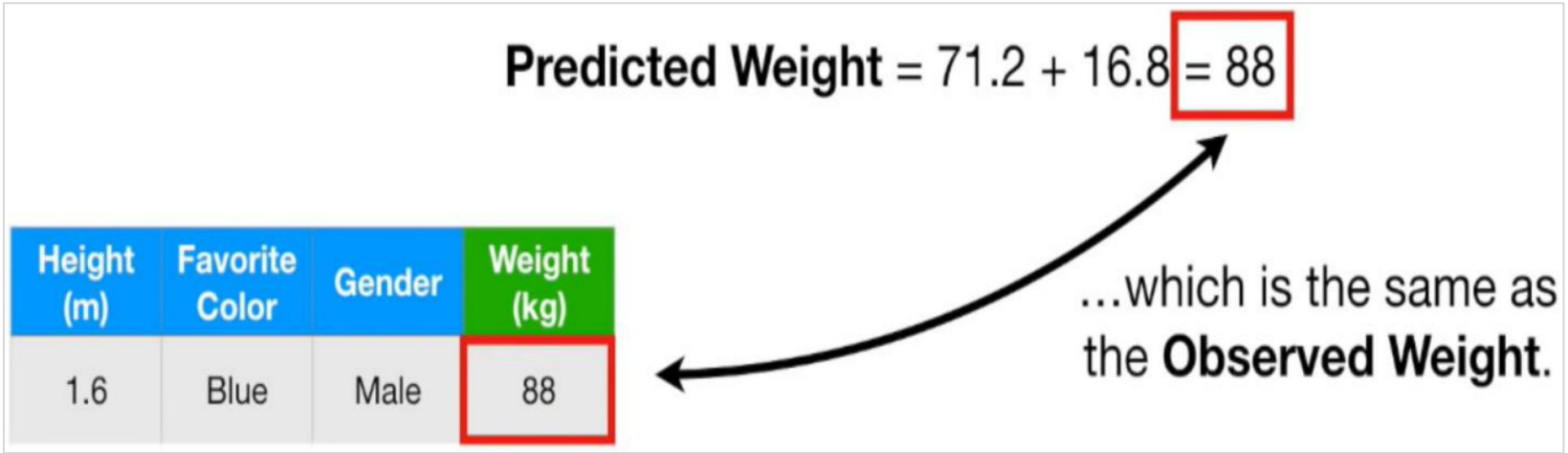
예측 몸무게 = 71.2 + 16.8 = 88kg

관측값과 정확히 일치 — 과적합

예측 88kg = 실제 88kg. 놀라운 일이 아니다

왜 문제인가

- 훈련 데이터의 잔차를 그대로 더했으니 맞는 게 당연
- 훈련 데이터에 너무 과적합된 모델
- 새 데이터에는 잘 안 맞는다 (편향 작고 분산 큼)



해결책 — 학습률(Learning Rate)을 곱한다

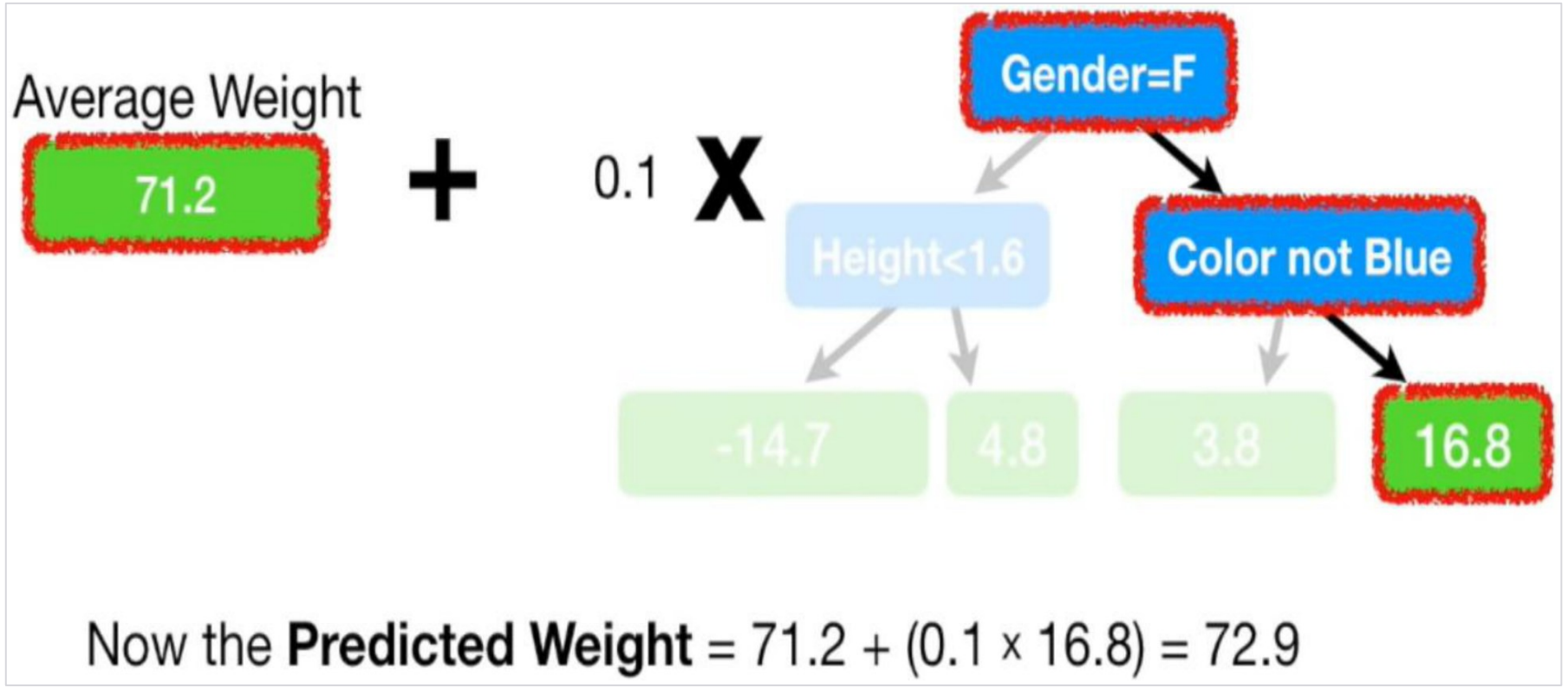
학습률은 0~1 사이 값. 잔차를 예측하는 트리의 출력에 곱해 한 번에 조금씩만 반영한다



$$F_m(x) = F_{m-1}(x) + \nu\, h_m(x), \qquad 0 < \nu \leq 1$$

학습률 0.1로 다시 예측하면

예측 몸무게 = 71.2 + (0.1 × 16.8) = 72.9kg



첫 싱글 리프의 71.2kg보다 실제값 88kg에 조금 더 가까워졌다.

조금씩, 올바른 방향으로

72.9kg는 첫 예측 71.2kg보다 실제값 88kg에 가까워졌다

원리

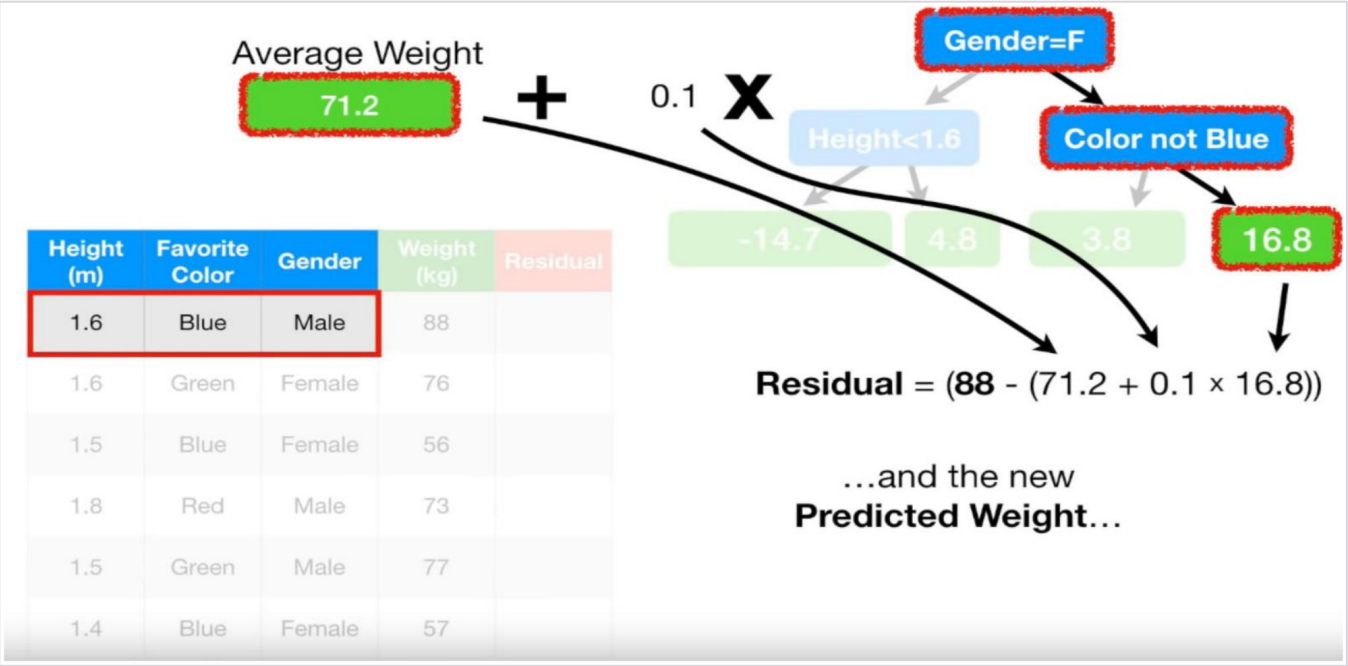
- 실제값에 가까워지는 방향으로 작은 걸음을 반복
- 이렇게 학습한 모델이 테스트 데이터에서 성능이 좋다
- Friedman: 작은 학습률(≤ 0.1)이 일반화에 유리



5단계 — 새 모델로 잔차를 다시 구한다

잔차 = 실제값 − (71.2 + 0.1 × 트리 출력)

첫 번째 사람: 88 − (71.2 + 0.1 × 16.8)



새 잔차 열

Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	15.1
1.6	Green	Female	76	4.3
1.5	Blue	Female	56	-13.7
1.8	Red	Male	73	1.4
1.5	Green	Male	77	5.4
1.4	Blue	Female	57	-12.7

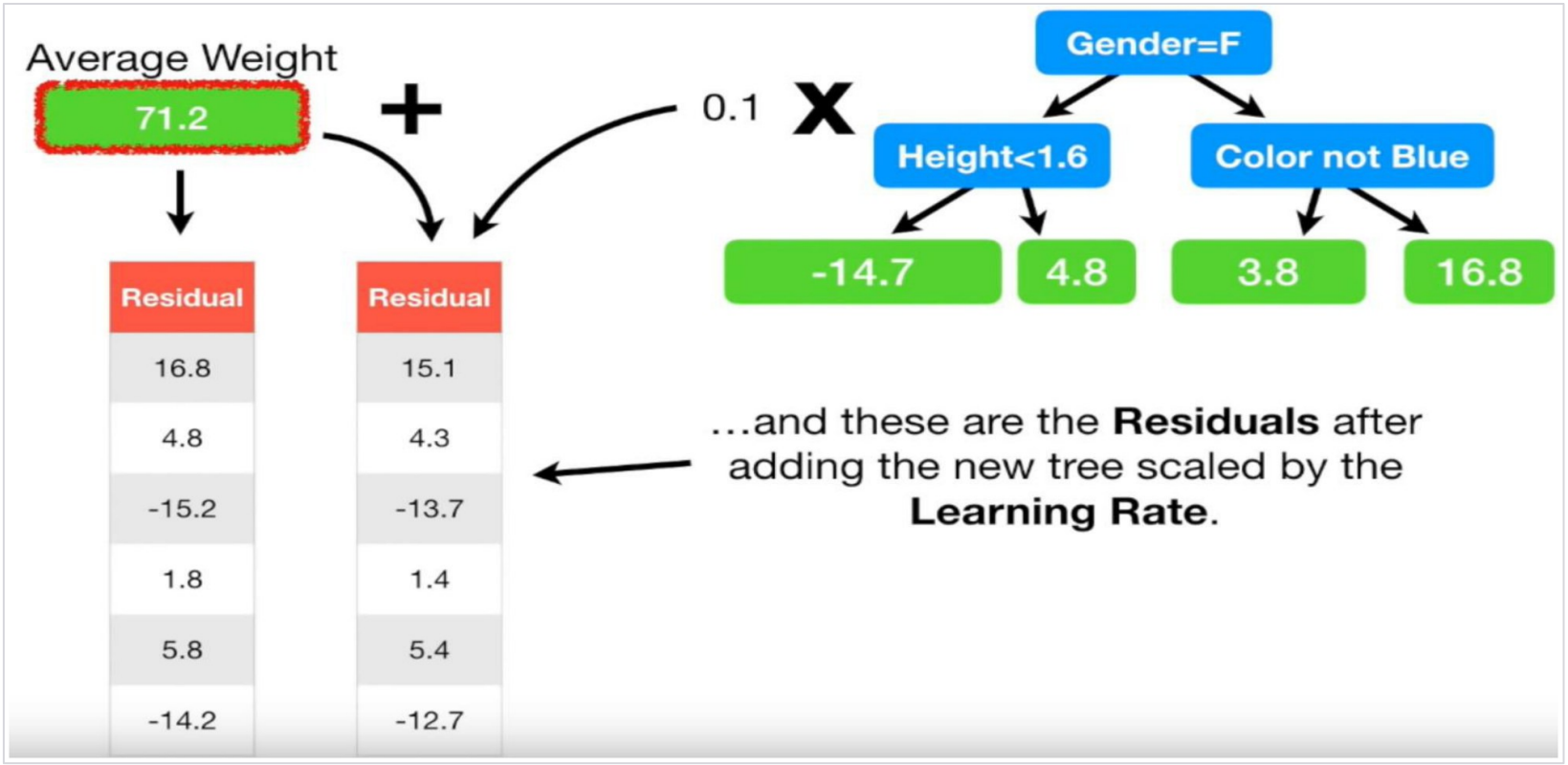
$$r = 88 - (71.2 + 0.1 \times 16.8) = 15.1$$

잔차가 모두 줄었다

왼쪽: 평균만 쓴 잔차 / 오른쪽: 트리 하나를 더한 뒤의 잔차

관찰

- 16.8 → 15.1, −15.2 → −13.7 …
- 모든 잔차의 크기가 작아졌다
- = 예측이 실제값에 조금씩 다가가고 있다



6단계 — 새 잔차로 두 번째 트리

두 값이 든 리프는 평균: (−12.7, −13.7) → −13.2, (1.4, 5.4) → 3.4

트리 모양

- 예제에서는 리프 4개 트리를 계속 쓴다
- 실제로는 반복마다 트리 모양이 달라진다
- 보통 리프 8~32개 트리 사용 (StatQuest 설명)

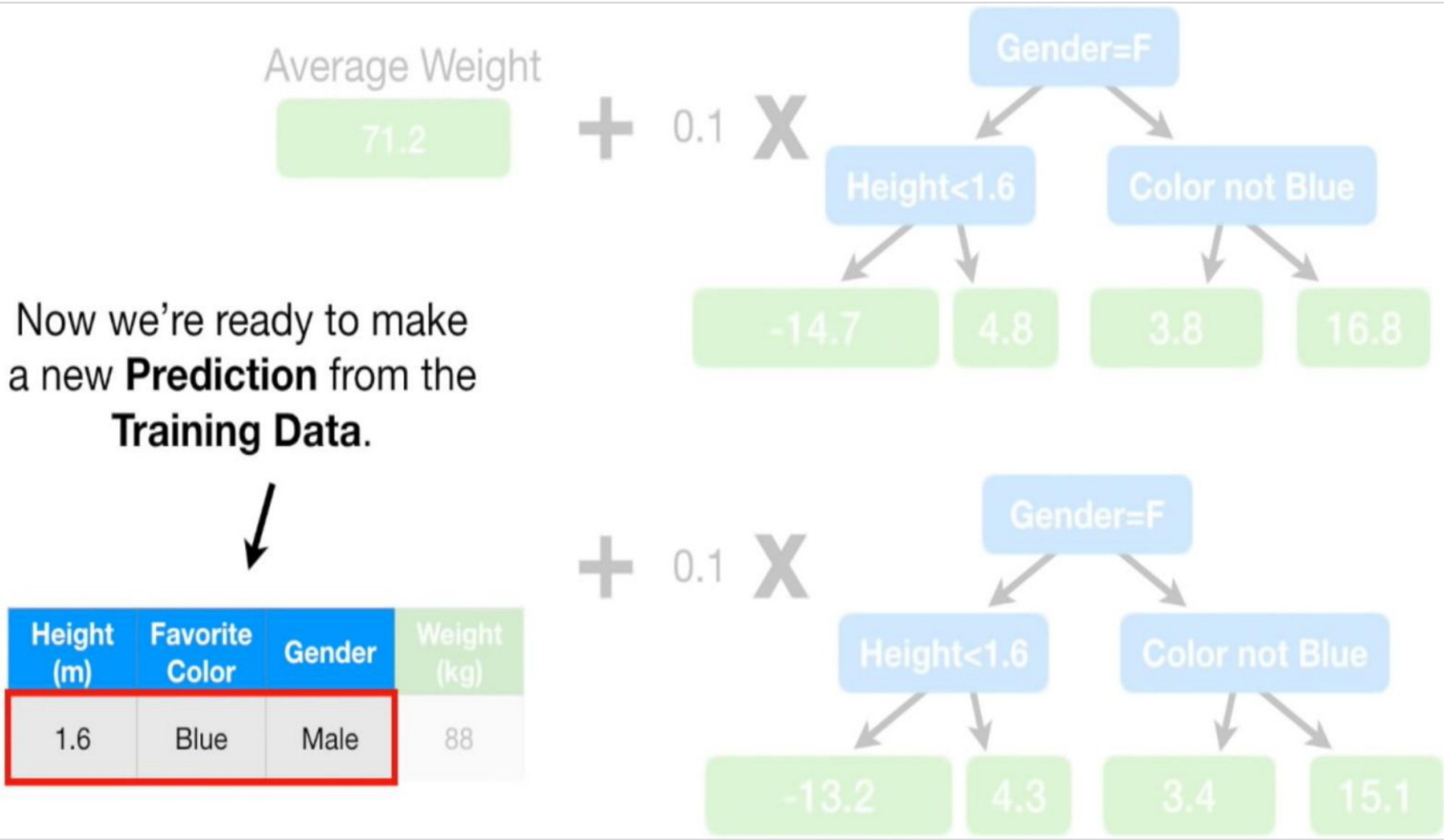


7단계 — 지금까지의 트리를 모두 더한다

초기 리프 + 0.1 × 첫 번째 잔차 트리 + 0.1 × 두 번째 잔차 트리

지금까지의 모델

- F_0 = 71.2 (싱글 리프)
- + 0.1 × 첫 번째 잔차 트리
- + 0.1 × 두 번째 잔차 트리
- 이 모델로 훈련 데이터를 다시 예측한다



7단계 — 키 1.6m · 남성 · 파랑의 예측

71.2 → 72.9 → 74.4kg: 실제값 88kg에 조금씩 다가간다



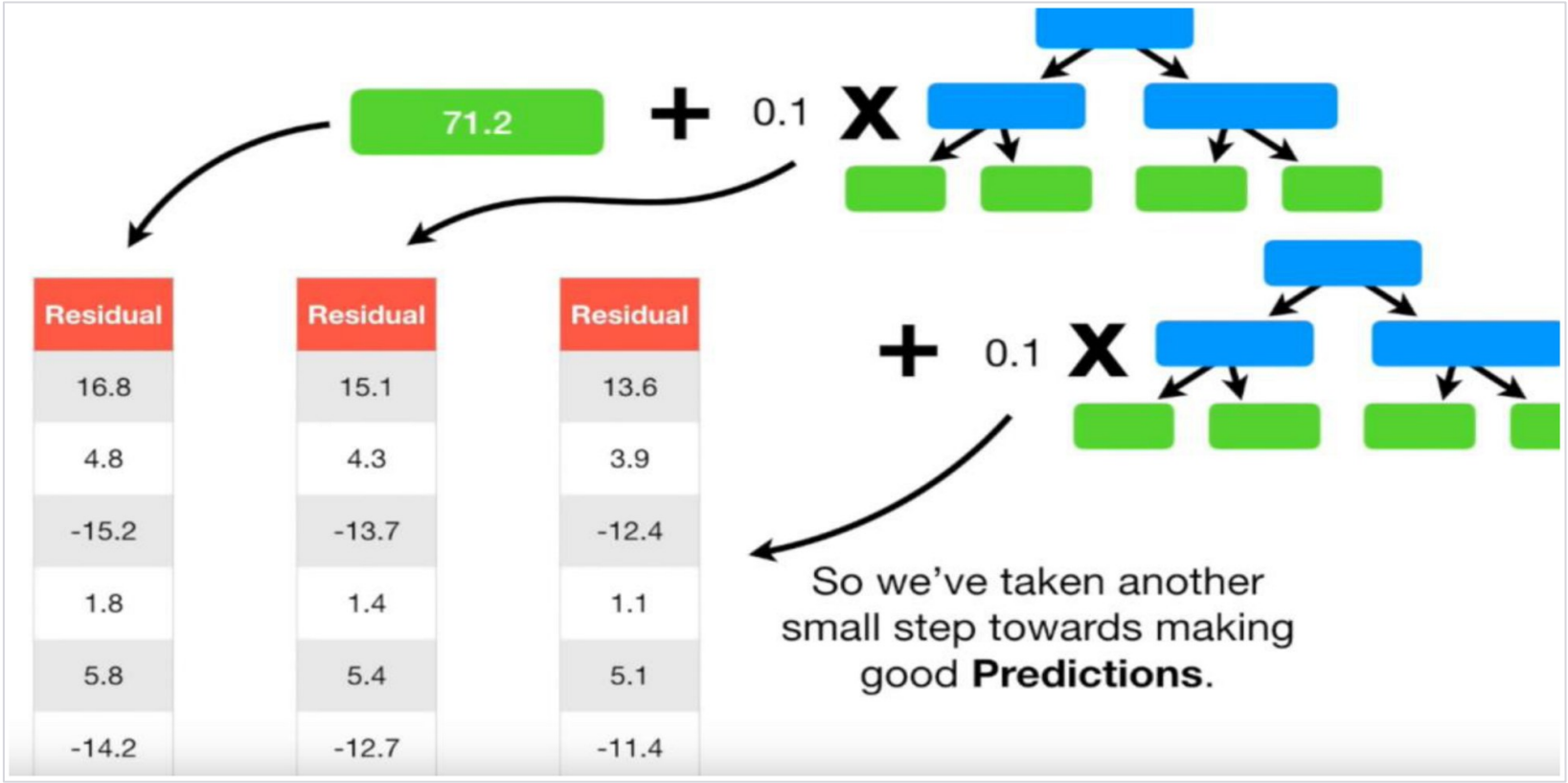
$$\hat{y} = 71.2 + 0.1 \times 16.8 + 0.1 \times 15.1 = 74.4$$

잔차가 한 걸음 더 줄었다

16.8 → 15.1 → 13.6: 반복할 때마다 작아진다

관찰

- 71.2 → 72.9 → 74.4 → … → 88을 향해
- 잔차 크기 감소 = 훈련 정확도 상승
- 다음 장: 잔차 제곱합으로 보면

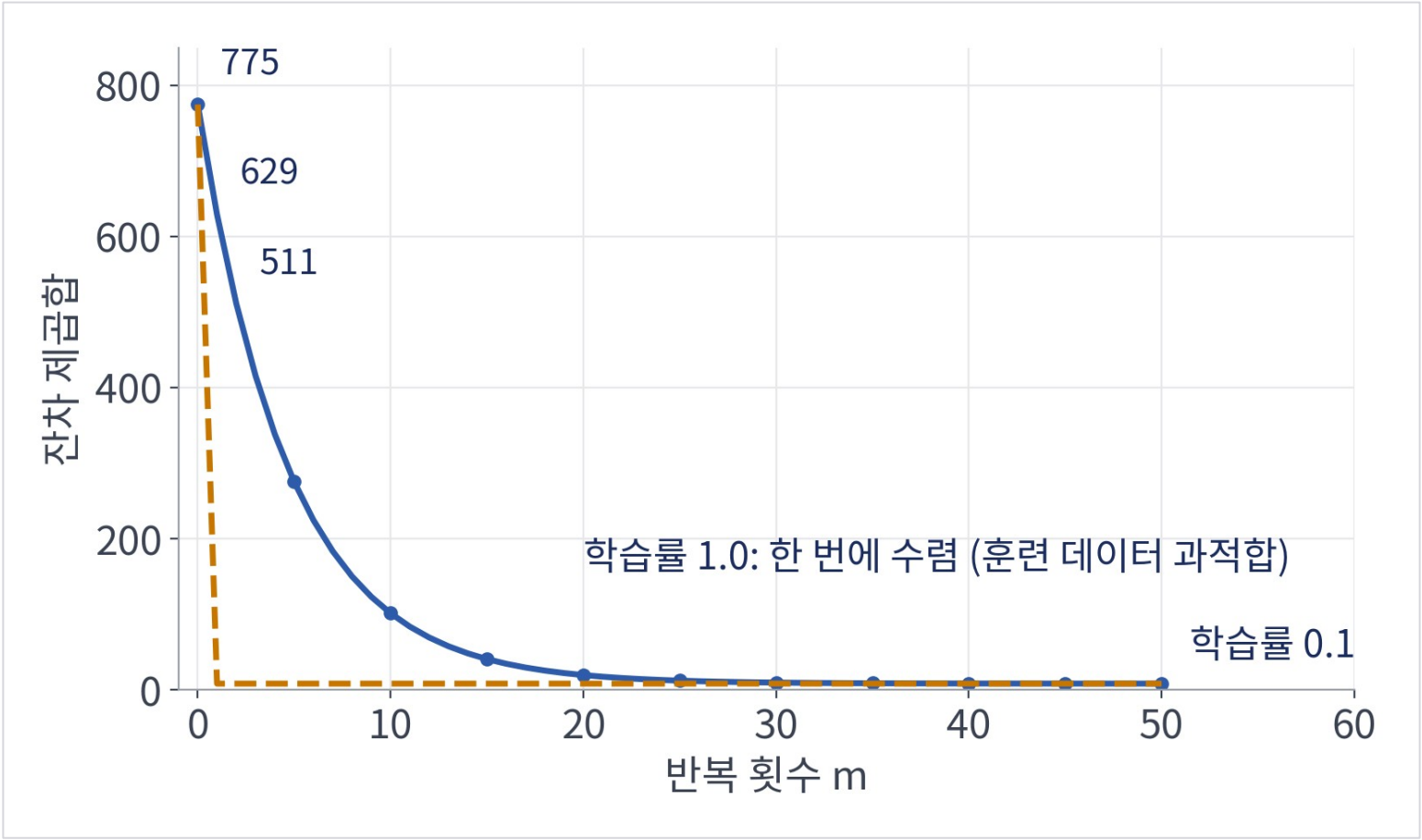


보강 — 잔차 제곱합으로 본 수렴 속도

원 예제의 트리 구조(리프 4개)를 고정하고 반복을 이어 가면

읽는 법

- 학습률 0.1: 775 → 629 → 511 … 서서히 감소
- 학습률 1.0: 한 번에 바닥 = 훈련 데이터를 외움
- 바닥(8.5)은 같은 리프 안의 차이 — 트리 구조의 한계

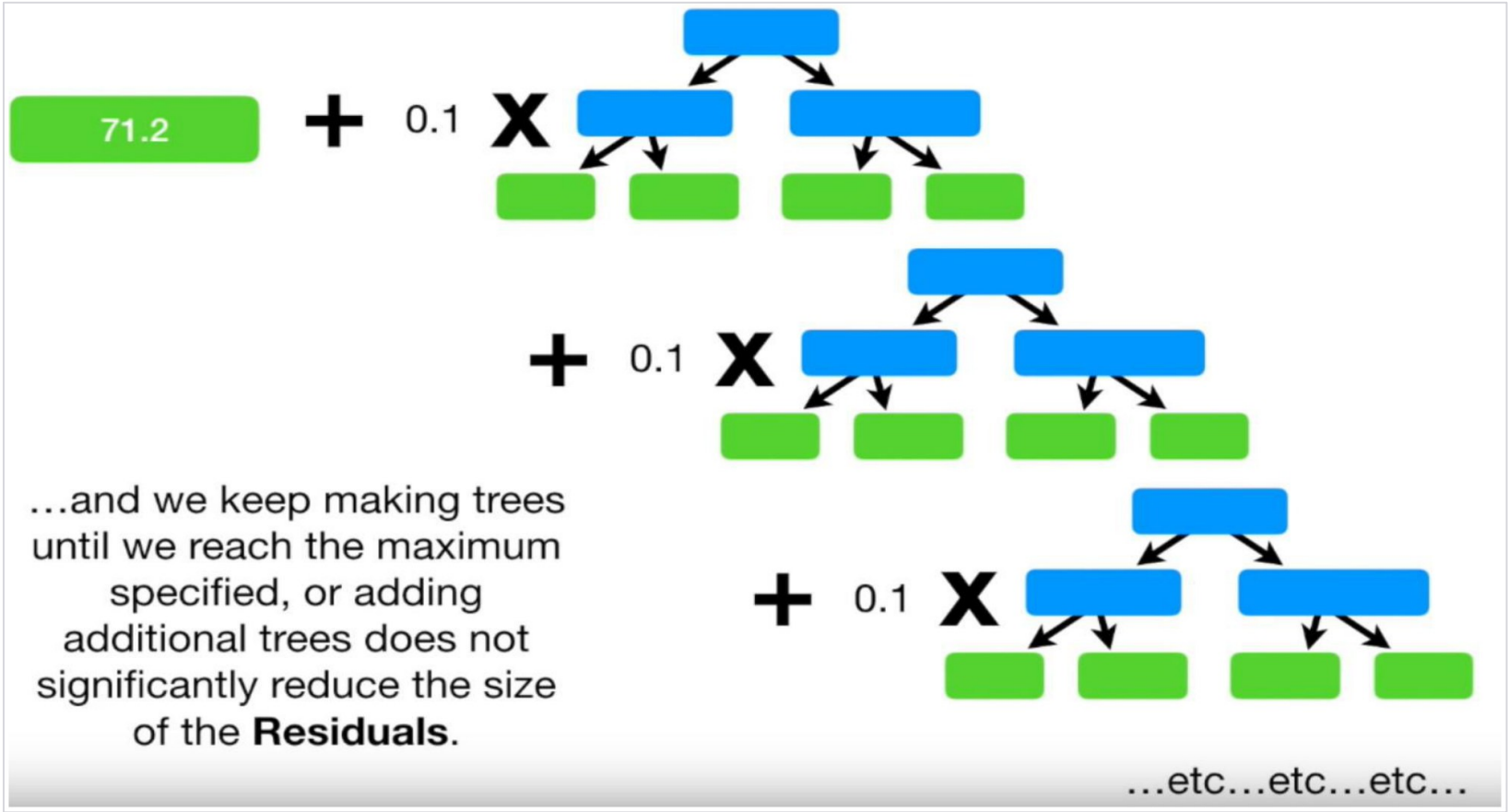


언제까지 반복하나

정해 둔 반복 횟수에 도달하거나, 트리를 더해도 잔차가 크게 줄지 않을 때까지

멈춤 조건

- n_estimators (최대 트리 수)
- 검증 성능이 개선되지 않으면 조기 종료 (n_iter_no_change)
- 학습률과 트리 수는 함께 조정한다

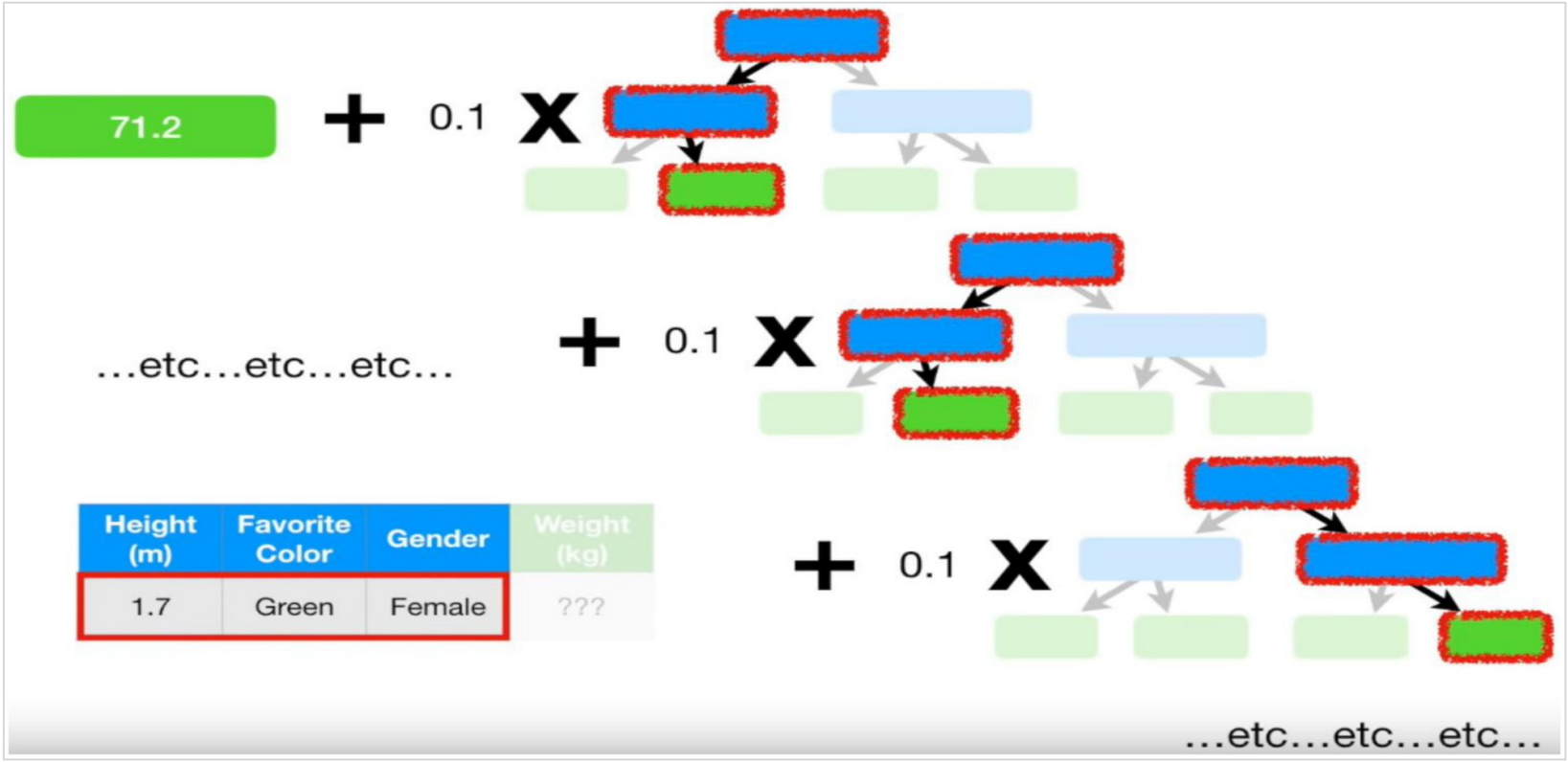


완성된 모델로 새 사람을 예측

키 1.7m · 초록 · 여성 → 초기값 71.2 + 모든 트리의 (학습률 × 잔차 예측)

결과

- 트리마다 해당 리프 값을 찾아 0.1을 곱해 더한다
- 원 강의 예제의 결과: 약 70kg
- GBM은 이런 방식으로 학습한다



보강 — 왜 "그래디언트" 부스팅인가

제곱 손실에서 손실의 음의 기울기(negative gradient)가 바로 잔차다

$$L = \tfrac{1}{2}(y - F)^2 \quad \Longrightarrow \quad -\frac{\partial L}{\partial F} = y - F \;=\; \text{residual}$$

함수 공간의 경사하강

- 보통의 경사하강: 파라미터 θ를 −∇L 방향으로 이동
- GBM: 예측값 $F(x_i)$ 자체를 −∂L/∂F 방향으로 이동
- 그 방향을 트리로 근사하고 학습률 ν만큼 걷는다

그래서 얻는 것

- 손실만 바꾸면 같은 틀로 분류 · 분위수 · 강건 회귀
- 몸무게 예제의 "잔차"는 특수한 경우 (제곱 손실)
- AdaBoost = 지수 손실을 쓴 특수한 경우

보강 — Friedman(2001)의 일반 알고리즘

손실 L을 정하면 아래 네 줄이 전부다. m = 1, …, M 반복

① 초기값

$$F_0(x) = \arg\min_{\gamma} \sum_{i=1}^{N} L(y_i, \gamma)$$

상수 하나 (제곱 손실이면 평균 71.2)

② 유사 잔차

$$r_{im} = -\left[\frac{\partial L\big(y_i, F(x_i)\big)}{\partial F(x_i)}\right]_{F=F_{m-1}}$$

손실의 음의 기울기

③ 리프 값

$$\gamma_{jm} = \arg\min_{\gamma} \sum_{x_i \in R_{jm}} L\big(y_i,\ F_{m-1}(x_i) + \gamma\big)$$

유사 잔차에 트리를 적합한 뒤 리프마다 최적 상수

④ 갱신

$$F_m(x) = F_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_{jm}\, \mathbf{1}(x \in R_{jm})$$

학습률 ν를 곱해 더한다

보강 — 손실함수가 바뀌면 유사 잔차가 바뀐다

GBM의 "다음 트리가 맞히는 대상"은 손실함수의 음의 기울기

Loss	$L(y, F)$	$-\partial L/\partial F$
Squared	$\frac{1}{2}(y-F)^2$	$y-F$
Absolute	$\lvert y-F\rvert$	$\mathrm{sign}(y-F)$
Huber	quadratic / linear	clipped residual
Log-loss	$-[y\log p+(1-y)\log(1-p)]$	$y-p$
Exponential	e^{-yF}	$y\,e^{-yF}$

실무 선택

- 제곱: 기본 회귀 (평균 예측)
- 절대 · Huber: 이상치에 강건 — 두꺼운 꼬리 수익률
- 로그 손실: 이진 분류 (부도 · 이탈)
- 분위수(quantile) 손실: VaR 같은 분위수 예측

보강 — 분류용 GBM은 로그 오즈를 쌓는다

분류에서도 회귀 트리를 쓴다. 트리가 더하는 대상은 확률이 아니라 로그 오즈(log-odds)

$$F_0 = \log \frac{\bar{p}}{1-\bar{p}}, \qquad p_i = \frac{1}{1+e^{-F(x_i)}}, \qquad r_{im} = y_i - p_i$$

절차

- 초기값 = 전체 양성 비율의 로그 오즈
- 유사 잔차 = 관측값(0/1) − 예측 확률
- 트리 적합 → 리프 값 계산 → ν배 해서 로그 오즈에 더함

리프 값 (로그 손실, 뉴턴 한 걸음)

$$\gamma_{jm} = \frac{\sum_{x_i \in R_{jm}} r_{im}}{\sum_{x_i \in R_{jm}} p_i(1-p_i)}$$

XGBoost는 이 "기울기 / 2차 미분" 구조를 모든 손실로 일반화했다

보강 — GBM 과대적합을 막는 다섯 가지 손잡이

하이퍼파라미터 (scikit-learn)	역할	요령
learning_rate (ν)	한 트리의 기여를 축소	0.01~0.1, 낮출수록 트리 수 ↑
n_estimators	최대 트리 수	n_iter_no_change로 조기 종료
max_depth · max_leaf_nodes	트리 하나의 복잡도 = 상호작용 차수	3~8 수준 (기본 depth 3)
subsample	트리마다 행 일부만 사용	0.5~0.8 → Stochastic GB (Friedman 2002)
min_samples_leaf	리프 최소 샘플	잡음 리프 방지

subsample < 1이면 OOB 개선량(oob_improvement_)도 볼 수 있다 — 2부의 OOB 발상이 부스팅에도 쓰인다

GBM을 발전시킨 모델 — XGBoost · LightGBM · CatBoost

원 강의: "더 빠르고 성능이 좋은 모델들이라고 보면 된다" → 무엇이 다른지 한 장으로

모델	핵심 아이디어	강점
XGBoost (2014, 논문 2016)	2차 테일러 근사 + 정규화 목적함수 · 결측 기본 방향 학습	정확도 · 병렬화, Kaggle 표준
LightGBM (2017)	히스토그램 분할 · 리프 우선(leaf-wise) 성장 · GOSS · EFB	대용량 데이터 속도
CatBoost (2017)	ordered boosting · 범주형 target statistics · 대칭 트리	범주형 많은 데이터, 기본값 성능
HistGradientBoosting	LightGBM 방식의 scikit-learn 구현 · NaN · 범주형 기본 지원	수만 건 이상이면 GBM보다 훨씬 빠름

다음 강의(고급 부스팅)에서 세 모델의 목적함수 · 분할 · 정규화를 자세히 다룬다

케이스 — 부스팅 트리로 만든 신용카드 연체 예측

Khandani · Kim · Lo (2010), Journal of Banking & Finance

2005.1–2009.4

대형 상업은행 고객의 거래 · 신용정보 데이터

85%

예측 연체율과 실제 연체율 회귀의 R^2

6~25%

추정 비용 절감 (총손실 대비)

무엇을 했나

- CART 계열 트리에 부스팅을 적용한 비선형 모델
- 카드 거래 내역 + 신용평가사 데이터를 결합
- 금융위기 직전 · 도중의 연체 급증을 표본외 예측

강의와의 연결

- 트리: 변수 간 상호작용을 자동으로 잡는다
- 부스팅: 약한 트리를 쌓아 편향을 줄인다
- 한도 축소 결정 → 손실 절감으로 연결

06

6부 · 종합

종합 — 네 모델을 한 표로, 그리고 현장으로

비교표 · 자산가격 예측 사례 · 최신 동향 · 실무 체크리스트 · 토론

네 모델 한눈에 비교

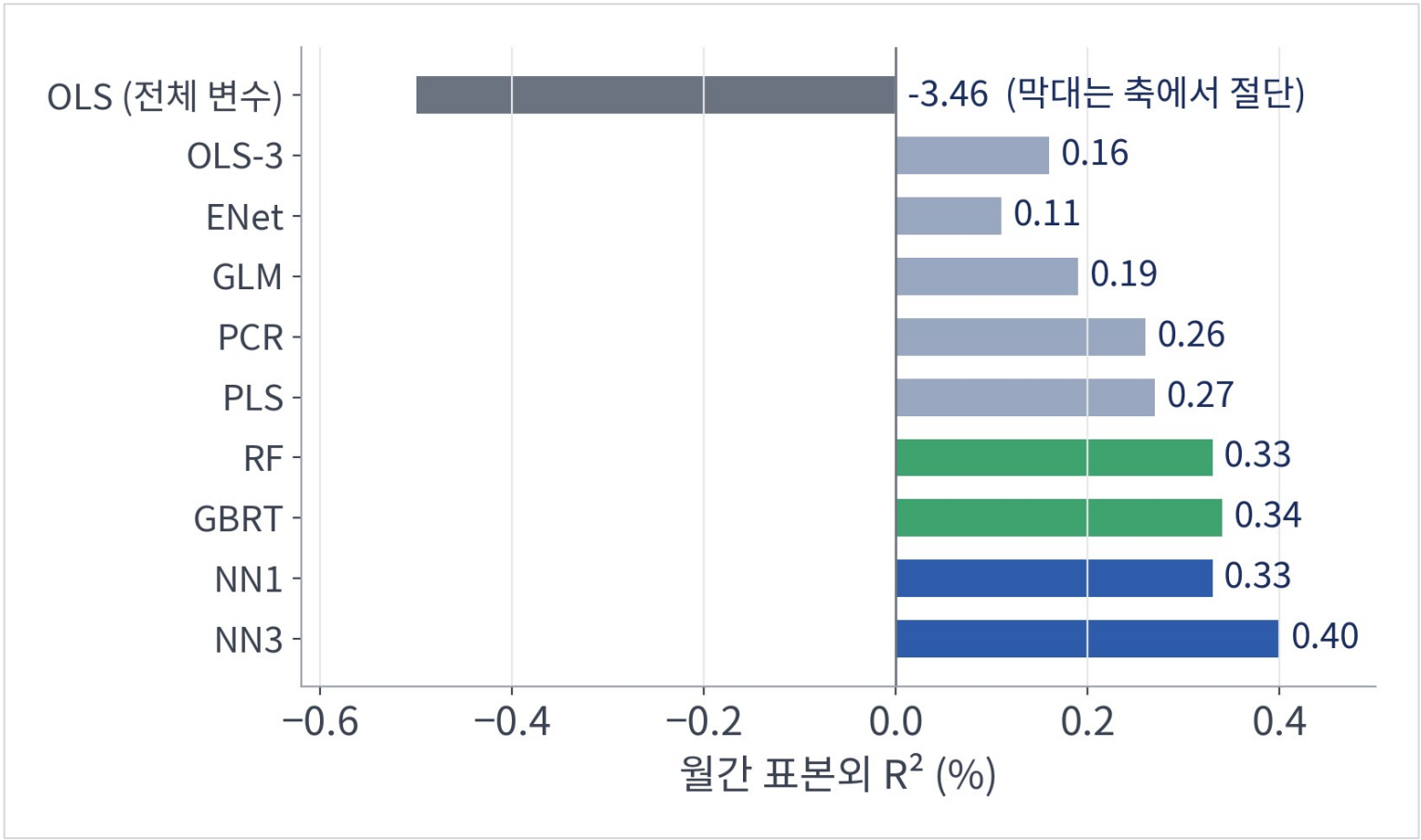
	의사결정트리	랜덤 포레스트	AdaBoost	그래디언트 부스팅
학습 방식	단일 트리	병렬 · 독립	순차 · 가중치	순차 · 잔차(기울기)
주로 줄이는 오차	—	분산	편향	편향
기본 학습기	깊은 트리	깊은 트리	stump	얕은 회귀 트리
결합	—	평균 · 다수결	α 가중 투표	ν × 합
핵심 손잡이	max_depth	n_estimators · max_features	n_estimators · learning_rate	learning_rate · n_estimators · depth
약점	과대적합 · 불안정	해석 어려움 · 외삽 불가	잡음 · 이상치에 민감	튜닝 필요 · 순차라 느림

케이스 — 트리 앙상블로 주식 수익률을 예측하면

Gu · Kelly · Xiu (2020, RFS): 미국 주식 약 3만 종목 · 1957~2016 · 기업 특성 94개 + 거시 · 산업 변수

핵심 발견

- 트리(RF · GBRT)와 신경망이 선형 모델을 이긴다
- 이유: 예측변수 간 상호작용
- 주요 신호: 모멘텀 · 유동성 · 변동성
- 월간 R^2 0.3~0.4%도 포트폴리오로는 큰 차이



최신 동향 — 표 데이터에서 트리 앙상블의 위치

이미지 · 텍스트는 딥러닝이 지배하지만, 표(tabular) 데이터에서는 GBDT가 오랫동안 기준선이었다

여전히 강한 이유

- Grinsztajn 외(NeurIPS 2022): 중간 규모 표 데이터 45종에서 트리 계열이 신경망보다 우세
- 불규칙한 함수 · 쓸모없는 특성에 강건
- 전처리(스케일링) 거의 불필요, 결측 · 범주형 기본 지원

새로운 경쟁자

- TabPFN v2 (Nature, 2025): 사전학습 트랜스포머로 소규모(약 1만 행 이하) 표 데이터에서 튜닝한 GBDT와 경쟁
- TabArena 등 벤치마크에서 순위 경쟁 중
- 실무 결론: GBDT를 기준선으로 두고 새 모델과 비교

금융 데이터에 적용할 때 체크리스트

시간 순서 지키기

무작위 K-fold 대신 시계열 분할 · 퍼징(purging)과 엠바고로 누수를 막는다.

타깃을 정상화

트리는 외삽을 못 한다. 가격 레벨 대신 수익률 · 변화율 · 순위를 예측한다.

결측을 정보로

결측 패턴이 체계적이다. 대체는 폴드 안에서, 결측 지시변수도 검토한다.

중요도 해석 주의

MDI는 연속형 · 상관 특성에 편향. permutation · SHAP으로 교차 확인한다.

토론 질문

Q1	루트 후보의 지니 차이가 0.004(0.364 vs 0.360)에 불과했다. 이것이 단일 트리의 불안정성과 어떤 관계인가?	**1부 · 분산**
Q2	RF에서 max_features를 1로 줄이면 트리 간 상관과 개별 트리 정확도는 각각 어떻게 변하나?	**2부 · 상관**
Q3	결측이 "실적 나쁜 기업의 공시 지연"처럼 MNAR일 때 근접도 대체만으로 충분한가?	**3부 · 결측**
Q4	AdaBoost와 GBM(학습률 0.1) 중 레이블 잡음이 많은 부도 데이터에 더 강건한 쪽은? 이유는?	**4·5부 · 손실**

종합

1

트리: 불순도를 가장 크게 줄이는 질문을 탐욕적으로 고른다

Gini · 엔트로피 · 가중평균 · 멈춤 규칙. 깊게 키우면 과대적합 → 깊이 · 가지치기로 제어.

2

RF: 부트스트랩 + 특성 무작위로 트리를 다르게 만들어 평균한다

평균해도 트리 간 상관만큼의 분산은 남는다. OOB로 검증하고, 근접도로 결측을 채운다.

3

부스팅: 앞 모델의 실수를 이어서 고친다

AdaBoost는 가중치를, GBM은 음의 기울기(잔차)를. 학습률 × 트리 수의 균형이 핵심.

다음: XGBoost · LightGBM · CatBoost — 같은 틀, 더 정교한 목적함수와 분할

참고문헌 · 그림 출처

- Breiman, Friedman, Olshen, Stone (1984). Classification and Regression Trees.
- Breiman (1996). Bagging predictors. Machine Learning 24. / Breiman (2001). Random forests. Machine Learning 45.
- Freund & Schapire (1997). A decision-theoretic generalization of on-line learning… JCSS 55.
- Friedman, Hastie, Tibshirani (2000). Additive logistic regression. Annals of Statistics 28.
- Friedman (2001). Greedy function approximation: a gradient boosting machine. Annals of Statistics 29. / (2002) Stochastic gradient boosting.
- Viola & Jones (2001). Rapid object detection using a boosted cascade of simple features. CVPR.
- Khandani, Kim, Lo (2010). Consumer credit-risk models via machine-learning algorithms. JBF 34. / Gu, Kelly, Xiu (2020). Empirical asset pricing via machine learning. RFS 33.
- Bryzgalova, Lerner, Lettau, Pelger (2025). Missing financial data. RFS 38. / Grinsztajn et al. (2022). NeurIPS. / Hollmann et al. (2025). TabPFN. Nature.
- scikit-learn 문서 (1.3~1.9 릴리스 노트: 트리 계열 결측 지원) · 그림: StatQuest(Josh Starmer) 강의, Müller & Guido 『파이썬 라이브러리를 활용한 머신러닝』, Géron 『핸즈온 머신러닝』 예제